LIC. EN **INTELIGENCIA ARTIFICIAL — LIC-EN-IA**



MATERIA

Fundamentos de la Inteligencia Artificial

FIA · PRIMER CUATRIMESTRE

UBP UNIVERSIDAD Blas Pascal

CONTACTO
0810 1223 3827
informes@ubp.edu.ar

WEB Y REDES
www.**ubp.edu.ar**

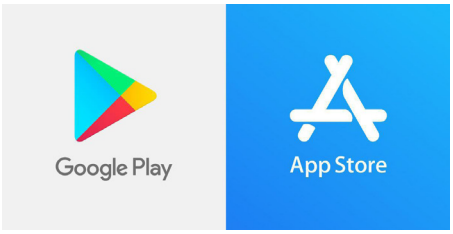
Códigos QR

Estimado Alumno:

Esta materia puede contener **Códigos QR** para agilizar el acceso a los contenidos audiovisuales que el docente ha desarrollado especialmente para Usted.

Para poder visualizar los videos utilice el lector de código QR que trae por defecto su teléfono o dispositivo móvil. En caso de no disponer de uno, puede descargarlo desde *Google Play* si su sistema operativo corresponde a Android o desde la *App Store* si utiliza IOS.





Una vez instalado el software en su dispositivo móvil:

1) Abra la aplicación
2) Apunte con la cámara de su dispositivo hacia los códigos QR que encuentre en el interior de su materia.



De esta manera estará visualizando los videos que el docente ha desarrollado y/o seleccionado especialmente para usted.

Índice

Bienvenida 4

Propósito 4

Objetivos 6

Competencias 6

Contenidos mínimos 8

Contenidos 8

Bibliografía 10

Planificación 10

Metodología 11

Evaluación 11

Mapa conceptual 13

Módulos

- *Módulo 1* 14
- *Módulo 2* 34
- *Módulo 3* 55
- *Módulo 4* 86
- *Módulo 5* 123

Importante

Lecturas básicas y complementarias disponibles desde plataforma.

Impresión total del documento141 páginas

Bienvenida

Bienvenidos a la materia:

"Fundamentos de la Inteligencia Artificial"

Propósito

Bienvenidos a *Fundamentos de Inteligencia Artificial*, asignatura que marca el primer acercamiento al extenso y cambiante mundo de la IA dentro de la carrera Licenciatura en Inteligencia Artificial. En este espacio proponemos acompañarlo en la construcción de una base sólida que permita comprender qué es la inteligencia artificial, por qué constituye un campo central en la actualidad y cómo se relaciona con los desafíos tecnológicos y sociales contemporáneos.

A lo largo del cursado recorreremos cinco módulos complementarios entre sí, que presentan la evolución histórica, los fundamentos teóricos y las metodologías que sustentan a los sistemas inteligentes actuales.

En el **Módulo 1**, comenzaremos por los conceptos iniciales: qué entendemos por inteligencia artificial, cuáles son sus enfoques clásicos, qué diferencia a la inteligencia humana de la racional, y cómo evolucionó la disciplina desde sus orígenes hasta el panorama contemporáneo. Luego, el **Módulo 2** nos permitirá ingresar al corazón conceptual de la materia a través del estudio de los agentes inteligentes. Analizaremos cómo perciben, razonan y actúan, qué elementos componen su diseño y cómo distintas configuraciones de entornos condicionan su comportamiento. Este módulo es fundamental porque introduce el marco teórico que sostiene toda la IA moderna.

En el **Módulo 3** avanzaremos hacia un eje central del curso: la resolución de problemas mediante búsqueda. Veremos cómo un agente puede formular un problema, representar el espacio de estados, explorar alternativas y seleccionar rutas que lo acerquen a su objetivo. Estudiaremos problemas clásicos y reales, junto con estrategias de búsqueda no informada e informada, y comprenderemos por qué estas técnicas constituyen la base de muchos sistemas de planificación.

El **Módulo 4** marca un cambio en la forma de razonar de los agentes: pasamos de la acción guiada por procedimientos a la acción guiada por conocimiento. Aquí abordaremos la representación simbólica, los lenguajes lógicos y los mecanismos de inferencia que permiten a un agente deducir nuevas verdades y tomar decisiones fundamentadas. Así, estaremos listos para entender cómo, en el aprendizaje automático, el conocimiento puede extraerse de los datos en lugar de estar preprogramado.

Finalmente, en el **Módulo 5** nos introduciremos en el paradigma dominante actual de la IA: el aprendizaje automático (Machine Learning). Trasladaremos el foco de las reglas

predefinidas hacia sistemas capaces de aprender patrones a partir de datos, explorando técnicas básicas de clasificación y regresión.

Cada módulo aporta una pieza indispensable para comprender la disciplina: la historia aporta contexto, los agentes brindan el marco estructural, la búsqueda permite planificar, la lógica posibilita razonar y el aprendizaje abre la puerta a sistemas capaces de mejorar con la experiencia. En conjunto, esta asignatura busca que ustedes adquieran una visión integral y crítica de la inteligencia artificial, desarrollando tanto los fundamentos conceptuales como las herramientas iniciales para avanzar con solidez en el resto de la carrera.

¡Empecemos!

Objetivos

- Reconocer los hitos más importantes en la evolución de la Inteligencia Artificial, con el fin de comprender cómo los avances actuales se relacionan con sus orígenes y los desafíos pasados.

- Analizar los conceptos de agente y entorno, junto con los distintos tipos de agentes inteligentes, para entender cómo se modelan los procesos de decisión en sistemas artificiales.

- Analizar los métodos de resolución de problemas mediante búsqueda y sus estrategias fundamentales para comprender cómo los sistemas inteligentes planifican, exploran alternativas y alcanzan soluciones óptimas o aproximadas.

- Aplicar diferentes enfoques de representación del conocimiento con el objetivo de entender cómo las máquinas pueden organizar información y razonar sobre ella en contextos concretos.

- Explorar los fundamentos del aprendizaje automático y sus principales paradigmas, a fin de valorar cómo los sistemas pueden mejorar su rendimiento a partir de la experiencia y los datos.

Competencias

Relación de la asignatura con las competencias de egreso

En la tabla siguiente se establece la relación de la asignatura con las competencias de egreso: Específicas, Genéricas Tecnológicas, Sociales, Políticas y Actitudinales de la carrera. Se incluyen las competencias de egreso a las que tributa su nivel de aporte (ninguno, bajo, medio y alto)

Competencias Genéricas Tecnológicas (CG)	Nivel
CG1. Identificación, formulación y resolución de problemas complejos mediante técnicas de ciencia de datos y desarrollo de sistemas de IA, empleando modelos matemáticos, algorítmicos y estadísticos para proponer soluciones basadas en datos.	Media
CG2. Concepción, diseño y desarrollo de proyectos de ciencia de datos y sistemas de inteligencia artificial, integrando metodologías de ingeniería, buenas prácticas de programación y principios éticos aplicados al uso de datos.	Media
CG3. Gestión, planificación, ejecución y control de proyectos vinculados con la construcción, entrenamiento, validación y despliegue de modelos de IA y soluciones de ciencia de datos, asegurando eficiencia, trazabilidad y colaboración interdisciplinaria.	Media
CG4. Utilización de técnicas, lenguajes y herramientas especializadas de análisis de datos, aprendizaje automático y desarrollo de modelos de IA, seleccionando las más adecuadas según la naturaleza del problema y la disponibilidad de recursos.	Baja

CG5. Generación de desarrollos tecnológicos y/o innovaciones basadas en inteligencia artificial y ciencia de datos, proponiendo soluciones originales y escalables que respondan a necesidades reales de organizaciones y comunidades.	Baja
Competencias Sociales, Políticas y Actitudinales (CSPA) -Adaptadas a IA	**Nivel**
CSPA6. Desempeñarse en equipos interdisciplinarios para diseñar, desarrollar y evaluar soluciones de IA, coordinando tareas, roles y acuerdos de trabajo.	Media
CSPA7. Comunicar de forma clara y efectiva decisiones técnicas vinculadas a IA (datos, modelos, métricas, riesgos y límites), adaptando el mensaje a públicos técnicos y no técnicos.	Baja
CSPA8. Actuar con ética y responsabilidad en el desarrollo y uso de sistemas de IA, considerando sesgos, privacidad, seguridad, transparencia y rendición de cuentas.	Baja
CSPA9. Evaluar y actuar frente al impacto social de soluciones de IA en contextos globales y locales, reconociendo efectos en personas, organizaciones y comunidades.	Baja
CSPA10. Sostener el aprendizaje continuo para actualizar conocimientos en IA, ciencia de datos y tecnologías asociadas, incorporando nuevas herramientas y buenas prácticas.	Baja
CSPA11. Promover la acción emprendedora mediante la identificación de oportunidades de innovación con IA, proponiendo soluciones viables con enfoque en valor, impacto y sostenibilidad.	Baja
Competencias Específicas (CE)[1]	**Nivel**
CE 1.1 CD Especificación, proyección y desarrollo de modelos de ciencia de datos y sistemas basados en IA, considerando requerimientos técnicos, éticos y de disponibilidad de datos.	Baja
CE 1.2 CD Especificación, proyección y desarrollo de infraestructuras y sistemas de comunicación de datos orientados a la adquisición, procesamiento y despliegue de modelos de IA.	Baja
CE 1.3 IA Especificación, proyección y desarrollo de software para análisis, entrenamiento, validación y operación de modelos de inteligencia artificial, integrando algoritmos y estructuras de datos avanzadas.	Baja
CE 2.1 SI–IA Proyección y dirección de procesos vinculados a la seguridad de datos, resguardo de modelos y protección de información sensible empleada en sistemas de IA y ciencia de datos.	Baja
CE 3.1 M–IA Establecimiento de métricas, normas y estándares de calidad para modelos analíticos, procesos de minería de datos y sistemas de inteligencia artificial, garantizando su confiabilidad y desempeño.	Baja
CE 4.1 C–IA Certificación del funcionamiento, evaluación de riesgos, validación y condición de uso de modelos de IA, pipelines de ciencia de datos y sistemas de software asociados, incluyendo aspectos de transparencia, auditoría y robustez.	Baja
CE 5.1 D–CD/IA Dirección y control de la implementación, operación y mantenimiento de soluciones de ciencia de datos y sistemas de inteligencia artificial, incluyendo infraestructura, monitoreo, mejora continua, seguridad y aseguramiento de calidad.	Baja

1 Competencias específicas (CE). Resolución ME 1254/2018 y Estándares RedUNCI, septiembre 2018.

Resultados de Aprendizaje

Resultado	Descripción
RA1	Elaborar una línea de tiempo argumentada sobre la evolución de la Inteligencia Artificial, identificando hitos y relacionándolos con avances y desafíos actuales mediante ejemplos.
RA2	Modelar situaciones problema mediante el enfoque de agente–entorno (PEAS), clasificar el tipo de entorno y justificar el tipo de agente inteligente adecuado para la tarea planteada.
RA3	Formular problemas como búsqueda en espacios de estados (estado inicial, acciones, objetivo y costo) y aplicar estrategias de búsqueda, comparando su desempeño a partir de trazas y/o métricas básicas.
RA4	Representar conocimiento de un dominio acotado utilizando al menos dos enfoques (p. ej., reglas, lógica, redes semánticas u ontologías) y realizar inferencias para responder consultas, verificando consistencia en casos de prueba.
RA5	Implementar y evaluar un modelo básico de aprendizaje automático, seleccionando el paradigma adecuado según el problema, reportando métricas de desempeño e interpretando resultados para proponer mejoras.

Contenidos mínimos

- Historia y evolución de la IA: Conceptos iniciales, hitos y avances en la IA.
- Fundamentos teóricos de IA: Agentes inteligentes, entornos y modelos de comportamiento.
- Algoritmos básicos de búsqueda: Búsqueda no informada e informada.
- Representación del conocimiento: Lógica proporcional y lógica de predicados.
- Aprendizaje Automático básico: Introducción a técnicas de clasificación y regresión

Contenidos

MÓDULO 1: INTRODUCCIÓN A LA INTELIGENCIA ARTIFICIAL

Unidad 1: Conceptos iniciales y definición de la IA

Concepto y enfoques de la IA. Inteligencia racional y humana. Características fundamentales: percepción, razonamiento, aprendizaje e interacción. Enfoques clásicos de la IA. IA débil vs. IA fuerte. Objetivos y alcances de la IA.

Unidad 2: Historia y evolución de la IA

Orígenes de la disciplina. Test de Turing y aportes de la cibernética. Conferencia de Dartmouth. Etapas de auge e "inviernos" de la IA. Sistemas expertos. Big Data y aprendizaje profundo. Estado del arte: modelos generativos, expansión contemporánea y desafíos éticos y sociales de la IA.

MÓDULO 2: AGENTES INTELIGENTES

Unidad 3: Agentes y racionalidad
Definición y componentes de un agente. Función del agente y programa del agente. Ciclo percepción–acción. Entorno. Concepto de racionalidad y medida de rendimiento. Racionalidad limitada, autonomía y aprendizaje.
Unidad 4: Tipos de agentes y entornos
Esquema REAS (Rendimiento, Entorno, Actuadores, Sensores). Clasificación de entornos: observables, deterministas, episódicos, estáticos, discretos y conocidos. Relación entre tipo de entorno y diseño del agente. Estructura general de un agente. Agentes reactivos simples y basados en modelo. Agentes basados en objetivos, utilidad y aprendizaje.

MÓDULO 3: RESOLUCIÓN DE PROBLEMAS

Unidad 5: Formulación de problemas
Problema de búsqueda: estado inicial, espacio de estados, objetivos, acciones. Abstracción, propiedades. Problemas estandarizados. Problemas reales.
Unidad 6: Algoritmos de búsqueda no informada
Árbol de búsqueda. Criterios de rendimiento: completitud, optimalidad, complejidad temporal/espacial. Búsqueda en anchura. Búsqueda de costo uniforme. Búsqueda en profundidad. Búsqueda con profundidad limitada. Búsqueda con profundización iterativa.
Unidad 7: Algoritmos de búsqueda informada y heurística
Función heurística. Búsqueda voraz del mejor primero. Búsqueda A*. Heurística admisible. Heurística consistente.

MÓDULO 4: REPRESENTACIÓN DEL CONOCIMIENTO Y RAZONAMIENTO LÓGICO

Unidad 8: Representación simbólica y conocimiento basado en lógica
Agente basado en conocimiento. Base de conocimiento. Axioma. Inferencia. Razonamiento lógico. Sintaxis. Semántica. Modelo.
Unidad 9: Lógica proposicional
Proposición. Conectivos lógicos. Implicación. Tablas de verdad. Validez. Satisfacibilidad. Modus Ponens. Resolución. Cláusulas de Horn. Encadenamiento.
Unidad 10: Lógica de primer orden
Predicado. Objetos. Relaciones. Funciones. Dominio. Interpretación. Igualdad. Variable. Cuantificadores. Razonamiento simbólico. Representación del conocimiento.

MÓDULO 5: INTRODUCCIÓN AL APRENDIZAJE AUTOMÁTICO

Unidad 11: Fundamentos del aprendizaje supervisado
Aprendizaje automático. Aprendizaje supervisado. Inducción. Clasificación. Regresión. Hipótesis. Espacio de hipótesis. Conjunto de entrenamiento. Conjunto de prueba. Sesgo. Varianza. Sobreajuste. Subajuste. Árbol de decisión. Entropía. Ganancia de información.
Unidad 12: Técnicas de clasificación y regresión
Modelo no paramétrico. Modelo k-NN. Clasificación en k-NN. Distancia Minkowski. Límite de decisión. Regresión en k-NN. Regresión no paramétrica. Regresión ponderada localmente. Normalización.

Bibliografía

Obligatoria

Material didáctico de producción institucional diseñado y desarrollado especialmente para cada módulo.

Complementaria

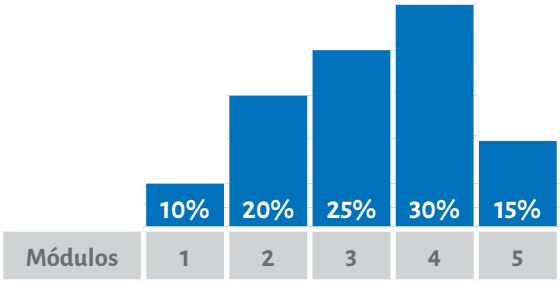
› Littman, M. L., Ajunwa, I., Berger, G., Boutilier, C., Currie, M., Doshi-Velez, F., Hadfield, G., Horowitz, M. C., Isbell, C., Kitano, H., Levy, K., Lyons, T., Mitchell, M., Shah, J., Sloman, S., Vallor, S., & Walsh, T. (2021). Gathering strength, gathering storms: The One Hundred Year Study on Artificial Intelligence (AI100) 2021 study panel report. Stanford University. http://ai100.stanford.edu/2021-report

› Maslej, N., Fattorini, L., Perrault, R., Gil, Y., Parli, V., Kariuki, N., Capstick, E., Reuel, A., Brynjolfsson, E., Etchemendy, J., Ligett, K., Lyons, T., Manyika, J., Niebles, J. C., Shoham, Y., Wald, R., Walsh, T., Hamrah, A., Santarlasci, L., Lotufo, J. B., Rome, A., Shi, A., & Oak, S. (2025). The AI Index 2025 annual report. AI Index Steering Committee, Institute for Human-Centered AI, Stanford University. https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf

› Turing, Alan M. (1950). Computing machinery and intelligence. Mind, 59(236), 433–460. https://doi.org/10.1093/mind/LIX.236.433

Planificación

Porcentaje estimativo por módulo según la cantidad y complejidad de contenidos y actividades:



Representación de porcentajes en semanas:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
M1	Módulo 2		✓	Módulo 3				Módulo 4					M5	✓

✓ Primera parte de la Evaluación Integradora
✓ Segunda parte de la Evaluación Integradora

Metodología

A) Estrategias de enseñanza: Este espacio curricular fue diseñado según lo previsto en el Sistema Institucional de Educación a Distancia (SIED) de la UBP (Resolución MECCYT 197/2019), las características de los destinatarios, las necesidades de esta disciplina, competencias que se espera promover en esta asignatura y el perfil del egresado.

En relación con lo anterior, para llevar a cabo la enseñanza se cuenta con:

- Contenidos y actividades de aprendizaje desarrollados en la plataforma virtual miUBP.
- Material bibliográfico especialmente seleccionado.
- Tutorías virtuales que implican diferentes espacios de interacción.

B) Materiales curriculares: Durante el cursado, se fomenta el estudio de los contenidos (presentados en diferentes soportes y lenguajes) que componen el programa, y la resolución de las actividades de aprendizaje. De esta forma, se introduce al alumno en los conceptos que se deben dominar en todo lo que respecta los fundamentos básicos de la Inteligencia Artificial. También, se invita a leer bibliografía que enriquece y profundiza dicho desarrollo. Por otra parte, se propone la resolución de actividades de aprendizaje tanto de carácter optativo como obligatorio que se encuentran debidamente identificadas en la plataforma. Algunas de ellas se centran en la apropiación de conceptos teóricos básicos de la asignatura y otras en la transferencia mediante un abordaje práctico a través del análisis de casos, ejercitaciones y situaciones problemáticas.

C) Modalidad de agrupamientos: En esta asignatura se proponen principalmente instancias de producción individual, en coherencia con los objetivos y contenidos a trabajar. De esta manera, el alumno puede desenvolverse en forma autónoma, reconociendo sus fortalezas y debilidades, a la vez que puede desarrollar sus propias estrategias de aprendizaje. Es importante destacar que para el estudio de la asignatura los alumnos pueden optar por agruparse libremente y resolver las actividades de aprendizaje planteadas.

D) Interacción: La comunicación con el docente tutor se realiza a través de los diferentes medios disponibles en la plataforma miUBP.

E) Formación práctica: En lo que se refiere a la formación práctica se hace foco en la resolución de problemas que implique el uso de los contenidos estudiados en la asignatura contextualizados en situaciones que tengan que ver con la carrera.

Evaluación

La metodología de evaluación propuesta se presenta conforme al Reglamento Académico General de la UBP.

Cada instrumento de evaluación contiene sus criterios de evaluación explicitados y el puntaje otorgado a cada consigna.

En cuanto a los criterios de acreditación, la escala cuantitativa utilizada para evaluar es del 1 al 100 y el puntaje mínimo que se requiere para la aprobación es de 50.

Regularidad:

Para alcanzar la condición de alumno regular usted deberá aprobar la Evaluación Integradora dentro de los plazos previstos para el cursado.

Si reprueba esta instancia podrá reelaborar la evaluación según los requerimientos del docente.

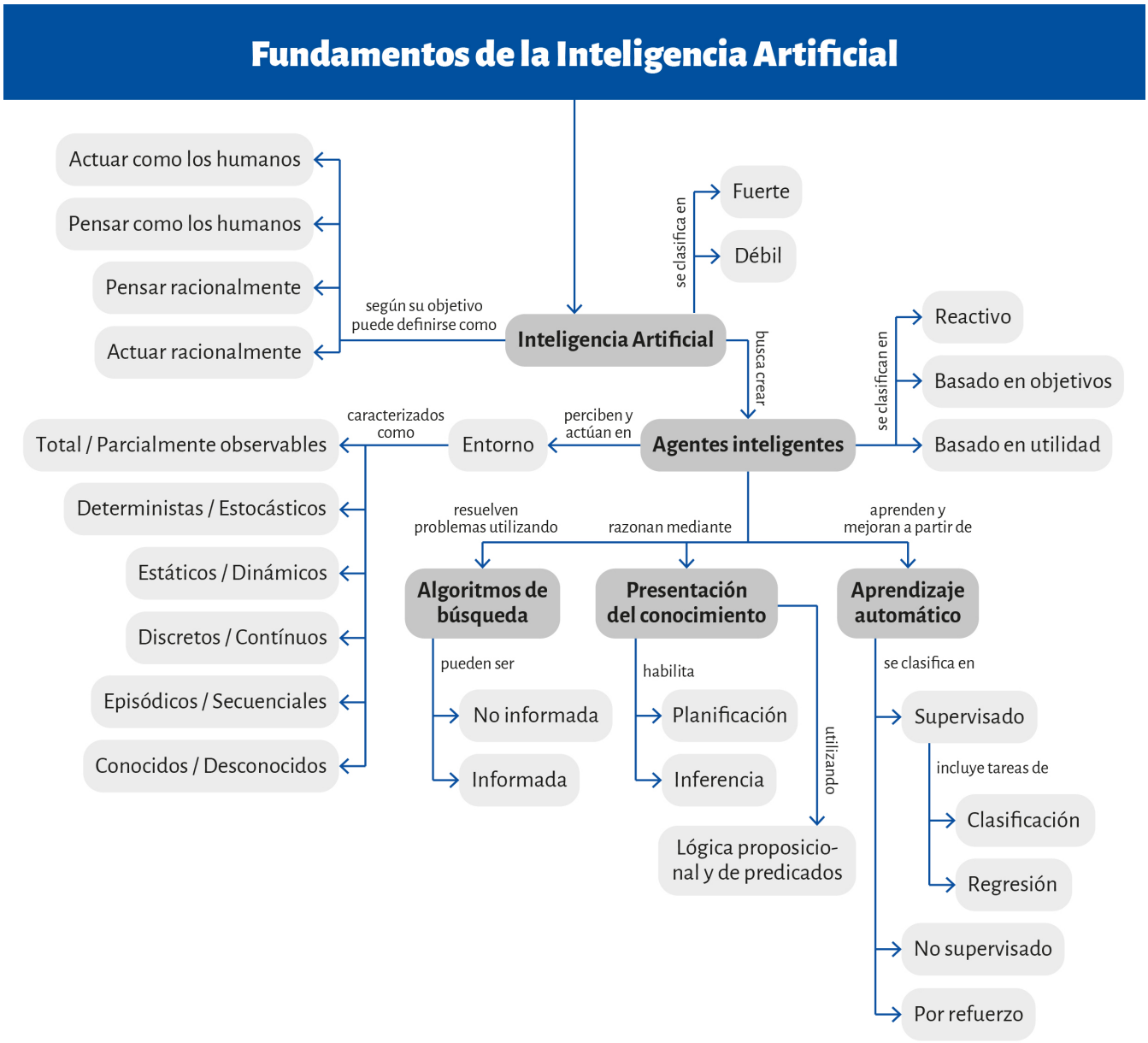
Examen Final:

Deberá rendir, en condición de regular, una evaluación final que se realizará a través de un examen escrito integrador, presencial, en el periodo que disponga la Universidad.

Mapa conceptual

Mapa conceptual animado

A continuación, lo/a invitamos a escuchar el recorrido por el mapa conceptual de la materia propuesto por el docente.



Módulo 1

Microobjetivos

- Reconocer las distintas definiciones de Inteligencia Artificial para comprender cómo se articulan las ideas de pensamiento humano, racionalidad y comportamiento inteligente en el desarrollo de la disciplina.
- Identificar los principales objetivos de la Inteligencia Artificial para analizar su papel en la automatización de tareas, la ampliación de capacidades humanas y la resolución de problemas complejos.
- Distinguir entre IA débil e IA fuerte para valorar sus diferencias conceptuales, sus alcances actuales y las implicancias éticas y filosóficas de cada enfoque.
- Comprender los hitos históricos más relevantes en la evolución de la Inteligencia Artificial para establecer conexiones entre los avances técnicos y los cambios de paradigma que definieron su desarrollo.
- Integrar los conocimientos teóricos, históricos y éticos desarrollados en el módulo para sentar una base conceptual sólida que permita avanzar hacia el estudio de los agentes inteligentes en el Módulo 2.

Contenidos

INTRODUCCIÓN A LA INTELIGENCIA ARTIFICIAL



Fuente: https://www.freepik.es/

Contenidos

Es posible que cuando escuche hablar de IA se le vengan a la mente imágenes de robots que piensan como humanos, de asistentes virtuales que responden preguntas, de autos que se manejan solos o de algoritmos que recomiendan películas y series. Todo eso for-

ma parte de la IA, pero también lo es mucho más, desde sistemas que detectan fraudes bancarios en fracciones de segundo hasta modelos que ayudan a diagnosticar enfermedades.

En este primer módulo el objetivo es construir juntos un mapa conceptual de la IA, entender qué es, de dónde viene, cómo evolucionó y cuáles son los dilemas que plantea hoy en día.

¡Empecemos!

Unidad 1: *Conceptos iniciales y definición de la IA*

La inteligencia artificial (IA) es una de las áreas más dinámicas e influyentes de la ciencia contemporánea. A lo largo de las últimas décadas, su desarrollo ha transformado múltiples ámbitos del conocimiento, desde la informática y la robótica hasta la medicina, la educación y la economía. Pero antes de profundizar en sus aplicaciones, resulta fundamental comprender qué entendemos por "inteligencia artificial" y qué significa realmente decir que una máquina puede ser inteligente.

Las personas solemos considerar la inteligencia como una de nuestras características más distintivas: la capacidad de razonar, aprender, comunicarnos, tomar decisiones y adaptarnos a contextos cambiantes. Desde tiempos antiguos, la humanidad ha intentado explicar cómo funciona la mente y cómo sería posible reproducir, mediante artefactos, algunos de esos procesos.



La inteligencia artificial surge justamente de ese intento por comprender y construir sistemas capaces de percibir su entorno, razonar sobre lo que perciben y actuar en consecuencia para alcanzar un objetivo. En este sentido, la IA no se limita a imitar el pensamiento humano, sino que busca desarrollar mecanismos racionales de acción y decisión que permitan a las máquinas desenvolverse con autonomía y aprender.

Veamos en detalle a qué hacemos referencia con cada una de estas características:

- **Percibir**: comprender e interpretar el mundo que los rodea a través de datos, de manera semejante a como nosotros utilizamos nuestros sentidos. Esto incluye "ver" con cámaras (visión por computadora), "oír" con micrófonos (reconocimiento de voz) y leer textos (procesamiento del lenguaje natural).

- **Razonar**: analizar información, extraer conclusiones lógicas, resolver problemas y tomar decisiones basadas en los datos disponibles, en lugar de limitarse a seguir un guion rígido y pre-escrito.

- **Aprender**: mejorar su desempeño con el tiempo al reconocer patrones en los datos y adaptarse a nueva información. Este es el núcleo del aprendizaje automático o Machine Learning (que veremos más adelante en esta materia), en el que una IA no está programada explícitamente para cada escenario, sino que aprende a partir de ejemplos y de la experiencia.

- **Interactuar**: comunicarse de manera efectiva, ya sea a través del lenguaje (como un chatbot), de acciones (como una aspiradora robot que navega en una habitación) o incluso generando nuevo contenido (como crear una imagen o redactar un párrafo).

Una analogía simple y cercana es una aplicación de navegación como Google Maps que no se limita a almacenar un mapa estático, sino que percibe su ubicación actual y los datos de tráfico en tiempo real, razona cuál es la ruta más rápida considerando distancia y congestión, aprende a partir de los datos colectados de sus usuarios para predecir patrones de tráfico futuros, e interactúa dando instrucciones paso a paso. Esto es la IA en acción.

Multimedia

Miremos este short de YouTube sobre qué es realmente la IA

https://www.youtube.com/shorts/J9JQLg4WBls

A lo largo de su historia, la IA se ha abordado desde distintas perspectivas, dependiendo de cómo se entienda la noción de "inteligencia" y del objetivo que se persiga al construir sistemas inteligentes. Estas perspectivas pueden agruparse según dos ejes:

1. Humanidad vs. racionalidad: si se intenta reproducir la forma en que piensan o actúan los humanos, o si se busca diseñar agentes que actúen de manera óptima, independientemente de cómo lo haría una persona.
2. Pensamiento vs. comportamiento: si se centra el interés en los procesos internos de razonamiento o en las acciones observables que un agente realiza en su entorno.

De la combinación de ambos ejes surgen cuatro enfoques fundamentales:

1. Actuar como los humanos: el enfoque del comportamiento inteligente

Uno de los primeros intentos por definir la inteligencia artificial se basó en la imitación del comportamiento humano. La idea central es que una máquina puede considerarse inteligente si logra actuar de un modo que resulte indistinguible del de una persona.

Este principio fue formalizado a mediados del siglo XX por Alan Turing mediante una propuesta experimental conocida como la prueba de Turing. En este experimento, una persona mantiene una conversación escrita con dos interlocutores ocultos: uno humano y otro artificial. Si, al finalizar el diálogo, no puede determinar con certeza cuál de los dos es la máquina, se concluye que el sistema ha mostrado un comportamiento inteligente.

Para lograrlo, una máquina debe combinar múltiples capacidades:

- Procesar lenguaje natural, para comunicarse de forma fluida.
- Representar conocimiento, para almacenar y utilizar información sobre el mundo.
- Razonar y resolver problemas, para responder con coherencia y creatividad.
- Aprender de la experiencia, para adaptarse a nuevas situaciones.

En versiones más completas de esta idea —a veces llamadas "prueba total"— se incluye además la percepción visual y auditiva y la capacidad de interactuar físicamente con el entorno, lo que implica incorporar visión por computadora, reconocimiento del habla y robótica.

Este enfoque tiene una intención clara: valorar la inteligencia por sus resultados, sin importar cómo se alcanzan internamente. Así como los ingenieros aeronáuticos aprendieron a volar sin imitar a las aves, la IA puede aspirar a comportamientos inteligentes sin replicar la mente humana paso a paso. Lo importante es el desempeño observable, es decir, que el sistema actúe de manera coherente, comprensible y eficaz ante los estímulos que recibe.

2. Pensar como los humanos: el enfoque cognitivo

Mientras el enfoque anterior se centra en lo que la máquina *hace*, este se interesa por cómo lo hace. Aquí el objetivo es construir sistemas que no solo obtengan resultados correctos, sino que además reproduzcan los procesos mentales que las personas utilizan para razonar y resolver problemas.

Esta perspectiva está estrechamente vinculada con la psicología cognitiva y las neurociencias, que estudian cómo el cerebro percibe, recuerda, asocia ideas, aprende y toma decisiones.

Para investigar estos procesos, se emplean métodos como:

- La introspección, que consiste en analizar la propia experiencia mental.
- Los experimentos psicológicos, que observan el comportamiento ante distintas tareas.
- Las técnicas de neuroimagen, que permiten estudiar la actividad cerebral mientras una persona piensa o actúa.

Una vez formuladas teorías suficientemente precisas sobre cómo pensamos, es posible convertirlas en modelos computacionales que simulen ese razonamiento. Si el sistema responde a los problemas de manera similar a una persona —en el orden, la forma y el tiempo de sus razonamientos—, se considera que refleja aspectos del pensamiento humano.

Los primeros programas de resolución de problemas intentaron imitar los pasos mentales que un individuo seguiría para encontrar una solución lógica o matemática. Por ejemplo, el "Solucionador General de Problemas" o "GPS" de Allen Newell y Herbert Simon, cuyo objetivo era imitar las capacidades humanas para resolver problemas, no solo para encontrar una solución correcta, sino para seguir los mismos pasos de razonamiento que seguiría una persona. Utilizaba una técnica llamada análisis de medios y fines para encontrar una solución. Esto implicaba definir el problema en términos de objetivos y subobjetivos, y utilizar "operadores" para transformar el problema desde su estado inicial al estado objetivo.

Estas experiencias fueron la base de la llamada ciencia cognitiva, que combina conocimientos de la informática, la psicología, la lingüística y la biología para comprender cómo surge la inteligencia.

No obstante, este enfoque presenta desafíos: el pensamiento humano es extremadamente complejo y no siempre racional o predecible. Por ello, aunque la simulación de procesos mentales ha contribuido a entender mejor la mente, su aplicación práctica en sistemas artificiales requiere simplificaciones y abstracciones que la alejan de la realidad psicológica.

3. Pensar racionalmente: el enfoque de las leyes del pensamiento

La tercera perspectiva se origina mucho antes de la era digital, en la tradición filosófica que buscaba establecer las reglas del razonamiento correcto, es decir, procesos de razo-

namiento irrefutables. Desde la antigüedad, se intentó formalizar las formas válidas de pensamiento mediante la lógica, un sistema que permite derivar conclusiones verdaderas a partir de premisas verdaderas.

Bajo este enfoque, un sistema inteligente es aquel que piensa de acuerdo con los principios de la razón, utilizando procedimientos lógicos que garantizan conclusiones coherentes. Por ejemplo, si un sistema sabe que *todos los humanos son mortales* y que *Sócrates es humano*, puede deducir lógicamente que *Sócrates es mortal*. Estas reglas, conocidas como *silogismos*, sentaron las bases de la inferencia racional. Veremos más en profundidad este tema en el 4to módulo.

La lógica moderna permitió extender esta idea a la representación del conocimiento sobre objetos, relaciones y propiedades del mundo. Los sistemas basados en este principio pueden, en teoría, resolver cualquier problema que pueda expresarse en un lenguaje lógico formal.

Sin embargo, este enfoque también presenta un límite importante, ya que el razonamiento lógico clásico supone que la información disponible es completa y certera, lo cual rara vez ocurre en el mundo real. En la vida cotidiana, la información es ambigua o incierta, y las decisiones deben tomarse aun sin tener garantías absolutas. Por eso, para lograr comportamientos realmente inteligentes, las máquinas deben combinar la lógica con métodos que gestionen la incertidumbre y la probabilidad, integrando la racionalidad con la flexibilidad.

4. Actuar racionalmente: el enfoque de los agentes inteligentes

El cuarto enfoque, y el más influyente en la inteligencia artificial contemporánea, propone definir la inteligencia no por lo que una máquina piensa, sino por cómo actúa frente a su entorno. Según esta visión, un sistema inteligente es aquel que actúa racionalmente, es decir, que elige las acciones que aumentan la probabilidad de alcanzar sus metas.

Esta definición introduce un concepto clave: el de agente racional. Profundizaremos más sobre qué es un agente y particularmente uno racional en el siguiente módulo, por ahora solo lo describiremos como cualquier entidad que percibe su entorno mediante sensores y actúa sobre él a través de mecanismos de salida o actuadores. Por ejemplo, un robot puede tener cámaras y micrófonos como sensores, y brazos mecánicos o ruedas como actuadores; un programa informático puede recibir datos de entrada y responder mediante acciones en la pantalla o decisiones en un sistema.

Un agente racional selecciona sus acciones en función de la información disponible, las consecuencias esperadas y los objetivos que persigue. En contextos inciertos, busca maximizar su resultado esperado, evaluando riesgos, beneficios y probabilidades. Esta idea es compatible con teorías matemáticas y computacionales desarrolladas en otros campos, como la economía, la teoría de la decisión y el control automático.

Sin embargo, en entornos complejos la racionalidad perfecta es inalcanzable. Los agentes no disponen de tiempo ni de recursos infinitos para calcular todas las posibilidades. Por ello, la inteligencia práctica se basa en la racionalidad limitada: actuar de manera suficientemente buena dadas las condiciones y restricciones del entorno.

Este enfoque permite integrar razonamiento lógico, aprendizaje automático, percepción y acción en un mismo marco teórico, y constituye hoy el paradigma predominante de la disciplina. Desde esta perspectiva, la meta de la inteligencia artificial es diseñar agentes que perciban, razonen, aprendan y actúen en el mundo de forma autónoma, adaptativa y beneficiosa para las personas, como mencionamos al inicio de esta unidad.

Finalidades y alcances de la IA

Cuando hablamos de IA, no solo nos referimos a "qué es" o "cómo funciona", también importa qué se busca lograr con ella. Los objetivos de la IA son diversos y van desde metas muy concretas y prácticas hasta preguntas profundas sobre nuestra propia mente.

Seguro le pasó de enfrentarse a tareas rutinarias, de esas que consumen tiempo y resultan monótonas. La IA puede encargarse de ellas con una velocidad y precisión impresionantes, liberando a las personas para que se concentren en trabajos más creativos y estratégicos. Pensemos en una línea de ensamblaje automatizada, en un sistema que procesa cientos de reclamos de seguros al mismo tiempo, o en el filtro de spam que evita que su casilla de correo se llene de mensajes no deseados. En todos estos casos, la IA no solo reemplaza el trabajo manual, sino que lo hace de manera más eficiente.

Sin embargo, la IA no siempre viene a sustituirnos, muchas veces funciona en sociedad con nosotros. Es como tener una herramienta que potencia lo que ya sabemos hacer. Por ejemplo, un médico que usa un sistema de IA para detectar señales tempranas de una enfermedad en un estudio clínico, o un escritor que revisa sus textos con un corrector gramatical. En ambos casos, la IA no reemplaza al profesional, sino que lo ayuda a ser más preciso y eficiente.

A su vez, existen problemas que son demasiado grandes o complicados para que una persona los pueda analizar en su totalidad. Aquí la IA juega un papel clave: puede simular escenarios de cambio climático, estudiar el genoma humano completo para buscar posibles curas o medicamentos, u organizar enormes redes de logística global, como hace Amazon con millones de paquetes. En pocas palabras, puede procesar información y variables que serían imposibles para nosotros manejar solos.

Énfasis

Para muchos investigadores, crear IA no solo es una cuestión tecnológica, sino también científica y filosófica. Al tratar de construir un sistema que razone, aprenda o incluso parezca consciente, nos vemos obligados a preguntarnos: ¿qué significa realmente razonar?, ¿cómo aprendemos?, ¿qué es la conciencia? Este camino nos ayuda no solo a mejorar nuestras máquinas, sino también a entendernos mejor como seres humanos.

En resumen, los objetivos de la IA se mueven en distintos niveles: desde lo práctico e inmediato, como sacarnos de encima tareas repetitivas, hasta lo más ambicioso, como desentrañar los misterios de la mente humana.

Actividad

Lo/a invito a profundizar estos conceptos con la realización de la **actividad 1** propuesta para este módulo. En la misma leeremos el artículo de Turing y debatiremos sobre sus propuestas en la IA moderna.

IA débil vs. IA fuerte

Uno de los debates más importantes en Inteligencia Artificial es la diferencia entre lo que llamamos IA débil e IA fuerte. No se trata de pensar en términos de "buena" o "mala", sino en qué tan especializada o general es la inteligencia de una máquina.

Respecto a la IA débil podemos decir que se trata de una inteligencia que está diseñada para hacer muy bien una sola cosa. No entiende realmente lo que hace, no tiene conciencia ni autoconciencia, simplemente simula comportamientos inteligentes en un contexto muy concreto.

Un ejemplo claro es el filtro de spam en su correo electrónico. Es excelente detectando mensajes sospechosos y sacándolos de su bandeja de entrada, pero si le pide que traduzca un texto o le recomiende una serie, no tiene la menor idea. Su conocimiento es profundo, pero muy limitado.

En nuestra vida diaria, casi toda la IA que usamos es de este tipo:

- Los algoritmos de recomendación de YouTube o Netflix.
- Los asistentes de voz como Siri o Alexa.
- El reconocimiento facial de su celular.
- Incluso los autos autónomos que, aunque parecen muy avanzados, están enfocados solo en conducir.

La clave es esta: la IA débil **da la impresión de ser inteligente**, pero en el fondo lo que hace es reconocer patrones y optimizar tareas específicas.

Ahora imaginemos algo mucho más ambicioso: una IA capaz de razonar, aprender y aplicar su conocimiento en diferentes situaciones, igual que una persona. Eso es lo que llamamos IA fuerte o **Inteligencia Artificial General (AGI)**.

Puede pensarlo con la analogía de un niño. Un chico puede aprender a jugar un videojuego, después usar la lógica y los reflejos del juego para practicar un deporte, y más tarde aplicar las habilidades sociales que aprendió en ese deporte para llevarse mejor con sus compañeros en la escuela. Su inteligencia es flexible, se adapta, y no se limita a un solo ámbito.

Énfasis

La característica clave de la IA fuerte es que no solo parecería inteligente: **realmente lo sería**. Podría pensar, razonar y aprender en el sentido más amplio de la palabra. ¿Existe hoy en día? La respuesta es no. La IA fuerte todavía es un concepto teórico, un objetivo lejano que muchos investigadores persiguen. Desarrollarla sería uno de los mayores logros científicos de nuestra época, pero también nos plantea preguntas profundas: ¿qué significa realmente ser consciente?, ¿qué responsabilidades tendríamos frente a una máquina con mente propia?, ¿qué riesgos y beneficios traería?

Lo/la invito a preguntarle al agente que más use si se considera IA débil o fuerte, ¿cuál piensa que será la respuesta?

Unidad 2: Historia y evolución de la IA

En la unidad anterior nos aproximamos a la definición general de la inteligencia artificial como el campo que estudia cómo construir sistemas capaces de razonar, aprender y actuar racionalmente. En esta unidad, daremos un paso más: exploraremos la evolución histórica de la disciplina, sus principales etapas y los enfoques que marcaron su desarrollo. Comprender esta trayectoria nos permite reconocer que la IA no es un fenómeno reciente ni exclusivamente técnico, sino el resultado de más de ocho décadas de reflexión, experimentación y descubrimiento interdisciplinario.

A lo largo de su historia, la IA ha transitado múltiples paradigmas: desde las primeras redes neuronales artificiales hasta el aprendizaje profundo contemporáneo. Cada período estuvo marcado por una combinación de avances teóricos, desarrollos tecnológicos, expectativas y desilusiones. Conocer este recorrido nos brinda una perspectiva crítica sobre las potencialidades y limitaciones actuales de la disciplina.

Desarrollemos cada una de estas etapas con mayor detalle:

1. Los inicios de la inteligencia artificial (1943–1956)

El origen de la IA puede situarse en los trabajos de Warren McCulloch y Walter Pitts (1943), quienes propusieron el primer modelo formal de neuronas artificiales. Inspirados por la fisiología del cerebro, la lógica proposicional de Russell y Whitehead y la teoría de la computación de Turing, mostraron que una red de neuronas binarias podía representar proposiciones lógicas y calcular funciones computables. Este modelo integró biología, lógica y matemáticas, dando lugar a una visión mecanicista del pensamiento.

Poco después, Donald Hebb (1949) propuso una regla de aprendizaje —conocida hoy como aprendizaje hebbiano— según la cual las conexiones entre neuronas se fortalecen cuando se activan conjuntamente. Este principio sigue siendo una base conceptual del aprendizaje automático moderno.

A comienzos de la década de 1950, Marvin Minsky y Dean Edmonds construyeron la primera computadora basada en una red neuronal, llamada SNARC, y Alan Turing introdujo ideas fundamentales en su artículo Computing Machinery and Intelligence (1950), donde formuló el célebre Test de Turing como criterio para evaluar inteligencia en máquinas. Turing anticipó, además, conceptos como el aprendizaje automático, los algoritmos genéticos y el aprendizaje por refuerzo, y advirtió que la creación de máquinas inteligentes podría tener implicaciones profundas para la humanidad.

En 1956, John McCarthy, junto con Minsky, Shannon y Rochester, organizó el Dartmouth Workshop, considerado el acto fundacional de la inteligencia artificial como disciplina científica. Allí se acuñó el término "artificial intelligence" y se sostuvo la hipótesis de que todo aspecto de la inteligencia humana puede describirse con precisión suficiente como para ser simulado por una máquina. Aunque no hubo resultados inmediatos, el encuentro estableció la agenda de investigación que guiaría el desarrollo posterior del campo.

2. Entusiasmo y grandes expectativas (1952–1969)

Tras Dartmouth comenzó una etapa de optimismo. Los investigadores buscaban demostrar que las máquinas podían resolver tareas típicamente humanas: jugar, razonar, aprender y comprender lenguaje. Allen Newell y Herbert Simon desarrollaron el Logic Theorist y luego el General Problem Solver (GPS) que vimos anteriormente en la primera unidad, programas capaces de resolver problemas mediante la manipulación de símbolos. Estos experimentos inspiraron la formulación de la hipótesis del sistema físico de símbolos, según la cual cualquier sistema inteligente —humano o artificial— debe operar sobre estructuras simbólicas.

De todo el trabajo exploratorio realizado durante este período, quizás el más influyente a largo plazo fue el de Arthur Samuel sobre juegos de damas. Usando métodos que ahora llamamos aprendizaje por refuerzo, los programas de Samuel aprendieron a jugar a un alto nivel aficionado. De este modo, refutó la idea de que las computadoras sólo pueden hacer lo que se les dice que hagan: su programa aprendió rápidamente a jugar mejor que su creador. El programa de Samuel fue el precursor de sistemas posteriores como TD-GAMMON, que estaba entre los mejores jugadores de backgammon del mundo, y AlphaGo, que conmocionó al mundo al derrotar al campeón mundial humano en Go.

En paralelo, John McCarthy creó el lenguaje LISP (1958), que se transformó en la herramienta predominante de la programación en IA durante décadas. En un artículo titulado Programas con sentido común, avanzó una propuesta conceptual para sistemas de IA basados en el conocimiento y razonamiento. El documento describe el Advice Taker, un programa hipotético que encarnaría el conocimiento general del mundo y podría usarlo para derivar planes de acción. El concepto se ilustró con axiomas lógicos simples que bastaban para generar un plan para conducir al aeropuerto. El programa también fue diseñado para aceptar nuevos axiomas en el curso normal de operación, lo que le permitía alcanzar competencia en nuevas áreas sin ser reprogramado. El Advice Taker, por lo tanto, encarnaba los principios centrales de representación del conocimiento y razonamiento: que es útil tener una representación formal y explícita del mundo y su funcionamiento y ser capaz de manipular esa representación con procesos deductivos. El documento influyó en el curso de la IA y sigue siendo relevante hoy en día.

Durante esta etapa también se desarrollaron los primeros programas que resolvían problemas matemáticos y lógicos, y se exploraron entornos simplificados o micromundos, que poseían dominios limitados. Por ejemplo, el programa SAINT de James Slagle (1963) fue capaz de resolver problemas de integración de cálculo en forma cerrada típicos de los cursos universitarios de primer año. El programa ANALOGY de Tom Evans (1968) resolvió problemas de analogía geométrica que aparecen en las pruebas de coeficiente intelectual. El programa STUDENT de Daniel Bobrow (1967) resolvió problemas de álgebra.

El micromundo más famoso es el mundo de los bloques, que consiste en un conjunto de bloques sólidos colocados sobre una mesa (o más a menudo, una simulación de una mesa). Una tarea típica en este mundo es reorganizar los bloques de cierta manera, utilizando una mano robótica que puede recoger un bloque a la vez. El mundo de los bloques fue el hogar del proyecto de visión de David Huffman (1971), del trabajo de visión y propagación de restricciones de David Waltz (1975), de la teoría del aprendizaje de Patrick

Winston (1970), del programa de comprensión del lenguaje natural de Terry Winograd (1972) y del planificador de Scott Fahlman (1974).

También florecieron los primeros trabajos basados en las redes neuronales de McCulloch y Pitts. El trabajo de Shmuel Winograd y Jack Cowan (1963) mostró cómo una gran cantidad de elementos podría representar colectivamente un concepto individual, con un aumento correspondiente en robustez y paralelismo. Los métodos de aprendizaje de Hebb fueron mejorados por Bernie Widrow, quien llamó a sus redes «adalines», y por Frank Rosenblatt (1962) quien desarrolló el perceptrón, una red capaz de aprender relaciones simples entre datos. Si bien sus capacidades eran limitadas, el perceptrón representó un avance importante hacia los modelos de aprendizaje automático.

3. Una dosis de realidad (1966–1973)

El entusiasmo inicial pronto dio lugar a una fase de desilusión. Muchos sistemas funcionaban bien en ejemplos pequeños, pero fracasaban en problemas reales. Herbert Simon había predicho que las máquinas serían campeonas de ajedrez y probarían teoremas importantes en pocos años, pero esas metas tardarían décadas en cumplirse.

Las causas del estancamiento fueron principalmente dos. Primero, los investigadores basaban sus programas en suposiciones intuitivas sobre el pensamiento humano (la llamada "introspección informada" sobre cómo los humanos realizan una tarea), sin un análisis formal del problema.

La segunda causa del fracaso fue la falta de apreciación de la intratabilidad de muchos de los problemas que la IA estaba intentando resolver. La mayoría de los primeros sistemas de resolución de problemas funcionó probando diferentes combinaciones de pasos hasta que se encontró la solución. Esta estrategia funcionó inicialmente porque los micromundos contenían muy pocos objetos y, por lo tanto, muy pocas acciones posibles y secuencias de solución muy cortas. Antes de que se desarrollara la teoría de la complejidad computacional, se pensaba ampliamente que "escalar" a problemas más grandes era simplemente una cuestión de hardware más rápido y memorias más grandes. El optimismo que acompañó el desarrollo de la demostración de teoremas de resolución, por ejemplo, pronto se atenuó cuando los investigadores no lograron probar teoremas que involucraran más de unas pocas docenas de hechos. La explosión combinatoria —el crecimiento exponencial de las posibilidades a explorar— hacía inviable resolver problemas de gran escala con los métodos existentes.

La publicación del informe Lighthill (1973) en el Reino Unido marcó un punto crítico: desaconsejó financiar proyectos de IA por considerar que no habían cumplido sus promesas. Además, el libro Perceptrons de Minsky y Papert (1969) mostró las limitaciones matemáticas de los modelos neuronales de la época, provocando un abandono casi total de esa línea de investigación. Este período fue conocido como el primer invierno de la IA.

4. La era de los sistemas expertos (1969–1986)

Tras la crisis, la disciplina se reorientó hacia el uso del conocimiento especializado. En lugar de buscar una inteligencia general, los investigadores comenzaron a construir pro-

gramas capaces de resolver problemas concretos en dominios específicos. Nacieron así los sistemas expertos, que codificaban reglas y heurísticas de expertos humanos.

El proyecto DENDRAL en la Universidad de Stanford fue pionero en aplicar este enfoque para inferir estructuras moleculares a partir de datos experimentales. Más tarde, MYCIN se utilizó para diagnosticar infecciones médicas, aplicando reglas con distintos grados de certeza. Estos sistemas demostraron que el conocimiento experto podía producir resultados prácticos y confiables, marcando el auge comercial de la IA durante los años ochenta.

El crecimiento generalizado de las aplicaciones a problemas del mundo real condujo al desarrollo de una amplia gama de herramientas de representación y razonamiento. Algunos se basaron en la lógica, por ejemplo, el lenguaje Prolog se hizo popular en Europa y Japón, y la familia PLANNER en los Estados Unidos. Otros, siguiendo la idea de frames de Minsky (1975), adoptaron un enfoque más estructurado, reuniendo hechos sobre tipos particulares de objetos y eventos y organizándolos en una gran jerarquía taxonómica análoga a una taxonomía biológica.

En 1981, el gobierno japonés anunció el proyecto "Quinta Generación", un plan de 10 años para construir computadoras inteligentes masivamente paralelas que ejecuten Prolog. En respuesta, Estados Unidos formó la Microelectronics and Computer Technology Corporation (MCC), un consorcio diseñado para asegurar la competitividad nacional. En ambos casos, la IA fue parte de un esfuerzo amplio, incluido el diseño de chips y la investigación sobre interfaces humanas. En Reino Unido, el informe Alvey restableció la financiación eliminada por el informe Lighthill. Sin embargo, ninguno de estos proyectos cumplió sus ambiciosos objetivos en términos de nuevas capacidades de IA o impacto económico.

Cuando las expectativas superaron los resultados, la industria de la IA sufrió una nueva caída: su segundo invierno. Resultó difícil construir y mantener sistemas expertos para dominios complejos, en parte porque los métodos de razonamiento utilizados por los sistemas se rompieron ante la incertidumbre y en parte porque los sistemas no podían aprender de la experiencia.

5. El resurgimiento de las redes neuronales (1986–presente)

A mediados de los años ochenta, varios equipos reinventaron el algoritmo de aprendizaje por retropropagación (backpropagation) desarrollado por primera vez a principios de la década de 1960, que permitió entrenar redes neuronales multicapa. Este hallazgo reavivó el interés por los modelos conexionistas, que buscaban emular el aprendizaje distribuido del cerebro. Los resultados, difundidos por Rumelhart y McClelland (1986), impulsaron la idea de que la inteligencia podía emerger del procesamiento paralelo de múltiples unidades simples.

Aunque al principio se los consideró competidores de los modelos simbólicos, los enfoques conexionistas y estadísticos empezaron a complementarse. Las redes neuronales demostraron ser especialmente eficaces para tareas perceptivas, como el reconocimiento de voz e imágenes, gracias a su capacidad de aprender directamente a partir de ejemplos.

6. Razonamiento probabilístico y aprendizaje automático (1987–presente)

El agotamiento de los sistemas expertos promovió una aproximación más rigurosa, basada en las matemáticas, la estadística y la experimentación. En este nuevo paradigma, la IA incorporó la incertidumbre y la probabilidad como parte esencial de su razonamiento. Se hizo más común construir sobre teorías existentes que proponer otras completamente nuevas, basar las afirmaciones en teoremas rigurosos o una sólida metodología experimental en lugar de en la intuición, y mostrar relevancia para aplicaciones del mundo real en lugar de ejemplos de juguete.

Judea Pearl (1988) desarrolló las redes bayesianas, un marco formal para representar dependencias entre variables y realizar inferencias probabilísticas. Este enfoque permitió construir sistemas más robustos y explicativos. Al mismo tiempo, Richard Sutton unió el aprendizaje por refuerzo con la teoría de los procesos de decisión de Markov, dando origen a un campo con aplicaciones en robótica, control y planificación.

Durante este período, la IA comenzó a reconectarse con disciplinas vecinas como la teoría del control, la investigación operativa y la optimización. Se establecieron competencias y bancos de datos estándar, como ImageNet o MNIST, que permitieron medir el progreso y consolidar la investigación empírica.

7. La era del big data (2001–presente)

El crecimiento exponencial del poder de cómputo y la expansión de Internet generaron enormes volúmenes de información: textos, imágenes, videos y registros digitales. Este fenómeno, conocido como big data, cambió el modo de construir sistemas inteligentes. Con grandes conjuntos de datos, los algoritmos de aprendizaje automático lograron resultados cada vez más precisos.

Se demostró que el aumento del volumen de datos podía mejorar el rendimiento más que el refinamiento de los algoritmos. Así, tareas como el reconocimiento de palabras o de objetos visuales alcanzaron niveles de precisión inéditos. En 2011, el sistema Watson de IBM venció a campeones humanos en el concurso Jeopardy!, mostrando el potencial de combinar grandes datos, procesamiento del lenguaje natural y razonamiento probabilístico.

La disponibilidad de big data y el cambio hacia el aprendizaje automático ayudaron a la IA a recuperar atractivo comercial.

8. El auge del aprendizaje profundo (2011–presente)

El aprendizaje profundo (deep learning) representa la consolidación de las redes neuronales multicapa como el núcleo de la IA moderna. A partir de 2012, los modelos desarrollados por el equipo de Geoffrey Hinton en la Universidad de Toronto lograron superar ampliamente los resultados previos en la competencia ImageNet, revolucionando el campo del reconocimiento visual.

Estos sistemas, basados en capas jerárquicas de representación, se extendieron rápida-

mente al reconocimiento de voz, la traducción automática, el diagnóstico médico y los videojuegos. El caso de AlphaGo, que derrotó al campeón mundial de Go en 2016, simboliza la madurez de este paradigma.

El aprendizaje profundo se apoya en tres pilares: la disponibilidad de grandes volúmenes de datos, el uso de hardware especializado (GPU y TPU) y la formulación de algoritmos eficientes. Su éxito ha impulsado una nueva ola de entusiasmo, inversión y debate ético sobre el papel de la IA en la sociedad.



Énfasis

En síntesis, la historia de la IA es una sucesión de olas: entusiasmo, desencanto, ajustes y renovaciones. Cada etapa dejó aprendizajes que nutrieron a la siguiente, y cada fracaso contribuyó a comprender mejor la complejidad del desafío. Mirar hacia atrás no solo nos permite reconocer los hitos que dieron forma a esta disciplina, sino también entender que el camino de la Inteligencia Artificial siempre ha estado marcado por la interacción entre sueños audaces y realidades técnicas. Esa tensión entre lo que aspiramos y lo que podemos alcanzar sigue siendo, aún hoy, la esencia de su desarrollo.



Actividad

Realice la **actividad 2** propuesta para este módulo, donde haremos un poco más de énfasis en los inviernos de la IA y sus posteriores auges.

Estado del arte y tendencias actuales en 2025

Tras haber explorado la historia y evolución de la IA, es momento de situarnos en su presente: una disciplina que ya no sólo experimenta avances técnicos, sino que se encuentra inmersa en procesos sociales, económicos, éticos y normativos de alcance global.

El Estudio de Cien Años de la Universidad de Stanford sobre la IA (también conocido como AI100) convoca paneles de expertos para proporcionar informes sobre el estado del arte en la IA. Su informe de 2021, titulado "Gathering Strength, Gathering Storms", describe el panorama actual como un equilibrio entre una capacidad cada vez mayor y tensiones crecientes. Documenta un período en el que los sistemas de IA se integraron profundamente en la vida cotidiana —en la medicina, el comercio, la logística, la educación y el entretenimiento—, mientras que su impacto social se volvió demasiado grande como para considerarlo una preocupación secundaria. A su vez, en 2025 se publicó una versión actualizada del AI Index Report, que aporta un panorama cuantitativo actualizado de investigación, desarrollo, adopción y gobernanza de la IA.

El informe AI100 2021 advierte que la IA ha dado un salto desde entornos de laboratorio hacia la vida cotidiana, lo cual incrementa la urgencia de comprender sus efectos negativos potenciales. Este cambio significa que la pregunta ya no es únicamente **"¿podremos construir máquinas inteligentes?"** sino **"¿cómo deben integrarse máquinas inteligentes en sociedades humanas?"**. En consecuencia, los temas de privacidad, sesgos, transparencia, impacto laboral y equidad se han vuelto parte integral del debate.

El AI Index Report 2025 refuerza esta perspectiva mostrando que el campo de la IA está en plena expansión cuantitativa: publicaciones, patentes, modelos de frontera y adopción industrial crecen a tasas muy elevadas. Este crecimiento exige revisar tanto las capacidades técnicas como los mecanismos de supervisión, gobernanza y responsabilidad.

Así, la investigación de la IA adquiere una doble dimensión: crear nuevas capacidades y gestionarlas de modo socialmente responsable.

El informe destaca que ya se observa IA en la salud, en la educación, en el transporte y en la economía —pero que su integración plantea riesgos concretos, como sistemas con errores, dependencia tecnológica o desplazamiento laboral. Por ejemplo, se señala que no es tanto una máquina superinteligente la preocupación principal, sino una máquina incompetente en contextos vitales, lo que puede producir daño si se despliega sin supervisión adecuada.

Por su parte, el índice cuantifica variables como costos de inferencia (consulta a modelos) y volumen de modelos de frontera, indicando que los costos se han reducido drásticamente, lo cual acelerará la adopción de IA en sectores diversos. También subraya que el dominio de la industria tecnológica en el desarrollo de IA sigue siendo muy fuerte (~90% de los modelos "notables" provienen de la industria en 2024) y que la brecha global (por ejemplo, entre EE.UU., China y Europa) persiste, aunque se reduce. Las aplicaciones no sólo crecen en número, sino también en escala, lo que exige atención desde la gobernanza.

A su vez, el índice identifica tres pilares tecnológicos que alimentan el estado del arte: (1) la caída en los costos de hardware y ejecución de modelos, (2) el incremento en la capacidad para realizar tareas de lenguaje, visión y multimodalidad, (3) la integración cada vez mayor en infraestructura científica, industrial y de salud. Por ejemplo, se documenta que el coste de ejecutar un modelo de rendimiento equivalente al de GPT-3.5 pasó de decenas de dólares por millón de tokens a centavos en unos 18 meses. Esto hace que lo que antes era tecnología de nicho esté llegando a escenarios de amplio impacto.

El AI100 2021, con mayor enfoque cualitativo, advierte que, aunque las capacidades técnicas avanzan, los sistemas aún enfrentan límites en generalidad, contexto y adaptabilidad humana. Esta combinación de avances tecnológicos y persistencia de retos conceptuales exige que el profesional en IA (o el estudiante de sus fundamentos) no se centre solo en la técnica, sino también en su contextualización.

Los datos del AI Index Report 2025 muestran que la producción académica, las patentes y el gasto en IA han alcanzado cifras de crecimiento acelerado: el número de publicaciones relacionadas con IA supera varios cientos de miles al año. Sin embargo, también emergen retos estructurales: desigualdad de género en la investigación, concentración de poder técnico y económico en pocas corporaciones, brechas internacionales y preocupaciones de gobernanza (privacidad, seguridad, impacto ambiental). Por ejemplo, el informe indica que menos de una cuarta parte de los investigadores en IA son mujeres y que la mayoría de los desarrollos provienen de EE.UU. y China, lo que plantea retos de diversidad e inclusión.

El AI100 2021 ya señalaba que los peligros de la IA no eran solo tecnológicos, sino también sociales, institucionales y éticos. Conjuntamente, estos informes apuntan a que el desarrollo de IA entra en una fase donde la supervisión, el diseño ético y la gobernanza son tan importantes como el diseño algorítmico.

En síntesis, la inteligencia artificial se encuentra en un punto de doble transformación: técnicamente más fuerte que nunca, pero también más entrelazada con los sistemas éti-

cos, económicos y políticos que la rodean. El informe AI100 caracteriza este momento como uno de "fuerza creciente y tormentas en formación": los mismos avances que permiten nuevos descubrimientos amplifican también desigualdades y responsabilidades. A medida que la IA continúe progresando, el éxito se medirá no solo por la precisión o el rendimiento, sino por la forma en que los sistemas inteligentes logren alinearse con los valores humanos y contribuir al bienestar colectivo.



Actividad

Con lo visto hasta aquí, resolvamos la de este módulo, para investigar un poco más sobre el estado del arte de la IA y sus capacidades hoy en día.



Lectura complementaria

Si desea profundizar más sobre el Estado del Arte de la IA puede leer el documento ***informe_AI100_2021*** *Disponible en Plataforma* y el Reporte de 2025 en https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf



Recursos

A su vez, lo/a invito a explorar la página web del AI Index Report 2025 en https://hai.stanford.edu/ai-index/2025-ai-index-report

A lo largo de este primer módulo recorrimos el camino que llevó a la IA desde una idea casi filosófica hasta una realidad presente en cada aspecto de nuestra vida. Comprendimos sus múltiples definiciones, los objetivos que la impulsan y los hitos que marcaron su desarrollo. Vimos que la IA no es solo un conjunto de algoritmos, sino una construcción humana que combina conocimiento, creatividad y responsabilidad. Este recorrido inicial nos prepara para avanzar hacia el siguiente módulo, donde exploraremos los fundamentos teóricos y los modelos de agentes inteligentes, el corazón conceptual que permitirá entender cómo una máquina puede percibir su entorno, tomar decisiones y actuar de manera racional. Comenzaremos a construir la base técnica que sostiene la Inteligencia Artificial moderna.

¡Continuemos!

Módulo 1 | Glosario

Agente inteligente: entidad que percibe su entorno y actúa sobre él. Es la unidad básica de análisis en IA moderna.

Algoritmo: conjunto finito y ordenado de pasos o instrucciones que permiten resolver un problema o realizar una tarea de manera sistemática. En IA, los algoritmos determinan cómo un sistema procesa datos, aprende o toma decisiones.

Aprendizaje automático (Machine Learning): rama de la Inteligencia Artificial que permite a las máquinas mejorar su desempeño con la experiencia, aprendiendo patrones a partir de datos sin ser programadas explícitamente para cada tarea.

Aprendizaje hebbiano: principio formulado por Donald Hebb (1949) que establece que la conexión entre dos neuronas se fortalece cuando se activan simultáneamente. Constituye una base teórica para los modelos de aprendizaje en redes neuronales.

Backpropagation (retropropagación): algoritmo utilizado para ajustar los pesos de las redes neuronales multicapa, minimizando el error entre la salida real y la esperada.

Big Data: conjunto de técnicas y tecnologías que permiten almacenar, procesar y analizar grandes volúmenes de datos de manera eficiente, base del desarrollo de la IA moderna.

Deep Learning (aprendizaje profundo): subcampo del aprendizaje automático basado en redes neuronales artificiales con múltiples capas, capaz de procesar información compleja como imágenes, texto o sonido.

IA débil (o estrecha): tipo de inteligencia artificial diseñada para realizar tareas específicas de forma muy eficiente, sin poseer conciencia ni comprensión general.

IA fuerte (o general): tipo teórico de inteligencia artificial con capacidades cognitivas comparables a las humanas: puede razonar, aprender y adaptarse a diferentes contextos. Aún no existe de manera práctica.

Inteligencia Artificial (IA): disciplina dedicada a diseñar y construir agentes que perciban su entorno y actúen sobre él de manera racional.

LISP: lenguaje de programación creado por John McCarthy en 1958, ampliamente utilizado en IA durante décadas por su facilidad para manipular listas y símbolos.

Percepción: capacidad de un agente para captar información de su entorno a través de sensores (visuales, auditivos, textuales, etc.) y convertirla en datos procesables.

Perceptrón: tipo de red neuronal simple desarrollada por Frank Rosenblatt en 1958, capaz de aprender relaciones lineales entre datos. Fue precursor de las redes modernas.

Razonamiento: proceso mediante el cual un sistema de IA analiza información y extrae conclusiones lógicas o toma decisiones en función de los datos y objetivos disponibles.

Red neuronal: conjunto de nodos o "neuronas" conectadas que procesan información en paralelo. Simula el aprendizaje y la percepción humana en tareas complejas.

Sesgo algorítmico: distorsión o parcialidad presente en los resultados de un sistema de IA, producto de datos de entrenamiento que reflejan prejuicios o desigualdades humanas.

Sistemas expertos: programas diseñados para resolver problemas específicos utilizando una base de conocimientos y reglas de inferencia similares al razonamiento humano.

Test de Turing: experimento propuesto por Alan Turing para evaluar si una máquina puede exhibir un comportamiento inteligente indistinguible del de un ser humano mediante el lenguaje.

Módulo 1 | Actividades

ACTIVIDAD 1
Debatiendo con Turing

Lectura complementaria

Lea el artículo original de Turing sobre la IA (Turing, 1950). ***maquinaria_computacional_e_inteligencia_Turing*** *Disponible en Plataforma*

En ese texto, Turing analiza varias objeciones a su propuesta y a su prueba de inteligencia. ¿Qué objeciones siguen siendo válidas? ¿Son válidas sus refutaciones? ¿Se le ocurren nuevas objeciones derivadas de los avances producidos desde que escribió el artículo?

En el artículo, predice que para el año 2000 una computadora tendría un 30% de posibilidades de superar una prueba de Turing de cinco minutos con un interrogador sin experiencia. ¿Qué posibilidades cree que tendría una computadora hoy? ¿Y dentro de otros 25 años?

Escriba sus respuestas en un documento y envíelas a su tutor/a por medio de la plataforma correspondiente.

ACTIVIDAD 2
Inviernos de la IA

Durante el desarrollo del módulo, hemos visto que los avances en inteligencia artificial no han sido lineales y que la disciplina atravesó distintos períodos de auge y declive. Estos momentos de retroceso, conocidos como "inviernos de la IA", se caracterizaron por una caída abrupta en la inversión, la investigación y el interés social y mediático en este campo.

A continuación, se le solicita que describa las causas de cada uno de estos colapsos y del auge de interés que los precedió. Escriba sus respuestas en una página y envíelas a su tutor/a por medio de la plataforma correspondiente.

ACTIVIDAD 3
¿Qué puede hacer la IA hoy?

Ayudándose de un agente, analice si las siguientes tareas pueden ser resueltas actualmente por computadoras:

a) Jugar un partido decente de ping-pong.
b) Conducir por el centro de El Cairo, Egipto.
c) Conducir por Victorville, California.
d) Comprar la comida para una semana en el mercado.
e) Comprar la comida para una semana en Internet.
f) Jugar una partida decente de bridge a nivel competitivo.
g) Descubrir y demostrar nuevos teoremas matemáticos.
h) Escribir una historia intencionadamente divertida.

i) Dar asesoramiento jurídico competente en un área especializada del derecho.
j) Traducir del inglés hablado al sueco hablado en tiempo real.
k) Realizar una operación quirúrgica compleja.

Para aquellas tareas que aún resultan inviables, examine cuáles son las principales dificultades y prediga cuándo, si corresponde, podrían superarse. Escriba sus respuestas en una página y envíelas a su tutor/a por medio de la plataforma correspondiente.

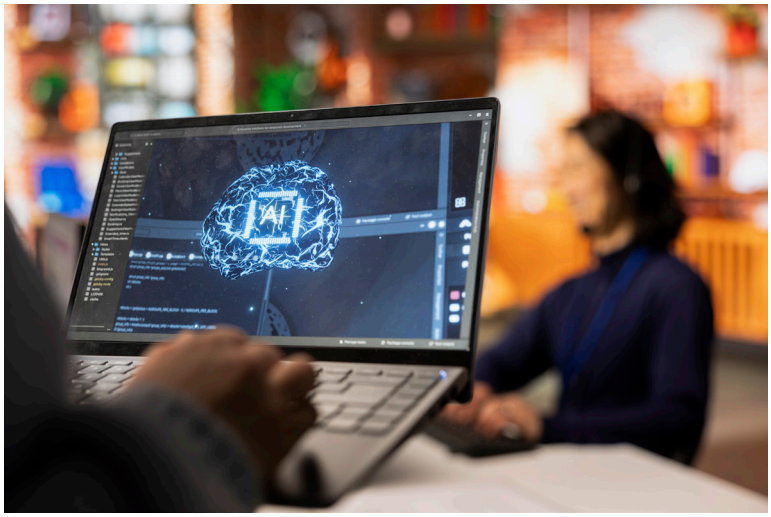
Módulo 2

Microobjetivos

- Comprender los fundamentos del concepto de agente inteligente y su ciclo percepción–acción, con el propósito de interpretar cómo un sistema percibe su entorno y actúa racionalmente para alcanzar objetivos definidos.
- Identificar los componentes esenciales de un agente para analizar su interacción dentro de sistemas reales y simulados.
- Distinguir las diferentes estructuras de agentes a fin de seleccionar el modelo más adecuado según la complejidad y naturaleza del problema a resolver.
- Analizar las propiedades de los entornos para determinar cómo influyen en el diseño y desempeño de los agentes inteligentes.

Contenidos

AGENTES INTELIGENTES



Fuente: https://www.freepik.es/

¡Felicitaciones por haber completado el primer módulo!

Damos comienzo al segundo módulo de la materia Fundamentos de Inteligencia Artificial. En el módulo anterior, aprendimos que la IA, desde sus orígenes, busca construir sistemas capaces de percibir su entorno, razonar y actuar de manera semejante, aunque no idéntica, a como lo haría un ser humano. Comprendimos también que la historia de la IA ha estado guiada por el objetivo de diseñar máquinas racionales, es decir, sistemas que puedan tomar decisiones adecuadas para alcanzar determinados fines.

En esta unidad daremos un paso más: exploraremos con mayor profundidad qué es un agente, cómo se relaciona con su entorno y qué significa que su conducta sea racional.

Unidad 3: Agentes y racionalidad

Énfasis

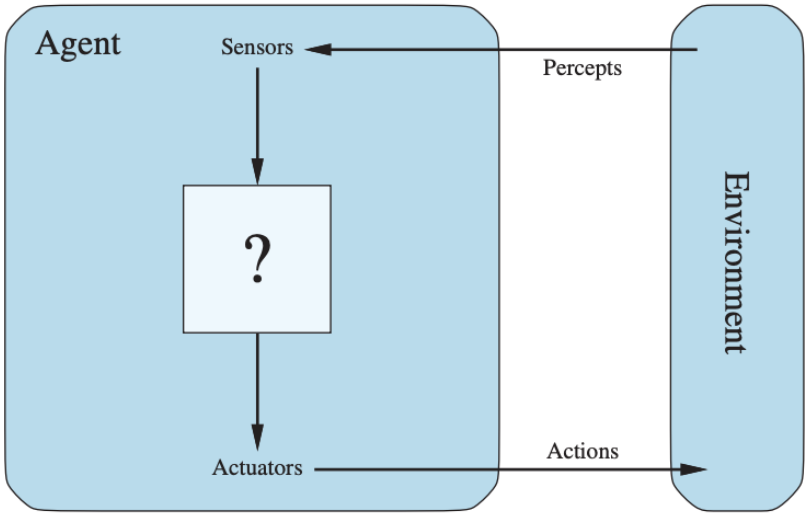
Un agente es una entidad que percibe su entorno mediante sensores y actúa sobre él mediante actuadores. Es, en esencia, una entidad que actúa con un propósito.

Para comprenderlo mejor, veamos sus componentes fundamentales:

- Agente: Es la entidad que toma las decisiones. No es necesariamente un robot físico, puede ser un programa de software, un sistema de control o cualquier entidad con capacidad de actuación. El núcleo del agente es su función del agente, que es el mapeo o algoritmo que transforma las secuencias de percepciones en acciones.
- Entorno: Es el mundo externo donde el agente opera y con el cual interactúa, determina qué información puede percibir el agente y qué consecuencias tienen sus acciones. Puede ser tan simple como una cuadrícula en un videojuego o tan complejo como el mundo real.
- Sensores: Son los dispositivos o canales a través de los cuales el agente recibe información sobre el entorno. Estas percepciones constituyen la entrada del agente.
- Actuadores: Son los mecanismos que permiten al agente influir y modificar el entorno. Las acciones que realiza constituyen su salida.

Esta definición es deliberadamente amplia porque busca abarcar tanto a los seres humanos y animales, como a los robots físicos y los programas informáticos que toman decisiones autónomas.

Un agente humano, por ejemplo, percibe el mundo mediante sus sentidos (vista, oído, tacto, olfato, gusto) y actúa a través de órganos como las manos, las piernas o utilizando la voz. Un agente robótico, en cambio, puede tener cámaras, sensores infrarrojos o ultrasónicos, y sus actuadores se corresponden con motores, brazos articulados o mecanismos de desplazamiento. Por último, un agente de software, también llamado agente virtual o digital, percibe el entorno a través de flujos de datos como archivos, mensajes de red o entradas del usuario, y actúa mediante la generación de respuestas, el envío de información, o la ejecución de comandows dentro de un sistema informático.



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 37.

El ciclo básico de operación de cualquier agente es un ciclo de percepción y acción: el agente recibe percepciones del entorno, las interpreta, y responde con **acciones** que modifican ese entorno. El conjunto de todas las percepciones recibidas a lo largo del tiempo conforma la secuencia de percepción, que constituye la base de su experiencia. Este esquema es general y permite comparar sistemas muy distintos. Por ejemplo, un termostato implementa un ciclo simple: percibe la temperatura, la compara con un valor objetivo y enciende o apaga la calefacción. Un asistente virtual realiza el mismo proceso, pero con mayor complejidad: analiza texto o voz, interpreta intenciones y produce respuestas adecuadas.



El ciclo percepción-acción es la estructura mínima que define a un agente. Todo comportamiento inteligente emerge de este bucle.

En general, la elección de acción de un agente en cualquier instante dado puede depender de su conocimiento incorporado y de toda la secuencia de percepción observada hasta la fecha, pero no de nada que no haya percibido. Al especificar la elección de acción del agente para cada secuencia de percepción posible, hemos dicho más o menos todo lo que hay que decir sobre el agente.

Matemáticamente hablando, decimos que el comportamiento de un agente se describe mediante la *función del agente* que asigna cualquier secuencia de percepción dada a una acción determinada. Si pudiéramos registrar cómo responde un agente ante cada situación imaginable, obtendríamos una tabla que describiría completamente su comportamiento. Sin embargo, dicha tabla sería prácticamente infinita, ya que la cantidad de percepciones posibles crece sin límite con el tiempo.

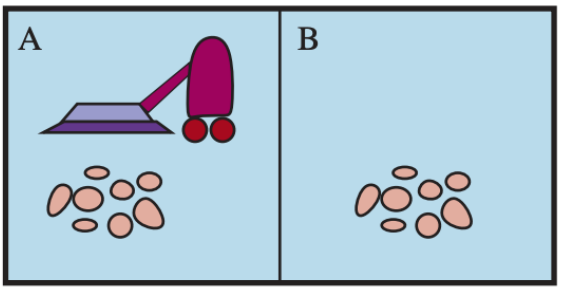
En la práctica, lo que implementamos no es la función abstracta del agente, sino un *programa del agente:* un sistema computacional concreto que opera dentro de una máquina física y que intenta aproximarse a esa función ideal. El programa del agente constituye la "mente" o mecanismo de decisión, mientras que los sensores y actuadores serían el "cuerpo" a través del cual se relaciona con el mundo.



Es importante mantener estos dos conceptos bien diferenciados: la función del agente es una descripción matemática abstracta, mientras que el programa del agente es una implementación concreta que se ejecuta dentro de algún sistema físico.

Por ejemplo, un sistema de conducción autónoma, como los vehículos automatizados actuales, implementa un programa que interpreta continuamente información visual de cámaras y sensores de proximidad, detecta peatones, señales y obstáculos, y ejecuta acciones como frenar, girar o acelerar. Su comportamiento observable puede describirse mediante una función del agente, pero su implementación real se basa en complejos algoritmos de percepción, predicción y control, lo que sería su programa de agente.

Para entender mejor estas ideas, usemos un ejemplo simple, el mundo de las aspiradoras, que consiste en una aspiradora robot en un mundo compuesto por cuadrados que pueden estar sucios o limpios. A continuación, veamos una configuración con sólo dos cuadrados, A y B.



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 38.

El agente percibe su ubicación y el estado de limpieza de la celda actual, y puede ejecutar cuatro acciones básicas: moverse a la derecha, moverse a la izquierda, aspirar la suciedad o no hacer nada. Una función de agente muy simple es la siguiente: si la celda actual está sucia, aspirar; de lo contrario, desplazarse hacia la otra celda. Una tabla parcial de esta función del agente sería:

Secuencia de percepción	Acción
[A, Limpio]	Derecha
[A, Sucio]	Aspirar
[B, Limpio]	Izquierda
[B, Sucio]	Aspirar
[A, Limpio], [A, Limpio]	Derecha
[A, Limpio], [A, Sucio]	Aspirar
—	—
—	—
—	—
[A, Limpio], [A, Limpio], [A, Limpio]	Derecha
[A, Limpio], [A, Limpio], [A, Sucio]	Aspirar
—	—
—	—
—	—

Veremos un ejemplo del programa del agente más adelante cuando hablemos de la estructura de los agentes. Pero si nos enfocamos en la tabla, podemos definir distintos agentes del mundo de las aspiradoras simplemente completando la columna de la derecha de distintas maneras. Aquí surge una pregunta fundamental: ¿cuál es la forma correcta de completar esta tabla? Es decir, ¿cómo evaluamos si un agente es "bueno" o "malo", "inteligente" o "ineficiente"? La respuesta se vincula con la noción de racionalidad, que veremos a continuación.

Para terminar, resulta importante destacar que el concepto de "agente" no pretende dividir el mundo entre objetos que lo son y los que no lo son, sino que constituye una herramienta conceptual para analizar sistemas. Podemos considerar a una calculadora, por ejemplo, como un agente que responde con el número "4" cuando percibe "2 + 2 =". Sin embargo, tal análisis no aporta demasiado para comprender su funcionamiento. En cam-

bio, cuando aplicamos esta noción a sistemas que deben adaptarse, aprender o tomar decisiones en entornos cambiantes como un robot, un asistente virtual o un algoritmo de recomendación, el concepto de agente se vuelve sumamente útil.

El concepto de racionalidad

Énfasis

Decimos que un agente es racional cuando actúa de la mejor manera posible para alcanzar sus objetivos, teniendo en cuenta lo que percibe, lo que sabe y las acciones que puede realizar.

La pregunta, entonces, sería: ¿qué significa para un agente actuar de la mejor manera posible? En IA, lo definimos con un concepto filosófico llamado *consecuencialismo*: evaluamos el comportamiento de un agente por sus consecuencias. Cuando un agente se introduce en un entorno, genera una secuencia de acciones según las percepciones que recibe. Esta secuencia de acciones hace que el entorno pase por una secuencia de estados. Si la secuencia es deseable, entonces el agente ha funcionado bien. Para evaluar si un agente actúa correctamente, es necesario contar con un criterio que nos permita determinar si su comportamiento fue exitoso. Este criterio se conoce como medida de rendimiento, y representa una forma de valorar los resultados de las acciones del agente dentro del entorno.

Ahora bien, los humanos tienen deseos y preferencias propios, por lo que la noción de racionalidad aplicada a los humanos tiene que ver con su éxito a la hora de elegir acciones que produzcan secuencias de estados del entorno que sean deseables desde *su punto de vista*. Las máquinas, por otro lado, no tienen deseos ni preferencias propias, la medida de rendimiento está, al menos inicialmente, en la mente del diseñador de la máquina o en la mente de los usuarios para los que se ha diseñado la máquina. Vamos a ver más adelante que algunos diseños de agente tienen una representación explícita de su medida de rendimiento, mientras que en otros diseños estará implícita, el agente puede hacer lo correcto, pero no sabe por qué.

Definir una buena medida de rendimiento no siempre es sencillo. Pensemos nuevamente en el mundo de las aspiradoras, que consiste en dos casillas (A y B) que pueden estar limpias o sucias. Si establecemos que el rendimiento del agente se mida únicamente por la cantidad de suciedad aspirada en un turno de 8 horas, este podría comportarse de forma absurda: limpiar la suciedad, vaciarla nuevamente al piso y volver a aspirarla indefinidamente, acumulando puntos sin lograr un entorno verdaderamente limpio.

Una medida más adecuada sería aquella que premie los estados deseados del entorno, por ejemplo, otorgando puntos por cada celda limpia en cada momento y penalizando el consumo de energía o los movimientos innecesarios.

Énfasis

Como regla general, es mejor diseñar medidas de rendimiento en función de lo que realmente se espera conseguir como resultado, en lugar de en función de cómo se cree que debería comportarse el agente.

Diseñar la medida de rendimiento de forma incorrecta puede conducir a comportamientos no deseados o incluso perjudiciales, un riesgo que debe evitarse especialmente en los sistemas autónomos actuales.

¿Qué determina la racionalidad de un agente?

La racionalidad de un agente no depende solo de sus acciones, sino también del contexto en que actúa. En particular, se define a partir de cuatro elementos:

1. La medida de rendimiento, que establece el criterio de éxito.
2. El conocimiento previo que el agente posee sobre su entorno.
3. Las acciones disponibles, es decir, lo que efectivamente puede hacer.
4. La secuencia de percepciones recibida hasta el momento, que representa su experiencia acumulada.

Un agente racional, entonces, es aquel que elige en cada momento la acción que maximiza su desempeño esperado, considerando lo que ha percibido y lo que sabe del entorno. Volvamos al ejemplo de la aspiradora que limpia un cuadrado si está sucio y se mueve al otro cuadrado si no (describimos su función del agente en la tabla más arriba). ¿Podemos decir que es un agente racional? Para poder responder a esta pregunta primero necesitamos definir cuál es su medida de rendimiento, qué es lo que se sabe del entorno, y qué sensores y actuadores posee el agente. Asumamos lo siguiente:

- La medida de rendimiento otorga un punto por cada cuadrado limpio en cada intervalo de tiempo, durante una "vida útil" de 1000 intervalos de tiempo.
- La "geografía" del entorno se conoce de antemano (2 cuadrados, A y B) pero la distribución de la suciedad y la ubicación inicial del agente se desconocen.
- Las casillas limpias permanecen limpias y al aspirar se limpia la casilla actual. Las acciones *Derecha* e *Izquierda* mueven al agente una casilla, excepto cuando esto llevaría al agente fuera del entorno, en cuyo caso el agente permanece donde está.
- Las únicas acciones disponibles son *Derecha*, *Izquierda* y *Aspirar*.
- El agente percibe correctamente su ubicación y si esa ubicación contiene suciedad.

Bajo estas circunstancias, podemos decir que el agente es racional. Su rendimiento esperado es al menos tan bueno como el de cualquier otro agente. Sin embargo, si el entorno cambia y las circunstancias son diferentes, veremos que el agente puede volverse irracional. Por ejemplo, una vez que toda la suciedad esté limpia, el agente oscilará innecesariamente de un lado a otro; si la medida de rendimiento incluye una penalización de un punto por cada movimiento, el agente tendrá un mal desempeño. Un mejor agente para este caso no haría nada una vez que esté seguro de que todos los cuadrados están limpios. Si los cuadrados limpios pueden volver a ensuciarse, el agente debe ocasionalmente verificarlos y volver a limpiarlos si es necesario. Si se desconoce la geografía del entorno, el agente deberá explorarlo. Esto muestra que la racionalidad no es una característica fija del agente, sino una relación entre su comportamiento, su entorno y los objetivos definidos.

Racionalidad no es perfección

Un agente racional no es necesariamente perfecto. La perfección implicaría conocer con certeza todas las consecuencias de sus acciones, ser omnisciente, algo que en la práctica es imposible. Un ejemplo cotidiano ayuda a comprender esta diferencia, supongamos que una persona cruza la calle porque no ve vehículos cerca. La decisión es racional, ya

que se basa en la información que tiene. Si ocurre un accidente por un hecho imprevisible, como la caída de un objeto desde un avión, no puede decirse que haya actuado irracionalmente. La acción fue la mejor posible de acuerdo con lo que se sabía en ese instante.

De la misma forma, un agente racional puede cometer errores si el entorno cambia de manera inesperada, pero eso no lo hace irracional. Lo importante es que haya elegido su acción en función de la evidencia disponible en ese momento y de su conocimiento previo.

Un agente verdaderamente racional no solo actúa, sino que también recopila información cuando esta puede mejorar sus percepciones futuras. Este tipo de acciones —conocidas como acciones de exploración o de información— permiten reducir la incertidumbre sobre el entorno. Una muestra de recopilación de información puede ser la exploración que debe realizar una aspiradora robot en un entorno inicialmente desconocido.

Además de explorar, un agente racional debe *aprender* lo más que pueda de lo que percibe. A medida que interactúa con el entorno, puede descubrir patrones y ajustar su comportamiento para mejorar su desempeño. Un robot aspirador que detecta qué zonas de una casa acumula más polvo y decide recorrerlas con mayor frecuencia está aplicando aprendizaje. De modo similar, un asistente virtual que adapta sus respuestas a las preferencias del usuario se vuelve progresivamente más racional porque utiliza lo aprendido para actuar mejor.

Autonomía y adaptación

En la medida en que un agente se basa en el conocimiento previo de su diseñador en lugar de en sus propias percepciones y procesos de aprendizaje, decimos que el agente carece de autonomía. Un agente racional debe ser autónomo: debe aprender todo lo que pueda para compensar los conocimientos previos parciales o incorrectos. Por ejemplo, una aspiradora robot que aprende a predecir dónde y cuándo aparecerá más suciedad funcionará mejor que uno que no lo hace.

En la práctica, rara vez se requiere una autonomía completa desde el principio: cuando el agente tiene poca o ninguna experiencia, tendría que actuar de forma aleatoria a menos que el diseñador le prestara alguna ayuda. Del mismo modo que la evolución dota a los animales de los reflejos necesarios para sobrevivir el tiempo suficiente para aprender por sí mismos, sería razonable dotar a un agente de inteligencia artificial de algunos conocimientos iniciales, así como de la capacidad de aprender. Tras adquirir suficiente experiencia en su entorno, el comportamiento de un agente racional puede llegar a ser efectivamente independiente de sus conocimientos previos. Por lo tanto, la incorporación del aprendizaje permite diseñar un único agente racional que tendrá éxito en una gran variedad de entornos. Un sistema de control de tráfico que ajusta sus tiempos de semáforo según la densidad real de vehículos, o un programa de diagnóstico médico que mejora sus predicciones a medida que incorpora nuevos casos, son ejemplos de *agentes adaptativos*, capaces de mejorar su racionalidad con la experiencia.

Unidad 4: Tipos de agentes y entornos

Ya tenemos una definición de racionalidad y estamos casi listos para pensar en construir agentes racionales. Pero primero tenemos que centrarnos en entender los *entornos de trabajo*, que son esencialmente los "problemas" para los que los agentes racionales son las "soluciones".

Empecemos por aprender cómo podemos especificar el entorno de trabajo de un agente. Cuando hablamos de la racionalidad de la aspiradora robot especificamos las medidas de rendimiento, el entorno y los actuadores y sensores del agente. Este conjunto de cuatro elementos es lo que denominamos **entorno de trabajo**, para describirlo con precisión se utiliza un esquema muy común conocido como REAS (Rendimiento, Entorno, Actuadores, Sensores). Definir correctamente cada uno de estos elementos es el primer paso antes de diseñar un agente, ya que de ellos depende su comportamiento racional.

Consideremos un ejemplo más complejo que el de la aspiradora robot como puede ser un taxista automatizado. Veamos la descripción REAS para el entorno de trabajo de este ejemplo:

Tipo de agente	Medidas de rendimiento	Entorno	Actuadores	Sensores
Taxista	Seguro, rápido, legal, viaje confortable, maximización del beneficio, minimizar el impacto en otros usuarios	Calles, tráfico, policía peatones, clientes, clima	Volante, acelerador, freno, guiño, bocina, pantalla, sintetizador de voz	Cámaras, sensores, velocímetro, GPS, tacómetro, acelerómetro sensores del motor, teclado

Ahora bien, veamos con más detalle cada elemento:

Hablemos primero de la medida de rendimiento, ¿cuál sería la que nos gustaría que nuestro conductor automatizado aspirara? Las cualidades deseables incluyen llegar al destino correcto, minimizar el consumo de combustible y el desgaste, minimizar el tiempo o el costo del viaje, minimizar las violaciones de las leyes de tránsito y las molestias a otros conductores, maximizar la seguridad y la comodidad de los pasajeros, y maximizar las ganancias. Obviamente, algunos de estos objetivos entran en conflicto, por lo que será necesario hacer concesiones.

A continuación, ¿cuál es el entorno de conducción al que se enfrentará el taxi? Cualquier taxista debe lidiar con una gran variedad de calles, desde caminos rurales, calles urbanas angostas y hasta autopistas de 12 carriles. Las calles contienen otro tráfico, peatones, animales callejeros, obras, autos de policía, charcos y baches. El taxi también debe interactuar con pasajeros potenciales y reales. También hay algunas opciones adicionales, como, por ejemplo, el taxi podría tener que manejar en el norte de la Argentina, donde la nieve rara vez es un problema, o en el sur, donde rara vez no lo es. Siempre podría conducir por la derecha, o podríamos querer que fuera lo suficientemente flexible como para conducir por la izquierda cuando esté en Reino Unido o Japón. Obviamente, cuanto más restringido sea el entorno, más fácil será el problema de diseño.

Los actuadores de un taxi automatizado incluyen los que están a disposición de un conductor humano: control del motor mediante el acelerador y control de la dirección y el frenado. Además, necesitará una salida a una pantalla o un sintetizador de voz para comunicarse con los pasajeros, y quizás alguna forma de comunicarse con otros vehículos.

Los sensores básicos del taxi incluirán una o más cámaras de vídeo para que pueda ver, así como sensores infrarrojos o sonares para detectar distancias a otros autos y obstáculos. Para evitar multas por exceso de velocidad, el taxi debería tener un velocímetro y, para controlar el vehículo correctamente, en especial en las curvas, debería tener un acelerómetro. Para determinar el estado mecánico del vehículo, necesitará la gama habitual de sensores que controlen el motor y el sistema eléctrico. Al igual que muchos conductores humanos, es posible que quiera acceder a señales GPS para no perderse. Por último, necesitará una pantalla táctil o micrófono para que el pasajero pueda solicitar un destino.

Actividad

Con lo aprendido, realice la **actividad 1** propuesta para este módulo.

Clasificación de los entornos de trabajo

En IA podemos pensar en una interminable variedad de entornos de trabajo. Sin embargo, podemos identificar un número pequeño de dimensiones o características a lo largo de las cuales podemos clasificarlos. Estas propiedades no solo ayudan a describir los contextos en los que los agentes operan, sino que también orientan el diseño del programa que los controla.

Multimedia

Lo/a invitamos a ingresar al siguiente enlace y leer las características de los entornos de agentes de inteligencia artificial

https://view.genially.com/696e3a7914056139 54f4d134/interactive-content-caracteristicas-de-los-entornos-de-agentes

Énfasis

El diseño de un agente racional depende directamente del tipo de entorno en el que debe actuar. Cuanto más incierto, dinámico y complejo sea ese entorno, mayor será la necesidad de incorporar aprendizaje, memoria, predicción y razonamiento para lograr un comportamiento inteligente.

Veamos un listado de ejemplos de entornos de trabajo y sus características:

Entorno de trabajo	Observable	Agentes	Determinista	Episódico	Estático	Discreto
Crucigrama	Totalmente	Individual	Determinista	Secuencial	Estático	Discreto
Ajedrez con reloj	Totalmente	Multi	Determinista	Secuencial	Semi	Discreto
Poker	Parcialmente	Multi	Estocástico	Secuencial	Estático	Discreto
Backgammon	Totalmente	Multi	Estocástico	Secuencial	Estático	Discreto
Taxi circulando	Parcialmente	Multi	Estocástico	Secuencial	Dinámico	Continuo
Diagnóstico médico	Parcialmente	Individual	Estocástico	Secuencial	Dinámico	Continuo
Análisis de imagen	Parcialmente	Individual	Determinista	Episódico	Semi	Continuo
Robot clasificador	Totalmente	Individual	Estocástico	Episódico	Dinámico	Continuo
Controlador de refinería	Parcialmente	Individua	Estocástico	Secuencial	Dinámico	Continuo
Tutor de inglés	Parcialmente	Multi	Estocástico	Secuencial	Dinámico	Discreto

Tengamos en cuenta que las propiedades no siempre son claras. Por ejemplo, hemos incluido la tarea de diagnóstico médico como agente único porque el proceso de la enfermedad en un paciente no se modela de forma rentable como un agente, pero un sistema de diagnóstico médico también podría tener que lidiar con pacientes tercos y personal escéptico, por lo que el entorno podría tener un aspecto multiagente. Además, el diagnóstico médico es episódico si se concibe la tarea como la selección de un diagnóstico a partir de una lista de síntomas, pero el problema es secuencial si la tarea puede incluir proponer una serie de pruebas, evaluar el progreso a lo largo del tratamiento, atender a múltiples pacientes, etc.

No incluimos una columna de "conocido/desconocido" porque, como explicamos anteriormente, esto no es estrictamente una propiedad del entorno. Para algunos entornos, como el ajedrez y el póker, es bastante fácil proporcionar al agente un conocimiento completo de las reglas, pero no obstante es interesante considerar cómo un agente podría aprender a jugar a estos juegos sin ese conocimiento.



Lo/a invito a realizar la **actividad 2** de este módulo para reforzar las categorías de entornos de trabajo.

La estructura de los agentes

Hasta este punto hemos analizado cómo los agentes perciben su entorno, qué tipos de entornos pueden encontrar y de qué manera puede evaluarse su comportamiento racional. Sin embargo, aún falta comprender cómo funcionan internamente, es decir, cómo transforman sus percepciones en acciones concretas.

leer para el final

En términos generales, un agente puede entenderse como la combinación de dos componentes:

- La *arquitectura*, que incluye los elementos físicos o virtuales que permiten percibir y actuar (sensores y actuadores).
- El *programa del agente*, que define el modo en que las percepciones se procesan para seleccionar las acciones adecuadas.

En una formulación sencilla, podemos expresarlo como:

Agente = Arquitectura + Programa

Es importante que el programa que elijamos sea adecuado para la arquitectura. Si el programa va a recomendar acciones como «caminar», es mejor que la arquitectura tenga piernas. La arquitectura puede ser una computadora personal normal y corriente, o puede ser un auto robótico con varias computadoras, cámaras y otros sensores a bordo. En general, la arquitectura pone a disposición del programa las percepciones de los sensores, ejecuta el programa y transmite las opciones de acción del programa a los actuadores a medida que se generan.

Todos los programas de agentes que vamos a explicar en esta unidad tienen el mismo esqueleto: toman como entrada para los sensores la percepción actual y devuelven una ac-

ción para los actuadores. Notemos la diferencia entre el *programa* del agente, que toma la percepción actual como entrada, y la *función* del agente, que puede llegar a depender de todo el historial completo de percepciones. El programa del agente no tiene más remedio que utilizar solo la percepción actual como entrada porque no hay nada más disponible del entorno. Si las acciones del agente necesitan depender de la secuencia completa de percepciones, entonces, el agente tendría que recordarla.

Veamos un ejemplo en pseudocódigo de un programa del agente bastante simple que almacena la secuencia de percepciones y después las compara con las secuencias almacenadas en la tabla de acciones para decidir qué hacer.

```
función AGENTE-DIRIGIDO-MEDIANTE-TABLA(percepción) devuelve una acción
    variables estáticas: percepciones, una secuencia, vacía inicialmente
                         tabla, una tabla de acciones, indexada por las secuencias de
                         percepciones, totalmente definida inicialmente

    añadir la percepción al final de las percepciones
    acción ← CONSULTA(percepciones, tabla)
    devolver acción
```

Fuente: elaboración propia.

Si quisiéramos construir un agente racional de esta forma, como diseñadores deberíamos construir una tabla en la que se especifique qué acción debe realizarse ante cada posible secuencia de percepciones. Este enfoque se conoce como agente basado en tabla, y aunque en teoría podría reproducir cualquier comportamiento racional, en la práctica es imposible de implementar. Por ejemplo, si consideramos un agente como un vehículo autónomo, la cantidad de percepciones que recibe en solo una hora de funcionamiento —imágenes, sonidos, lecturas de sensores, posiciones GPS— sería tan inmensa que la tabla requerida tendría un número de combinaciones superior al de los átomos del universo observable.



El desafío clave para la IA es descubrir cómo escribir programas que, en la medida de lo posible, produzcan un comportamiento racional a partir de un programa pequeño en lugar de una tabla enorme.

Profundicemos en los tipos básicos de programas de agentes que existen y las características de cada uno:

Agentes reactivos simples

El tipo más básico de agente se denomina reactivo simple. Este selecciona su acción únicamente en función de la percepción actual, sin considerar lo ocurrido en el pasado. Operan mediante reglas de condición-acción, del tipo:

Si ocurre la condición X, entonces realizar la acción Y

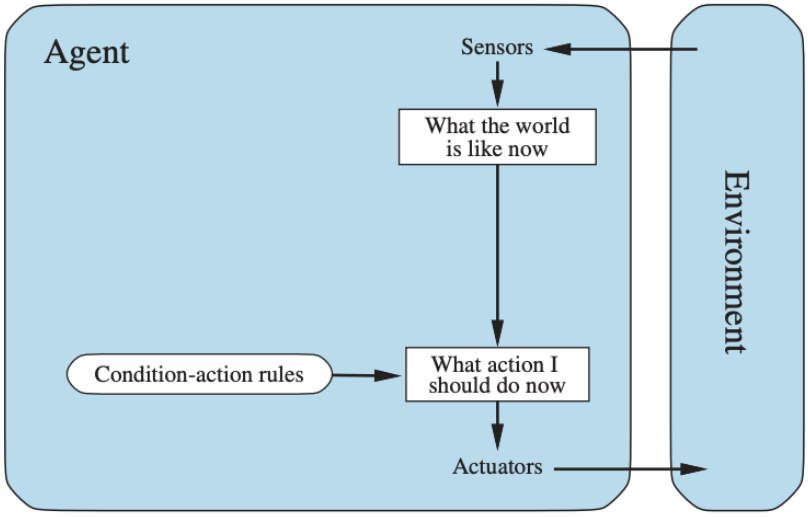
Un ejemplo es la aspiradora robot que aspira cuando detecta suciedad y se desplaza en caso contrario, su decisión se basa únicamente en la ubicación actual y en si esa ubicación contiene suciedad. Un programa del agente para este caso sería el siguiente:

```
función AGENTE-ASPIRADORA-REACTIVO([ubicación, estado]) devuelve una acción

    si estado = Sucio entonces devolver Aspirar
    de otra forma, si ubicación = A entonces devolver Derecha
    de otra forma, si ubicación = B entonces devolver Izquierda
```

Fuente: elaboración propia.

Vemos que la percepción se traduce directamente en una acción mediante un conjunto de reglas simples. Sin embargo, este programa es específico para un entorno en particular (sólo dos cuadrados, A y B). Un enfoque más general y flexible consiste en crear primero un intérprete de uso general para reglas de condición-acción y, a continuación, crear conjuntos de reglas para entornos de tareas específicos. La estructura de un agente con estas características sería:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 50.

El programa del agente para este caso sería:

```
función AGENTE-REACTIVO-SIMPLE(percepción) devuelve una acción
    variables estáticas: reglas, un conjunto de reglas condición-acción

    estado ← INTERPRETAR-ENTRADA(percepción)
    regla ← REGLA-COINCIDENCIA(estado, reglas)
    acción ← REGLA-ACCIÓN[regla]
    devolver acción
```

Fuente: elaboración propia.

La función INTERPRETAR-ENTRADA devuelve una descripción abstracta del estado actual a partir de la percepción, y la función REGLA-COINCIDENCIA retorna la primera regla del conjunto de reglas que coincide con la descripción del estado dada.

Este tipo de comportamiento es muy eficaz en entornos completamente observables y estables, donde las consecuencias de las acciones son predecibles y las reglas no cambian. Sin embargo, tiene grandes limitaciones en entornos dinámicos o inciertos, donde no toda la información está disponible o donde las mismas percepciones pueden requerir respuestas distintas según el contexto.

Por ejemplo, supongamos que la aspiradora robot pierde su sensor de ubicación y solo tiene el sensor de suciedad. En este caso solo tiene dos percepciones posibles: *Sucio* y *Limpio*. Puede aspirar en respuesta a *Sucio*, pero ¿qué debería hacer si percibe *Limpio*? Moverse hacia la izquierda falla (para siempre) si el agente comienza en el cuadrado A, y moverse hacia la derecha falla (para siempre) si comienza en el cuadrado B. Los bucles infinitos son a menudo inevitables para este tipo de agentes si operan en tornos parcialmente observables.

Es posible escapar de los bucles infinitos si el agente puede aleatorizar sus acciones. Por ejemplo, si la aspiradora percibe *Limpio*, podría tirar una moneda al aire para elegir entre *Derecha* e *Izquierda*. Es fácil demostrar que el agente llegará al otro cuadrado en una media de dos pasos. Entonces, si esa casilla está sucia, el agente la limpiará y la tarea se completará. Por lo tanto, un agente reactivo simple aleatorio podría superar a un agente reactivo simple determinista.

En entornos con un solo agente, la aleatoriedad no suele ser racional. Es un truco útil que ayuda a un agente reactivo simple en algunas situaciones, pero en la mayoría de los casos podemos obtener mejores resultados con agentes deterministas más sofisticados.

Agentes reactivos basados en modelo

La forma más efectiva de gestionar la observabilidad parcial es que el agente realice un seguimiento de la parte del mundo que no puede ver en ese momento. Es decir, el agente debe mantener algún tipo de estado interno que dependa del historial de percepciones y que, por lo tanto, refleje al menos algunos de los aspectos no observados del estado actual.

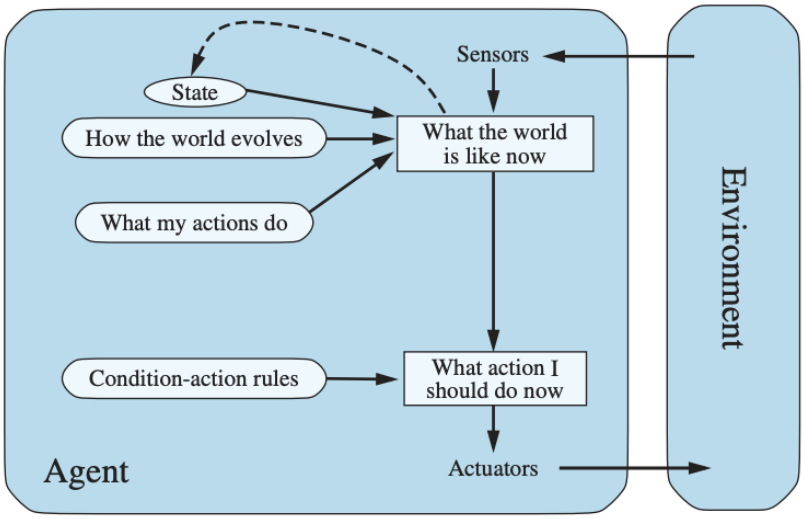
El agente mantiene un registro de "cómo cree que es el mundo en este momento" y lo actualiza continuamente en función de sus percepciones, de las acciones realizadas y del conocimiento que posee sobre cómo evoluciona el entorno. Para mantener esta representación interna, el agente necesita dos tipos de conocimiento fundamentales:

- Un *modelo de transición*, que describe cómo cambia el entorno en el tiempo, tanto por las propias acciones del agente como por factores externos. Por ejemplo: "Si giro el volante hacia la derecha, el vehículo se moverá hacia la derecha."

- Un *modelo sensorial*, que indica cómo se reflejan los estados del mundo en las percepciones del agente. Por ejemplo: "Cuando el vehículo que tengo adelante frena, se iluminan sus luces rojas traseras."

Un agente que utiliza ambos modelos se llama agente basado en modelos. Estos le permiten realizar un seguimiento del estado del mundo, en la medida de lo posible dadas las limitaciones de sus sensores.

La estructura de un agente reactivo basado en modelos es la siguiente:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 52.

Podemos ver en la imagen que la percepción actual se combina con el estado interno anterior para generar la descripción actualizada del estado actual, basada en el modelo del agente de cómo el mundo funciona. Un programa del agente para esta estructura sería:

```
función AGENTE-REACTIVO-CON-ESTADO(percepción) devuelve una acción
    variables estáticas: estado, una descripción actual del estado del mundo
                         reglas, un conjunto de reglas condición-acción
                         acción, la acción más reciente, inicialmente ninguna

    estado ← ACTUALIZAR-ESTADO(estado, acción, percepción)
    regla ← REGLA-COINCIDENCIA(estado, reglas)
    acción ← REGLA-ACCIÓN[regla]
    devolver acción
```

Fuente: elaboración propia.

En este programa la función ACTUALIZAR-ESTADO es la responsable de crear una nueva descripción del estado interno. Además de interpretar la nueva percepción a partir del conocimiento existente sobre el estado, usa información relativa a la forma en la que evoluciona el mundo para saber más sobre las partes de este que no están visibles, para lograrlo debe conocer cuál es el efecto de las acciones del agente sobre el estado del mundo.

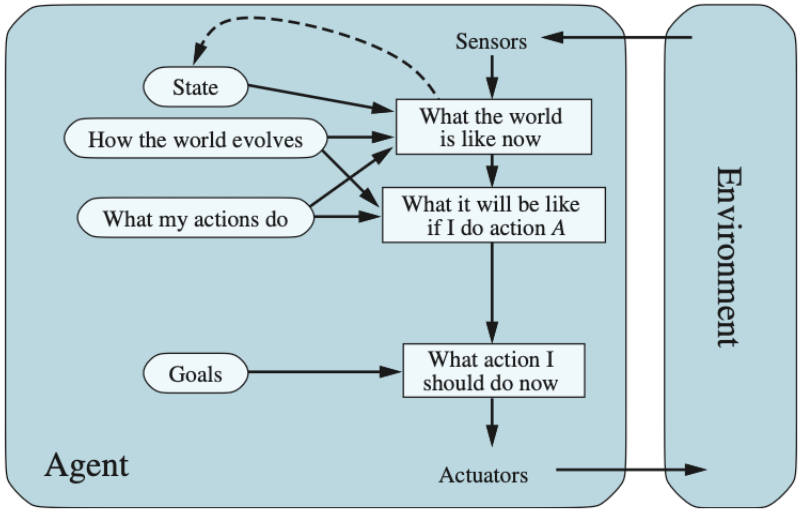
Rara vez es posible que el agente determine con exactitud el estado actual de un entorno parcialmente observable. En cambio, el recuadro titulado «cómo es el mundo ahora» de la imagen representa la «mejor suposición» del agente (o, a veces, las mejores suposiciones, si el agente baraja múltiples posibilidades). Por ejemplo, un taxi autónomo es posible que no pueda ver lo que hay adelante del camión que se ha detenido delante de él y solo pueda imaginar qué es lo que está causando el atasco. Por lo tanto, la incertidumbre sobre la situación actual puede ser inevitable, pero el agente aun así tiene que tomar una decisión.

Por eso, estos agentes son especialmente útiles en entornos parcialmente observables, donde el conocimiento es incompleto y deben tomar decisiones basadas en conjeturas razonables. En la práctica, los agentes basados en modelo son la base de muchos siste-

mas de control autónomo, robots industriales y asistentes inteligentes, que requieren mantener una representación del entorno para actuar de forma coherente incluso ante información limitada.

Agentes basados en objetivos

Conocer sobre el estado actual del entorno no siempre es suficiente para decidir qué hacer. Por ejemplo, en un cruce de calles, el taxi puede girar a la izquierda, girar a la derecha o seguir recto. La decisión correcta depende de a dónde quiere llegar. En otras palabras, además de una descripción del estado actual, el agente necesita algún tipo de información sobre su objetivo que describa situaciones que son deseables, por ejemplo, llegar a un destino particular. El programa del agente puede combinar esto con el modelo (la misma información que se utilizó en el agente reactivo basado en modelos) para elegir acciones que logren el objetivo. La estructura del agente basado en objetivos sería:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 54.

A veces, la selección de acciones basada en objetivos es sencilla. Por ejemplo, cuando la satisfacción de un objetivo es el resultado inmediato de una sola acción. En otras situaciones, puede ser más complicado, por ejemplo, cuando el agente tiene que considerar secuencias complejas para encontrar una forma de lograr el objetivo. Búsqueda y planificación son los subcampos de la IA dedicados a encontrar secuencias de acciones que logren los objetivos del agente.

La toma de decisiones de este tipo es bastante diferente de las reglas de condición–acción que describimos anteriormente, ya que implica la consideración del futuro, tanto "¿qué pasará si hago tal cosa?" como "¿eso me hará feliz?". En los diseños de agentes reactivos, esta información no está representada explícitamente porque las reglas integradas se asignan directamente desde percepciones a acciones. El agente reactivo frena cuando ve las luces de freno, punto. No tiene idea por qué. Un agente basado en objetivos frena cuando ve las luces de freno porque esa es la única acción que predice que logrará su objetivo de no chocar con otros autos.

Aunque el agente basado en objetivos parece menos eficiente, es más flexible porque el conocimiento que respalda sus decisiones se representa explícitamente y puede modificarse. Por ejemplo, el comportamiento de un agente basado en objetivos puede cambiarse fácilmente para ir a un destino diferente, simplemente especificando ese destino como el objetivo. En cambio, las reglas del agente reactivo para cuándo girar y cuándo seguir recto solo funcionarán para un único destino, todas deben ser reemplazadas para ir a un lugar nuevo.

Agentes basados en utilidad

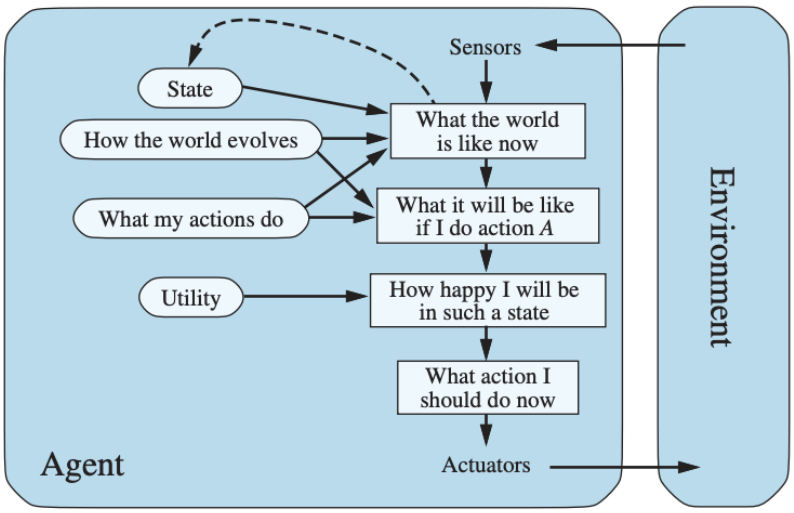
Los objetivos por sí solos no son suficientes para generar un comportamiento de calidad en la mayoría de los entornos. A modo de ejemplo, muchas secuencias de acciones llevarán al taxi a su destino (logrando así el objetivo), pero algunas son más rápidas, seguras, fiables o económicas que otras. Los objetivos solo proporcionan una distinción binaria cruda entre estados "felices" e "infelices". Una medida de rendimiento más general debería permitir una comparación de los diferentes resultados posibles y determinar qué tan deseables son. El concepto de utilidad expresa el grado de satisfacción o conveniencia asociado a cada estado del mundo.

Ya hemos visto que una medida de rendimiento asigna una puntuación a cualquier secuencia dada de estados del entorno, por lo que puede distinguir fácilmente entre formas más y menos deseables de llegar al destino del taxi. La función de utilidad de un agente es esencialmente una internalización de la medida de rendimiento. Siempre que la función de utilidad interna y la medida de rendimiento externa están de acuerdo, un agente que elige acciones para maximizar su utilidad será racional de acuerdo con la medida de rendimiento externa.

Al igual que los agentes basados en objetivos, un agente basado en la utilidad tiene muchas ventajas en términos de flexibilidad y aprendizaje. Además, existen dos tipos de casos en los cuales los objetivos son inadecuados, pero un agente basado en utilidad aún puede tomar decisiones racionales. Primero, cuando hay objetivos conflictivos, solo algunos de los cuales se pueden lograr (por ejemplo, velocidad y seguridad), la función de utilidad especifica la compensación adecuada. En segundo lugar, cuando hay varios objetivos que el agente puede lograr, ninguno de los cuales se puede alcanzar con certeza, la utilidad proporciona una forma en la que la probabilidad de éxito se puede sopesar con la importancia de los objetivos.

La observabilidad parcial y el no determinismo son omnipresentes en el mundo real y, por lo tanto, también lo es la toma de decisiones en condiciones de incertidumbre. Técnicamente hablando, un agente racional basado en utilidad elige la acción que maximiza la utilidad esperada de los resultados de la acción, es decir, la utilidad que el agente espera obtener, en promedio, dadas las probabilidades y utilidades de cada resultado. Un agente que posee una función de utilidad explícita puede tomar decisiones racionales con un algoritmo de propósito general que no depende de la función de utilidad específica que se maximiza. De esta manera, la definición "global" de racionalidad, que designa como racionales aquellas funciones del agente que tienen el mayor rendimiento, se convierte en una restricción "local" en los diseños de agentes racionales que se pueden expresar en un programa simple.

La estructura de un agente basado en utilidad sería como la siguiente:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 55.

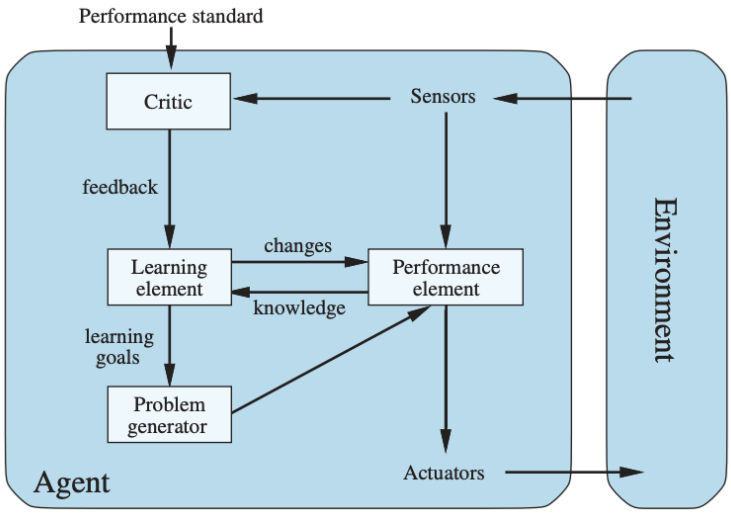
El agente combina su modelo del entorno con una función de utilidad que asigna un valor numérico a cada resultado posible. De esta manera puede comparar alternativas y seleccionar la acción que maximiza la utilidad esperada, es decir, aquella que promete el mejor balance entre beneficio y probabilidad de éxito. En la práctica, este tipo de agentes se emplea en sistemas de recomendación, algoritmos financieros, control de procesos, vehículos autónomos y toda clase de entornos donde la toma de decisiones debe equilibrar múltiples variables.

Agentes que aprenden

Hemos visto programas para agentes que poseen varios métodos para seleccionar acciones. Pero hasta ahora no hemos explicado cómo poner en marcha estos programas. Turing (1950), en su temprano y famoso artículo *Maquinaria computacional e Inteligencia*, consideró la idea de programar sus máquinas inteligentes a mano. Estimó cuánto tiempo podía llevar y concluyó que sería deseable utilizar algún método más rápido. Así, el método que propone es construir máquinas que aprendan y después enseñarles. Cualquier tipo de agente (basado en modelos, basado en objetivos, basado en utilizad, etc.) puede construirse como un agente que aprende.

Asimismo, el aprendizaje tiene otra ventaja: permite que el agente opere en entornos inicialmente desconocidos y llegar a ser más competente de lo que su conocimiento inicial por sí solo podría permitir.

Un agente que aprende puede dividirse en cuatro componentes conceptuales:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 56.

La distinción más importante es entre el elemento de aprendizaje, que es responsable de analizar los resultados obtenidos y modificar el programa del agente para hacerlo más eficaz, y el elemento de actuación, que es responsable de ejecutar las acciones y generar el comportamiento observable del agente. El elemento de actuación es lo que hemos visto antes como el agente completo: recibe percepciones y decide sobre las acciones. El elemento de aprendizaje usa la retroalimentación de las críticas sobre el comportamiento del agente y determina cómo el elemento de actuación debe modificarse para mejorar en el futuro.

El diseño del elemento de aprendizaje depende mucho del diseño del elemento de actuación. Cuando intentamos diseñar un agente que tenga capacidad de aprender, la primera pregunta a responder no es ¿cómo se puede enseñar a aprender?, sino ¿qué tipo de elemento de actuación necesita el agente para cumplir con su objetivo, una vez que haya aprendido cómo hacerlo? Dado un diseño para un agente, se pueden construir los mecanismos de aprendizaje necesarios para mejorar cada una de las partes del agente.

La crítica le dice al elemento de aprendizaje qué tan bien lo está haciendo el agente con respecto a un estándar externo. Es necesaria porque las percepciones en sí mismos no proporcionan indicación del éxito del agente. Por ejemplo, un programa de ajedrez podría recibir una percepción indicando que ha hecho jaque mate a su oponente, pero necesita un estándar de actuación para saber que esto es algo bueno, la percepción en sí misma no lo dice. Es importante que el estándar de rendimiento sea fijo. Conceptualmente, decimos que es externo porque el agente no debe poder modificarlo para que se ajuste a su propio comportamiento.

El último componente del agente que aprende es el generador de problemas. Es responsable de proponer acciones exploratorias que, aunque puedan ser menos óptimas a corto plazo, permitan adquirir nueva información y mejorar a largo plazo. Un ejemplo claro es el de un robot de limpieza que aprende qué zonas acumulan más suciedad y ajusta sus recorridos o un asistente digital que adapta su tono y estilo de respuesta a las preferencias del usuario.

El aprendizaje permite a los agentes operar en entornos desconocidos o cambiantes, desarrollando autonomía progresiva sin requerir una reprogramación constante. A medida que el agente acumula experiencia, puede corregir errores, refinar su modelo del entorno y mejorar la correspondencia entre sus acciones y los resultados deseados. Los distintos tipos de agentes descritos representan una escala evolutiva en la inteligencia artificial, desde comportamientos automáticos y rígidos hasta sistemas autónomos, adaptativos y racionales.

Comprender su estructura y funcionamiento nos permite entender cómo los sistemas inteligentes transforman información en acción y cómo logran comportarse de forma coherente y racional en entornos complejos.



Para finalizar, realice la **actividad 3** de este módulo.



Usted ya está en condiciones de realizar la Parte 1 de la Evaluación Integradora.

A partir de aquí, avancemos hacia el Módulo 3, donde vamos a profundizar en cómo los agentes basados en objetivos planifican y buscan soluciones efectivas. Aprenderemos cómo modelar problemas como espacios de estados, cómo definir estrategias de búsqueda y cómo aplicar algoritmos que permitan a los agentes razonar sobre el futuro y tomar decisiones informadas.

¡Continuemos!

Módulo 2 | Glosario

Acción: operación disponible para el agente en un estado; conecta estados en el espacio de estados.

Actuadores: mecanismos con los que el agente modifica el entorno.

Agente: entidad que percibe su entorno y actúa sobre él siguiendo una función de agente.

Autonomía: capacidad del agente de actuar sin intervención humana directa.

Consecuencialismo: principio según el cual la racionalidad de un agente se evalúa por los resultados de sus acciones y las consecuencias que generan.

Entorno: "mundo" con el que interactúa el agente; define percepciones, efectos y restricciones.

Entorno de trabajo (REAS): modelo que especifica un problema de IA mediante cuatro componentes: Rendimiento, Entorno, Actuadores y Sensores.

Función del agente: definición matemática que asocia cada secuencia de percepciones con una acción determinada. Representa el comportamiento ideal del agente.

Medida de rendimiento: criterio cuantitativo o cualitativo que permite evaluar el desempeño de un agente según los objetivos establecidos.

Percepto: entrada o información que el agente recibe a través de sus sensores en un momento determinado.

Programa del agente: implementación computacional que transforma los perceptos en acciones. Es la "mente" del agente dentro de una arquitectura física o virtual.

Racionalidad: selección de acciones que maximizan objetivos dado lo que el agente sabe y sus recursos.

Sensores: dispositivos o procesos que permiten al agente captar información del entorno, como cámaras, micrófonos o entradas de datos.

Utilidad: Valor numérico que mide el grado de satisfacción o conveniencia de un estado o resultado para un agente.

Módulo 2 | Actividades

ACTIVIDAD 1
Esquema REAS

Por cada uno de los siguientes agentes, construya un esquema REAS (Rendimiento, Entorno, Actuadores, Sensores):

- Aspiradora robot.
- Robot que juega al fútbol.
- Agente que permite comprar libros por internet.

ACTIVIDAD 2
Clasificación de entornos

Para cada uno de los siguientes casos, caracterice el entorno de trabajo considerando las seis categorías estudiadas:

Entorno de trabajo	Observable	Agentes	Determinista	Episódico	Estático	Discreto
Jugar al fútbol						
Jugar un partido de tenis						
Practicar tenis contra una pared						
Realizar un salto de altura						
Pujar por un artículo en una subasta						

ACTIVIDAD 3
Tipos de agentes

Considere un termostato sencillo que enciende la calefacción cuando la temperatura está al menos 3 grados por debajo del valor fijado y la apaga cuando se encuentra al menos 3 grados por encima. ¿Corresponde clasificar a este termostato como un agente reactivo simple, un agente reactivo basado en modelos o un agente basado en objetivos? Fundamente su respuesta.

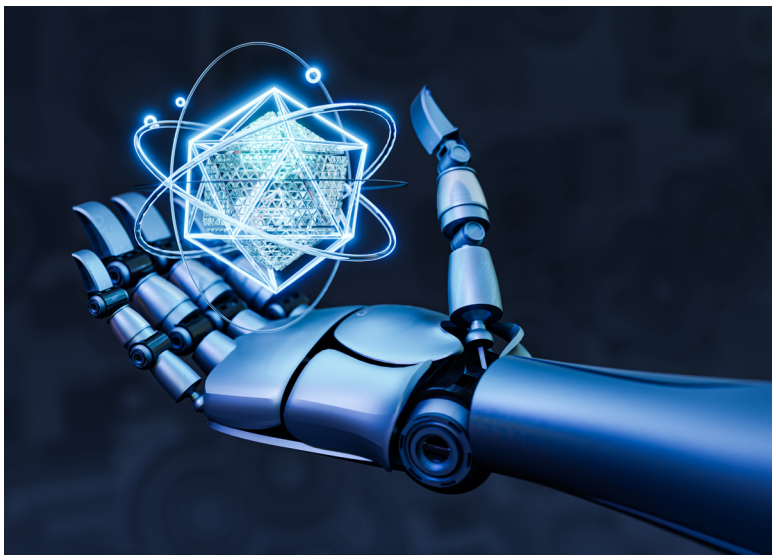
Módulo 3

Microobjetivos

- Describir los componentes formales de un problema de búsqueda para identificar las variables que intervienen en la construcción de modelos de resolución.
- Analizar los algoritmos de búsqueda en términos de completitud, optimalidad, complejidad temporal y espacial con el propósito de saber seleccionar estrategias adecuadas a las limitaciones de recursos y características del entorno.
- Integrar los conceptos de formulación, búsqueda y heurística para comprender cómo los agentes racionales planifican y eligen soluciones óptimas en distintos entornos.

Contenidos

RESOLUCIÓN DE PROBLEMAS



Fuente: https://www.freepik.es/

Contenidos

¡Llegamos al tercer módulo!

En los módulos anteriores aprendimos que la Inteligencia Artificial puede entenderse como el estudio de los agentes racionales, es decir, sistemas que perciben su entorno, razonan sobre él y actúan buscando maximizar su desempeño. Exploramos los distintos tipos de agentes y comprendimos cómo sus características estructurales los hacen más o menos adecuados para distintos tipos de entornos.

En este nuevo módulo damos un paso decisivo: pasamos del análisis de cómo un agente percibe y decide al estudio de cómo resuelve problemas. Cuando la acción correcta no es evidente de inmediato, el agente necesita planificar: pensar antes de actuar, explorar

alternativas y construir un camino hacia su objetivo. Este tipo de agente se llama agente resuelve problemas y el proceso computacional que lleva a cabo se llama búsqueda.

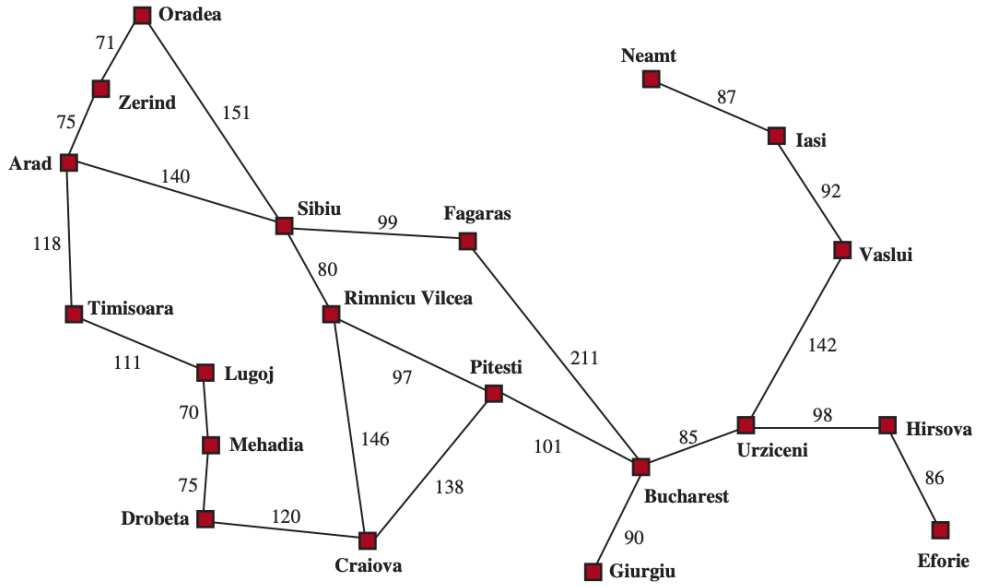
Vamos a aprender cómo un agente puede representar formalmente un problema, imaginar posibles secuencias de acciones y elegir aquella que lo conduzca a su meta de la manera más eficiente. Veremos, además, cómo se define un espacio de estados, qué significa formular un problema de manera abstracta y por qué el nivel de detalle adecuado puede determinar el éxito o el fracaso de la búsqueda.

¡Vamos!

El agente que resuelve problemas trabaja con una representación atómica del mundo: considera cada estado como una unidad completa, sin analizar su estructura interna. A lo largo de todo este módulo vamos a considerar que este mundo donde trabaja el agente es la representación de los entornos más simples: episódico, de un sólo agente, completamente observable, determinista, estático, discreto y conocido.

Imaginemos a un agente viajero en un recorrido por Rumania. Se encuentra en la ciudad de Arad y debe llegar a Bucarest, desde donde tiene un vuelo que sale al día siguiente. El agente observa los carteles y ve tres rutas posibles: una hacia Sibiu, otra hacia Timisoara y otra hacia Zerind. Ninguna de estas opciones es el objetivo, por lo que salvo que el agente esté familiarizado con la geografía de Rumania, no sabría qué dirección tomar.

Ahora supongamos que el agente tiene acceso a información sobre el mundo y dispone de un mapa del país, como el que vemos a continuación:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 64.

Con esta información, el agente puede iniciar un proceso racional de resolución de problemas compuesto por cuatro fases fundamentales:

1. **Formulación del objetivo**: define con claridad lo que desea lograr. En este caso, la meta es "llegar a Bucarest". Los objetivos permiten enfocar el comportamiento del agente, limitando las acciones posibles y organizando la búsqueda.

2. **Formulación del problema**: el agente abstrae los aspectos relevantes del mundo real, reduciéndolos a un modelo manejable. Decide que cada estado representa una ciudad, y cada acción, el desplazamiento entre ciudades conectadas por una ruta. Así, el único aspecto que cambia con cada acción es la ubicación actual.

3. **Búsqueda**: antes de actuar, el agente simula posibles secuencias de movimientos en su modelo, evaluando caminos hasta encontrar una solución, es decir, una secuencia de acciones que lo lleve desde el estado inicial (Arad) hasta el estado meta (Bucarest). Por ejemplo, puede descubrir la ruta Arad → Sibiu → Fagaras → Bucarest.

4. **Ejecución**: finalmente, el agente realiza las acciones planificadas en el mundo real, una a la vez, siguiendo el plan trazado hasta alcanzar el objetivo.

Cuando el entorno es totalmente observable, determinista y conocido, el resultado de cada acción está garantizado. En esos casos, el agente puede seguir la secuencia de pasos planeada sin necesidad de reconsiderar sus decisiones mientras actúa. A este tipo de comportamiento se lo denomina **bucle abierto** (open-loop), porque el agente no necesita retroalimentación durante la ejecución: la solución encontrada basta para llegar a la meta.

Sin embargo, si el modelo fuera inexacto o el entorno cambiante, por ejemplo, si una ruta estuviera cerrada o un desvío apareciera de improvisto, el agente necesitaría un sistema de **bucle cerrado** (closed-loop), capaz de percibir, evaluar y reajustar su comportamiento en función de la nueva información.

Problema de búsqueda y soluciones

Para que un agente pueda aplicar un método de resolución de problemas, debe contar con una descripción formal del entorno. Todo problema de búsqueda se define por los siguientes componentes:

› Espacio de estados: el conjunto de todos los posibles estados del mundo que el agente puede encontrarse. En el ejemplo, cada ciudad del mapa representa un estado.

› Estado inicial: el punto de partida del agente; en este caso, Arad.

› Conjunto de estados objetivo: los estados que cumplen la condición de éxito. Puede haber uno solo (Bucarest), un pequeño conjunto de estados objetivo alternativos (por ejemplo, "cualquier ciudad costera" o "todas las celdas limpias" en el mundo de las aspiradoras), o un objetivo definido por una propiedad que aplica a muchos estados (potencialmente un número infinito).

› Acciones posibles: el conjunto de movimientos o decisiones disponibles desde cada estado. Dicho de otra manera, dado un estado *s*, ACCIONES(*s*) retorna un conjunto finito de acciones que pueden ser ejecutadas en *s*. Cada una de estas acciones es aplicable en *s*. Por ejemplo:

$$ACCIONES(Arad) = \{IrSibiu, IrTimisoara, IrZerind\}$$

› Modelo de transición: una función que describe el resultado de cada acción. RESULTADO(*s*,*a*) devuelve el estado resultante de hacer la acción *a* en el estado *s*. Por ejemplo:

$$RESULTADO(Arad, IrZerind) = Zerind$$

› Función de costo de acción: podemos escribirla como COSTO-ACCION(s,a,s') y devuelve el costo numérico de aplicar la acción a en el estado s para alcanzar el estado s'. Un agente resuelve problemas utiliza esta función para representar su propia medida de rendimiento; por ejemplo, en el ejemplo de mapa, el costo de una acción puede ser la distancia recorrida o el tiempo de viaje.

Una secuencia de acciones forma un camino, y una solución es un camino desde el estado inicial hasta un estado objetivo. Asumimos que los costos de acción son aditivos; es decir, el costo total de un camino es la suma de los costos de acción individuales. Una solución óptima tiene el costo de camino más bajo entre todas las soluciones.

El espacio de estados puede representarse como un grafo en el que los nodos son los estados y las aristas representan las acciones disponibles. El mapa de Rumania que se encuentra más arriba puede interpretarse precisamente como ese grafo: las ciudades son nodos y las rutas son aristas etiquetadas con sus distancias.

La formulación de un problema consiste en construir un modelo abstracto que capture únicamente los aspectos relevantes de la situación real. En el ejemplo del viaje, la formulación "*estar en una ciudad y moverse hacia otra*" omite infinidad de detalles: el clima, el estado de la ruta, la música que se escucha dentro del vehículo, el tipo de combustible o el número de pasajeros. Todos estos factores son irrelevantes para el objetivo de "*encontrar un camino hacia Bucarest*".

Este proceso de simplificación se conoce como **abstracción**. El desafío consiste en elegir el nivel de detalle adecuado: demasiado detalle vuelve el problema inmanejable; demasiado poco puede volverlo irreal o inválido. Una buena abstracción mantiene tres propiedades esenciales:

1. Validez: cada acción abstracta debe corresponder a una acción realizable en el mundo real.
2. Utilidad: las acciones abstractas deben poder ejecutarse sin requerir planificación adicional.
3. Simplicidad: debe eliminar toda información innecesaria para la búsqueda.

Por ejemplo, la acción "conducir de Arad a Sibiu" puede representar un conjunto de acciones reales (acelerar, frenar, girar, etc.), pero basta con considerarla como una única transición porque, en el contexto del problema, su ejecución puede asumirse directa.

Énfasis

Sin la capacidad de construir y operar con abstracciones útiles, los agentes inteligentes se verían abrumados por la complejidad del mundo real. La abstracción es, por tanto, el primer paso hacia la racionalidad efectiva.

Una vez comprendido qué significa formular un problema de búsqueda y cómo abstraer los elementos esenciales del mundo real, es posible observar que la resolución de problemas abarca una enorme diversidad de entornos de trabajo. Algunos de ellos se han convertido en modelos de referencia para el estudio y comparación de algoritmos, mientras que otros surgen directamente de necesidades prácticas en entornos reales.

Podemos distinguir entonces entre **problemas estandarizados**, diseñados para ilustrar principios generales de búsqueda, y **problemas del mundo real**, donde las soluciones tienen aplicación práctica y suelen presentar una mayor complejidad. Veamos ambas categorías con algunos ejemplos.

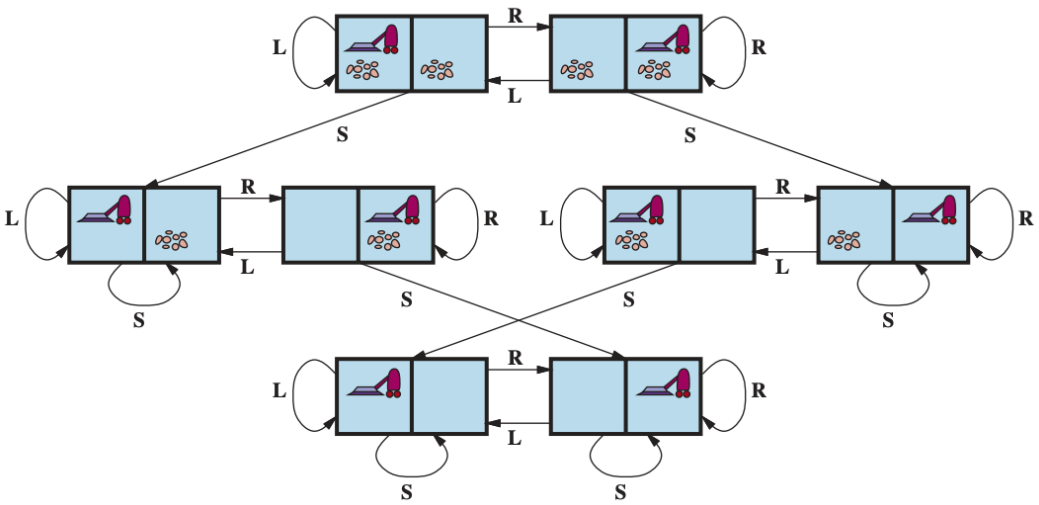
Problemas estandarizados

Énfasis

Los problemas estandarizados son entornos abstractos cuidadosamente definidos. Su propósito es facilitar el análisis, la experimentación y la evaluación de los algoritmos de búsqueda. Estos modelos se formulan con precisión matemática y permiten comparar de manera objetiva la eficiencia de distintas estrategias de resolución.

Un entorno clásico de estudio es el mundo de las celdas (grid world), en el que un agente se mueve dentro de una cuadrícula bidimensional compuesta por casillas adyacentes. El agente puede desplazarse hacia arriba, abajo, izquierda o derecha (y en algunos casos en diagonal), siempre que las casillas de destino no estén bloqueadas por obstáculos.

Cada celda puede contener objetos o elementos del entorno —como muros, suciedad o elementos que se puedan recoger—, y el objetivo suele consistir en alcanzar un punto específico o lograr un estado de limpieza total. Un ejemplo de este tipo de entorno es el mundo de las aspiradoras, ya presentado anteriormente, y podemos formular un problema para este caso de la siguiente manera:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 67.

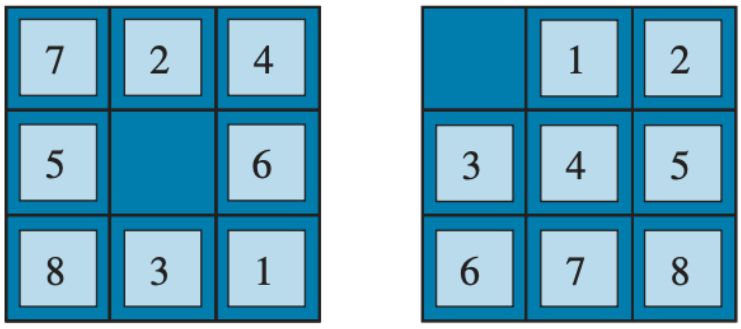
- Un estado del mundo dice qué objetos están en qué celdas. Para este caso, los objetos son el agente y cualquier suciedad. En la versión simplificada de dos cuadrados el agente puede estar en cualquiera de los dos, y cada cuadrado puede contener suciedad o no, por lo que habría $2 \times 2 \times 2 = 8$ estados posibles. En general, en un entorno de las aspiradoras con n celdas se tienen $n \times 2^n$ estados.
- El estado inicial puede ser cualquiera de los ocho.
- Las acciones disponibles dependen de la posición actual (por ejemplo, no se puede mover más a la izquierda si ya está en la primera celda).

- El modelo de transición indica el efecto de cada acción: aspirar limpia la celda, y los movimientos cambian la posición del agente.
- Los estados objetivo son aquellos en los que ambas celdas están limpias.
- Cada acción tiene un costo unitario, representando el esfuerzo o el tiempo de ejecución.

Este problema, aunque elemental, introduce de manera clara la idea de espacio de estados y permite visualizar cómo un agente explora ese espacio para hallar una secuencia de acciones que cumpla su objetivo.

A partir de este modelo, pueden formularse entornos de mayor complejidad, con múltiples celdas, distintos tipos de movimiento y objetos adicionales. Un ejemplo más desafiante es el rompecabezas Sokoban, donde el agente debe empujar cajas hacia posiciones de almacenamiento designadas, sin posibilidad de tirar de ellas. Cada movimiento altera el estado del tablero, y la cantidad de combinaciones posibles crece exponencialmente con el número de casillas y cajas. Resolver Sokoban implica planificar cuidadosamente, ya que una acción mal calculada puede dejar el problema sin solución.

Otro ejemplo clásico es el rompecabezas de fichas deslizantes, que puede adoptar distintas variantes. Una de las más conocidas es el 8-puzzle, cuya figura veremos a continuación, compuesto por un tablero de 3x3 con ocho piezas numeradas y un espacio vacío.



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 67.

En este entorno podemos formular el problema de la siguiente manera:

- Los estados describen la posición de cada ficha.
- El estado inicial puede ser cualquier disposición válida del tablero.
- Las acciones posibles dependen de la posición del espacio vacío: mover una ficha hacia arriba, abajo, izquierda o derecha.
- El modelo de transición define cómo cambian las posiciones tras cada movimiento.
- El estado objetivo se alcanza cuando las fichas están ordenadas según el patrón objetivo, en este caso con los números ordenados del 1 al 8 y el espacio vacío en la esquina inferior derecha.
- Cada acción tiene un costo unitario.

Este problema, aunque pequeño en apariencia, también encierra una gran complejidad. El número de configuraciones posibles del tablero es de *9!* aunque sólo la mitad de ellas son alcanzables desde cualquier estado inicial. Resolver el 8-puzzle requiere aplicar estrategias de búsqueda eficientes, ya que recorrer todo el espacio de estados sería inviable.

La formulación del problema abstrae detalles irrelevantes (como la forma exacta del tablero o el modo físico en que se mueven las fichas), conservando únicamente la información necesaria para representar los estados y sus transiciones.

Esta abstracción permite aplicar los mismos principios de búsqueda a otros entornos análogos, como el 15-puzzle (de 4x4) o el Rush Hour, en el que automóviles deben desplazarse en una cuadrícula hasta liberar un vehículo bloqueado.

Problemas reales

Énfasis

En contraste con los problemas estandarizados, los problemas del mundo real presentan una mayor complejidad debido a la cantidad de variables, restricciones y condiciones externas. Cada entorno real posee características particulares que dificultan la generalización.

El ejemplo del viaje en Rumania puede extenderse a situaciones más realistas, como los sistemas de navegación actuales que calculan rutas en mapas reales. Un agente de este tipo debe considerar factores adicionales como el tráfico, las obras viales, los costos de peaje y las condiciones meteorológicas. El objetivo no siempre es simplemente "llegar al destino", sino hacerlo de la manera más eficiente según distintos criterios: tiempo, distancia, consumo o seguridad.

El espacio de estados, en este caso, puede representarse como un grafo donde los nodos son intersecciones o ciudades, y las aristas son los caminos posibles. Cada arista tiene un costo que puede variar dinámicamente, por ejemplo, debido a embotellamientos, lo que obliga al agente a adaptar su plan incluso durante la ejecución. En la práctica, este tipo de problema combina búsqueda, planificación y aprendizaje adaptativo.

Otro ejemplo típico de problema de búsqueda real es la planificación de vuelos, donde el objetivo es encontrar un itinerario que conecte un punto de partida con un destino cumpliendo restricciones de horario, costo y disponibilidad.

En este tipo de entorno formulamos el problema de la siguiente manera:

- Los estados combinan la ubicación (aeropuerto), el tiempo actual y, a menudo, información histórica como los vuelos tomados o la categoría de tarifas.
- El estado inicial sería el aeropuerto de origen del usuario.
- Las acciones son los vuelos disponibles desde el aeropuerto actual, con distintas clases y horarios.
- El modelo de transición actualiza la ubicación y la hora tras tomar un vuelo.
- El estado objetivo es la ciudad de destino.
- El costo de acción depende de múltiples factores: precio del pasaje, duración del vuelo, escalas, esperas, comodidad o incluso la acumulación de puntos de viajero frecuente.

Estos problemas son típicos de los sistemas de recomendación de viajes o planificadores de rutas aéreas. Además, deben incluir planes de contingencia frente a demoras o cancelaciones, incorporando así la noción de incertidumbre y la necesidad de decisiones dinámicas.

En el ámbito logístico, los problemas de recorrido o de visita múltiple son comunes. El problema del viajante (Traveling Salesperson Problem, TSP) consiste en encontrar la ruta más corta que visite todas las ciudades de un conjunto dado y regrese al punto de partida. Aunque su enunciado es simple, la cantidad de combinaciones posibles crece factorialmente con el número de lugares a visitar, lo que lo convierte en uno de los problemas clásicos más difíciles de resolver de manera óptima.

Variantes del TSP se aplican en contextos reales como la planificación de rutas de transporte, el reparto de correo, el mantenimiento de redes eléctricas o la optimización de recorridos escolares.

El problema de disposición de circuitos (VLSI layout) es otro caso real de resolución mediante búsqueda. Aquí el objetivo es ubicar millones de componentes electrónicos en un chip de silicio minimizando el área total, los retardos de señal y las interferencias. El espacio de estados, en este caso, es gigantesco: cada componente puede colocarse en muchas posiciones posibles, y cada conexión introduce nuevas restricciones geométricas. Resolver este tipo de problema requiere dividirlo en subproblemas, como la distribución de celdas y el enrutamiento de canales, y aplicar algoritmos de búsqueda combinatoria y heurísticas de optimización.

La navegación robótica generaliza el problema de las rutas, permitiendo al agente generar sus propios caminos dentro de un espacio continuo. Un robot que se mueve en una superficie plana puede representarse en un espacio bidimensional, pero si tiene brazos, articulaciones o ruedas independientes, el número de dimensiones crece drásticamente: cada articulación o eje introduce una nueva variable de movimiento. El robot debe planificar trayectorias evitando obstáculos y compensando errores de percepción o control. Además, el entorno puede cambiar mientras se ejecutan las acciones, lo que obliga a integrar búsqueda, percepción y aprendizaje en tiempo real.

En la industria, el ensamblaje automatizado de piezas es un ejemplo clásico de aplicación práctica de algoritmos de búsqueda. El objetivo es determinar el orden correcto para ensamblar los componentes de un producto (como un motor o un dispositivo electrónico), minimizando tiempo y recursos. Si el orden elegido no es adecuado, puede ocurrir que una pieza impida colocar otra más adelante, obligando a desarmar lo ya construido. Cada acción debe verificarse para garantizar que sea físicamente realizable, lo que convierte este problema en una búsqueda geométrica de gran complejidad.

De manera análoga, en el campo de la biología computacional, el diseño de proteínas se formula como un problema de búsqueda en el que el agente busca una secuencia de aminoácidos que se pliegue en una forma tridimensional con propiedades específicas (por ejemplo, neutralizar una enfermedad). Estos problemas combinan el razonamiento estructural con la optimización numérica, y constituyen uno de los ejemplos más avanzados de resolución de problemas en entornos reales.

Hasta aquí, la formulación de problemas ha mostrado cómo el mismo marco conceptual: definir estados, acciones, transiciones, costos y metas, puede aplicarse a situaciones que van desde juegos de mesa hasta sistemas de navegación, robótica o diseño industrial.

En las próximas unidades del módulo, este marco se pondrá en acción al estudiar los algoritmos de búsqueda, que permiten a los agentes explorar sistemáticamente sus espacios de estados para encontrar soluciones óptimas o satisfactorias según el tipo de entorno.

Unidad 6: Algoritmos de búsqueda no informada

Ahora abordaremos el siguiente paso en el proceso de resolución de problemas: **cómo un agente explora ese espacio de estados** para encontrar una secuencia de acciones que lo lleve desde su situación inicial hasta el objetivo.

El conjunto de estrategias que estudiaremos en esta unidad se conoce como búsqueda no informada o búsqueda ciega, porque el agente no posee ningún conocimiento adicional sobre la distancia o cercanía al objetivo. Solo dispone de la información provista por la formulación del problema: los estados, las acciones y el modelo de transición.

Antes de entrar en los métodos específicos, necesitamos comprender cómo se representan y manipulan los caminos posibles que el agente evalúa.

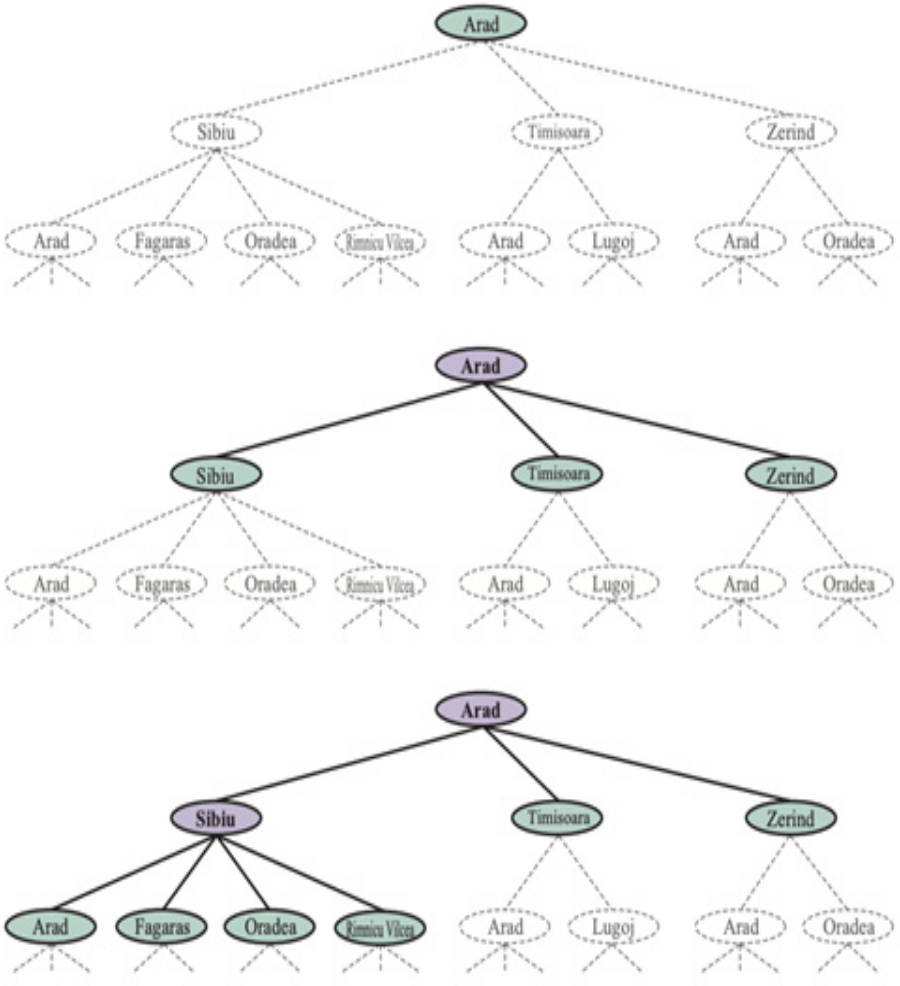
Un algoritmo de búsqueda toma como entrada un problema y produce como salida una solución o un aviso de fracaso. Para lograrlo, construye una estructura llamada árbol de búsqueda, que se superpone al grafo de estados definido por la formulación del problema.

En este árbol:

- Cada nodo representa un estado alcanzable del entorno.
- Las ramas representan las acciones que permiten pasar de un estado a otro.
- La raíz del árbol corresponde al estado inicial del problema.

El grafo de estados describe todas las posibilidades del mundo, mientras que el árbol de búsqueda representa las rutas exploradas por el agente en su intento de alcanzar el objetivo. Es importante distinguir ambos conceptos: el espacio de estados es una descripción general del entorno, mientras que el árbol de búsqueda es la exploración concreta que el agente realiza dentro de ese espacio.

Veamos en la figura siguiente los primeros pasos para encontrar un camino desde Arad a Bucarest:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 71.

El nodo raíz de este árbol de búsqueda es el estado inicial, *Arad*. Podemos expandir ese nodo considerando las ACCIONES disponibles para cada estado. Para ello, utilizamos a función RESULTADO para ver a dónde nos llevan esas acciones, y generamos un nuevo nodo (llamado nodo hijo o sucesor) para cada uno de los estados resultantes. Para cada nodo hijo, *Arad* sería el nodo padre.

Los tres nodos hijos generados son Sibiu, Timișoara y Zerind, ahora debemos elegir uno de ellos para continuar con el proceso. Esta es la esencia de la búsqueda, elegir una opción y dejar las otras de lado para más tarde. Supongamos que elegimos expandir Sibiu primero. La figura que vimos más arriba muestra el resultado: un conjunto de 4 nodos sin expandir (delineados en negrita). Llamamos a esto la frontera del árbol de búsqueda. Decimos que cualquier estado que ha tenido un nodo generado para él ha sido alcanzado (haya o no ese nodo sido expandido).

La frontera separa en dos regiones al grafo de estados, una región interior donde todos los estados han sido expandidos, y una exterior donde los estados aún no han sido alcanzados.

El proceso fundamental en toda búsqueda es la expansión de un nodo. Expandir un nodo significa aplicar todas las acciones posibles al estado que representa, generando nuevos nodos hijos o sucesores. Cada sucesor almacena la información necesaria para reconstruir la ruta seguida. Un nodo del árbol es representado por una estructura de datos con cuatro componentes fundamentales:

1. Estado: el punto del espacio de estados que representa.
2. Padre: el nodo desde el cual se llegó al estado actual.
3. Acción: la operación aplicada al estado del padre para generar este nodo.
4. Costo del camino: la suma de los costos de todas las acciones ejecutadas desde el inicio.

Esta información permite reconstruir el camino completo hacia la meta una vez que se encuentra una solución: basta con seguir los punteros de los nodos padres hasta llegar a la raíz. La expansión continúa hasta que uno de los nodos generados corresponde a un estado objetivo o hasta que no queden nodos en la frontera, lo cual indica que no existe solución.

La clave de cualquier estrategia de búsqueda es decidir qué nodo expandir a continuación. Esa elección determina la eficiencia, el orden de exploración y la calidad de la solución. Un enfoque general consiste en seleccionar el nodo con el menor valor de una función de evaluación *f(n)*. Según la definición de *f(n)*, surgen distintos algoritmos de búsqueda.

Esta idea se denomina búsqueda del mejor primero (best-first search), representada en el pseudocódigo que veremos a continuación:

Código

```
función BÚSQUEDA-MEJOR-PRIMERO(problema,f) devuelve un nodo solución o fallo
    nodo ← NODO(ESTADO=problema.INICIAL)
    frontera ← una cola de prioridad ordenada por f, con nodo como elemento
    alcanzado ← una tabla de búsqueda, con una entrada con clave problema.INICIAL
                y valor nodo

    mientras no VACÍO(frontera) hacer
        nodo ← POP(frontera)
        si problema.ES-OBJETIVO(nodo.ESTADO) entonces devolver nodo
        por cada hijo en EXPANDIR(problema, nodo) hacer
            s ← hijo.ESTADO
            si s no está en alcanzado o hijo.COSTO-CAMINO < alcanzado[s].COSTO-CAMINO
            entonces
                alcanzado[s] ← hijo
                agregar hijo a frontera
    devolver fallo

función EXPANDIR(problema,nodo) devuelve nodos
    s ← nodo.ESTADO
    por cada acción en problema.ACCCIONES(s) hacer
        s' ← problema.RESULTADO(s,acción)
        costo ← nodo.COSTO-CAMINO + problema.COSTO-ACCION(s,acción,s')
        devolver NODO(ESTADO=s', PADRE=nodo, ACCION=accion, COSTO-CAMINO=costo)
```

Fuente: elaboración propia

En cada iteración:

- Se selecciona el nodo de la frontera con el menor valor de *f(n)*.
- Si ese nodo representa un estado objetivo, se devuelve como solución.
- Si no, se expande generando sus hijos.
- Cada hijo se añade a la frontera si nunca ha sido alcanzado, o se actualiza si se encontró un camino más corto hacia él.

Esta estructura permite aplicar el mismo marco algorítmico a una gran variedad de estrategias. Por ejemplo, si *f(n)* mide la profundidad del nodo, el resultado es la búsqueda en anchura; si mide el costo acumulado, obtenemos la búsqueda de costo uniforme; y si combina el costo y una estimación de distancia al objetivo, se convierte en una búsqueda informada (que veremos más adelante).

A su vez necesitamos una estructura de datos para almacenar la frontera. La elección adecuada es una cola de algún tipo, ya que las operaciones en una frontera son:

- VACÍO(*frontera*): devuelve verdadero solo si no hay nodos en la frontera.
- POP(*frontera*): elimina el nodo superior de la frontera y lo devuelve.
- TOP(*frontera*): devuelve (pero no elimina) el nodo superior de la frontera.
- ADD(*nodo,frontera)*: inserta el nodo en un lugar apropiado en la cola.

Se utilizan tres tipos de colas en los algoritmos de búsqueda:

- Cola de prioridad: primero extrae el nodo con el costo mínimo de acuerdo con alguna función de evaluación, *f*. Se utiliza en la búsqueda del mejor primero y en algoritmos informados.
- Cola FIFO (First-In, First-Out): el primer nodo agregado es el primero en ser extraído; veremos que se utiliza en la búsqueda en anchura.
- Cola LIFO (Last-In, First-Out o pila): el último nodo agregado es el primero en extraerse; veremos que se usa en la búsqueda en profundidad.

Redundancia y ciclos

En un espacio de estados pueden existir múltiples caminos hacia el mismo punto. Por ejemplo, en el mapa de Rumania es posible llegar a Sibiu desde Arad de dos maneras: directamente (140 millas) o pasando por Zerind y Oradea (297 millas). El segundo camino es redundante, ya que lleva al mismo estado con un costo mayor. Si el algoritmo no detecta este tipo de repeticiones, puede desperdiciar recursos explorando rutas innecesarias o incluso caer en ciclos infinitos.

Existen tres formas de manejar la redundancia:

- Memorizar todos los estados alcanzados: se conserva la mejor ruta hacia cada uno y se ignoran las peores. Este enfoque es eficiente cuando la memoria lo permite.
- Ignorar el pasado: útil en problemas donde no existen rutas alternativas hacia un mismo estado, aunque con el riesgo de repetir trabajo.
- Detectar ciclos locales: verificando si el estado actual ya apareció entre los ancestros del nodo. Este método usa poca memoria, pero solo evita repeticiones cortas.

En la práctica, los algoritmos equilibran estos métodos según la naturaleza del problema y las limitaciones computacionales.

Evaluación del rendimiento de los algoritmos de búsqueda

Antes de profundizar en los distintos tipos de algoritmos de búsqueda no informada, consideremos los criterios que podemos utilizar para elegir entre ellos. Podemos evaluar el rendimiento de un algoritmo de las siguientes formas:

- Completitud: garantiza que el algoritmo encontrará una solución si existe o reportará el fallo correctamente si no la hay.
- Optimalidad de costo: asegura que la solución hallada tendrá el menor costo posible entre todas.
- Complejidad temporal: mide el número de pasos o de estados explorados hasta encontrar la solución.
- Complejidad espacial: evalúa la cantidad de memoria necesaria para almacenar nodos y estados intermedios.

Estos criterios permiten juzgar la eficiencia y aplicabilidad de cada método. Por ejemplo, un algoritmo puede ser completo, pero consumir demasiada memoria, o ser muy rápido pero incapaz de garantizar la mejor solución.

El rendimiento de la búsqueda depende, además, de las propiedades del problema:

- b, el factor de ramificación, que indica cuántos sucesores promedio tiene cada nodo;
- d, la profundidad de la solución óptima; y
- m, la longitud máxima de los caminos posibles.

Estas variables definen el tamaño del espacio de búsqueda y, por tanto, el esfuerzo necesario para recorrerlo.

Ya hemos visto la teoría necesaria para estudiar las distintas estrategias de búsqueda, es decir, los modos en que un agente puede recorrer su árbol de posibilidades para encontrar una solución.

Empecemos por los algoritmos de búsqueda no informada, aquellos en los que el agente carece de información sobre la cercanía al objetivo. Estos métodos se basan únicamente en la estructura del espacio de estados y en la exploración sistemática de caminos posibles. Aunque pueden parecer simples, constituyen la base de todas las técnicas de búsqueda más avanzadas y son esenciales para comprender los algoritmos heurísticos que se verán en la próxima unidad.

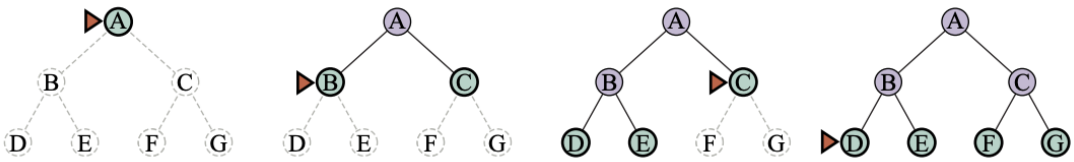
Por ejemplo, volvamos a nuestro agente en Arad con el objetivo de llegar a Bucarest. Un agente no informado sin conocimiento de la geografía rumana no tiene idea de si ir a Zerind o Sibiu es un mejor primer paso. En contraste, un agente informado (que veremos más adelante) conoce la ubicación de cada ciudad y sabe que Sibiu está mucho más cerca de Bucarest y, por lo tanto, es más probable que esté en el camino más corto.

Búsqueda en anchura (Breadth-First Search, BFS)

Este tipo de algoritmo explora el árbol de búsqueda nivel por nivel, comenzando desde el estado inicial y expandiendo todos los nodos de una misma profundidad antes de pasar al siguiente nivel. En términos prácticos:

1. Se expande primero la raíz (nivel 0).
2. Luego, todos los nodos hijos (nivel 1).
3. Posteriormente, los hijos de los hijos (nivel 2), y así sucesivamente.

Veamos un ejemplo de este proceso en un árbol binario, donde los nodos se expanden de izquierda a derecha a medida que se desciende en profundidad:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 76.

Podríamos implementar una búsqueda en anchura llamando a la función BÚSQUEDA-MEJOR-PRIMERO donde la función de evaluación *f(n)* es la profundidad del nodo, es decir, el número de acciones que se necesitan para llegar al nodo. Sin embargo, si utilizamos una cola FIFO en lugar de una cola de prioridad el algoritmo será más rápido y nos dará el orden correcto de los nodos: los nodos nuevos (que siempre son más profundos que sus padres) van al final de la cola, y los nodos antiguos, que son menos profundos que los nodos nuevos, se expanden primero.

Además, *alcanzado* puede ser un conjunto de estados en lugar de una asignación de estados a nodos, porque una vez que hemos alcanzado un estado, nunca vamos a poder encontrar un camino mejor hacia ese estado. Eso también significa que podemos hacer una prueba temprana de objetivo comprobando si un nodo es una solución tan pronto como se genera, en lugar de la prueba tardía de objetivo que utiliza la búsqueda del mejor primero, esperando hasta que un nodo se extraiga de la cola.

Veamos cómo sería el pseudocódigo con estas mejoras:

```
función BÚSQUEDA-EN-ANCHURA(problema) devuelve un nodo solución o fallo
    nodo ← NODO(ESTADO=problema.INICIAL)
    si problema.ES-OBJETIVO(nodo.ESTADO) entonces devolver nodo
    frontera ← una cola FIFO, con nodo como elemento
    alcanzado ← {problema.INICIAL}
    mientras no VACÍO(frontera) hacer
        nodo ← POP(frontera)
        por cada hijo en EXPANDIR(problema, nodo) hacer
            s ← hijo.ESTADO
            si problema.ES-OBJETIVO(s) entonces devolver hijo
            si s no está en alcanzado entonces
                agregar s a alcanzado
                agregar hijo a frontera
    devolver fallo
```

Fuente: elaboración propia

La búsqueda en anchura siempre encuentra una solución con un número mínimo de acciones, porque cuando está generando nodos a profundidad *d*, ya ha generado todos los nodos a profundidad *d-1*, por lo que, si uno de ellos fuera una solución, ya se habría encontrado. Eso significa que es óptimo en costo para problemas donde todas las acciones tienen el mismo costo, pero no para problemas que no tienen esa propiedad. De todas formas, sigue siendo completo: si existe una solución, la va a encontrar.

En términos de tiempo y espacio, imaginemos buscar en un árbol uniforme donde cada estado tiene *b* sucesores. La raíz del árbol de búsqueda genera *b* nodos, cada uno de los cuales genera *b* nodos más, para un total de b^2 en el segundo nivel. Cada uno de estos genera *b* nodos más, produciendo b^3 nodos en el tercer nivel, y así sucesivamente. Ahora supongamos que la solución está a profundidad *d*. Entonces, el número total de nodos generados es:

$$1 + b + b^2 + b^3 + \cdots + b^d = O(b^d)$$

Todos los nodos permanecen en memoria, por lo que tanto la complejidad de tiempo como la de espacio son $O(b^d)$. Como un ejemplo típico del mundo real, consideremos un problema con factor de ramificación *b=10*, velocidad de procesamiento de 1 millón de nodos/segundo y requerimiento de memoria de 1 Kbyte/nodo. Una búsqueda a profundidad *d=10* tomaría menos de 3 horas, pero requeriría 10 terabytes de memoria. Los requerimientos de memoria son un problema mayor para este tipo de algoritmos que el tiempo de ejecución. Igualmente, el tiempo sigue siendo un factor importante. A profundidad *d=14* incluso con memoria infinita, la búsqueda tardaría 3.5 años.

En general, los problemas de búsqueda de complejidad exponencial no pueden resolverse mediante una búsqueda no informada, salvo en los casos más sencillos.

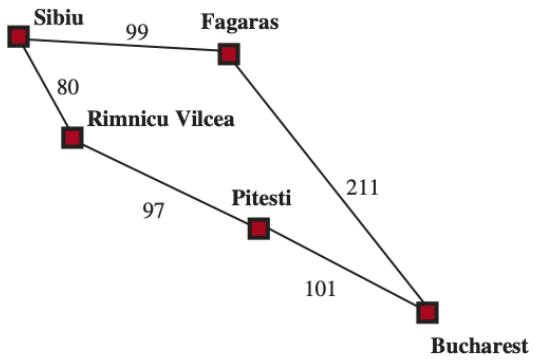
Búsqueda de costo uniforme

Cuando las acciones tienen diferentes costos, una opción obvia es usar la búsqueda del mejor primero donde la función de evaluación es el costo del camino desde la raíz hasta el nodo actual. Esto se llama Algoritmo de Dijkstra por la comunidad de la informática teórica, y búsqueda de costo uniforme por la comunidad de la IA.

La idea es que mientras la búsqueda en anchura se extiende en ondas de profundidad uniforme—primero profundidad 1, luego profundidad 2, y así sucesivamente—la búsqueda de costo uniforme se extiende en ondas de costo de ruta uniforme. El algoritmo se puede implementar como una llamada a BÚSQUEDA-MEJOR-PRIMERO con COSTO-CAMINO como la función de evaluación, como mostramos a continuación:

```
función BÚSQUEDA-COSTO-UNIFORME(problema) devuelve un nodo solución o fallo
    devolver BÚSQUEDA-MEJOR-PRIMERO(problema,COSTO-CAMINO)
```

Consideremos la siguiente figura, donde el problema es llegar desde Sibiu a Bucarest:



RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 78.

Los nodos sucesores de Sibiu son Rimnicu Vilcea y Fagaras, con un costo de 80 y 99 respectivamente. Expandimos el nodo de menor costo, Rimnicu Vilcea, agregando Pitesti con un costo de *80+97=177*. Ahora el nodo de menor costo es Fagaras, por lo que se expande, agregando Bucarest con un costo de *99+211=310*. Bucarest es el objetivo, pero el algoritmo prueba los objetivos solo cuando expande un nodo, no cuando genera un nodo, por lo que aún no ha detectado que este es un camino hacia el objetivo.

El algoritmo continúa, eligiendo Pitesti para expandirlo y agregando un segundo camino a Bucarest con un costo de *80+97+101=278*. Tiene un costo menor, por lo que reemplaza al camino anterior en *alcanzado* y es agregado a la *frontera*. Resulta que este nodo ahora tiene el costo más bajo, por lo que se considera a continuación, se define que es un objetivo y se devuelve. Tengamos en cuenta que si hubiéramos comprobado si hay un objetivo al generar un nodo en lugar de al expandir el nodo de menor costo, entonces habríamos devuelto un camino de mayor costo (el que pasa por Fagaras).

La complejidad de este algoritmo se caracteriza en términos de C^*, el costo de la solución óptima, y ϵ un límite inferior en el costo de cada acción, con $\epsilon > 0$. Entonces, la complejidad temporal y espacial en el peor escenario es $O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$ que puede ser mucho mayor que b^d. Esto se debe a que la búsqueda de costo uniforme puede explorar grandes árboles de acciones con bajos costos antes de explorar rutas que implican una acción de alto costo y quizás útil.

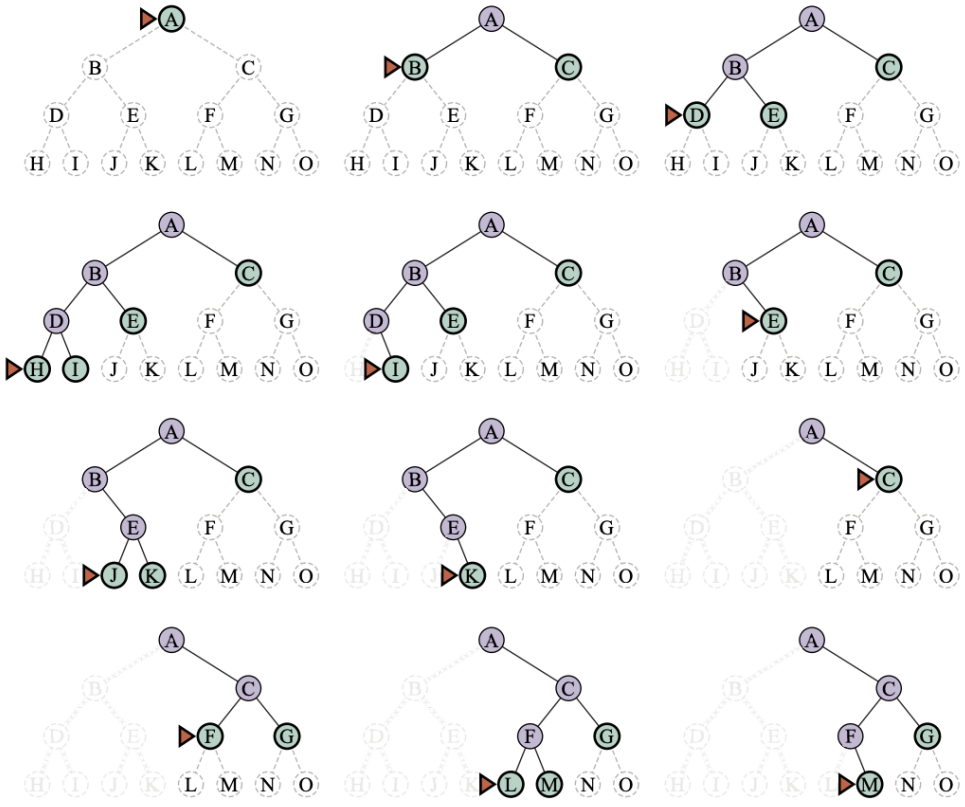
La búsqueda de costo uniforme es **completa** y **óptima**, porque la primera solución que encuentre tendrá un coste al menos tan bajo como el coste de cualquier otro nodo en la frontera**.**

Búsqueda en profundidad (Depth-First Search, DFS)

La búsqueda en profundidad sigue una estrategia completamente opuesta a la anterior. En lugar de explorar todos los nodos de un nivel antes de pasar al siguiente, se adentra en una sola rama del árbol hasta llegar al final, retrocediendo solo cuando no puede continuar.

El algoritmo se implementa mediante una pila (LIFO), lo que garantiza que el último nodo generado sea el primero en expandirse. De este modo, la búsqueda profundiza tanto como sea posible antes de retroceder a niveles previos y explorar nuevas alternativas.

Una búsqueda de este tipo sería:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 79.

El agente comienza en el nodo raíz, sigue una rama descendente hasta que alcanza un nodo sin sucesores, y luego retrocede al último nodo con opciones no exploradas para continuar por otra rama. La búsqueda en profundidad no es óptima en cuanto a costos; devuelve la primera solución que encuentra, incluso si no es la más barata.

Para espacios de estados finitos que son árboles, es eficiente y completo; para espacios de estados acíclicos, puede terminar expandiendo el mismo estado muchas veces a través de diferentes rutas, pero (eventualmente) explorará sistemáticamente todo el espacio.

En espacios de estado cíclicos, puede quedarse atascado en un bucle infinito; por lo tan-

to, algunas implementaciones de la búsqueda en profundidad comprueban cada nuevo nodo en busca de ciclos. Finalmente, en espacios de estado infinitos, la búsqueda en profundidad no es sistemática: puede quedarse atascada en un camino infinito, incluso si no hay ciclos. Por lo tanto, la búsqueda en profundidad es incompleta. Sin embargo, este algoritmo es útil para los problemas en los que una búsqueda en forma de árbol es factible, porque tiene bajo consumo de memoria, sólo necesita almacenar el camino actual y los nodos sucesores inmediatos.

Para un espacio de estados finito en forma de árbol como el que mostramos más arriba, una búsqueda en profundidad similar a un árbol tiene una complejidad temporal proporcional al número de estados y una complejidad espacial de solo *O (bm)* donde *b* es el factor de ramificación y *m* es la profundidad máxima del árbol. Algunos problemas que requerirían exabytes de memoria con la búsqueda en anchura se pueden manejar con solo kilobytes usando la búsqueda en profundidad.

Búsqueda con profundidad limitada (Depth-Limited Search, DLS)

Para evitar que la búsqueda en profundidad divague por un camino infinito, podemos usar la búsqueda con profundidad límitada, una versión de la búsqueda en profundidad en la que proporcionamos un límite de profundidad ℓ , y tratamos a todos los nodos en profundidad ℓ como si no tuvieran sucesores. La complejidad temporal es $O(b^{\ell})$ y la complejidad espacial es $O(b\ell)$. Desafortunadamente, si hacemos una mala elección para ℓ el algoritmo no logrará alcanzar la solución, haciéndolo incompleto nuevamente.

A veces, se puede elegir un buen límite de profundidad basándonos en el conocimiento del problema. Por ejemplo, en el mapa de Rumania hay 20 ciudades. Por lo tanto, $\ell = 19$ es un límite válido. Pero si estudiáramos el mapa cuidadosamente, descubriríamos que se puede llegar a cualquier ciudad desde cualquier otra ciudad en un máximo de 9 acciones. Este número, conocido como el diámetro del grafo de espacio de estados, nos da un mejor límite de profundidad, lo que conduce a una búsqueda de profundidad limitada más eficiente. Sin embargo, para la mayoría de los problemas no conoceremos un buen límite de profundidad hasta que hayamos resuelto el problema.

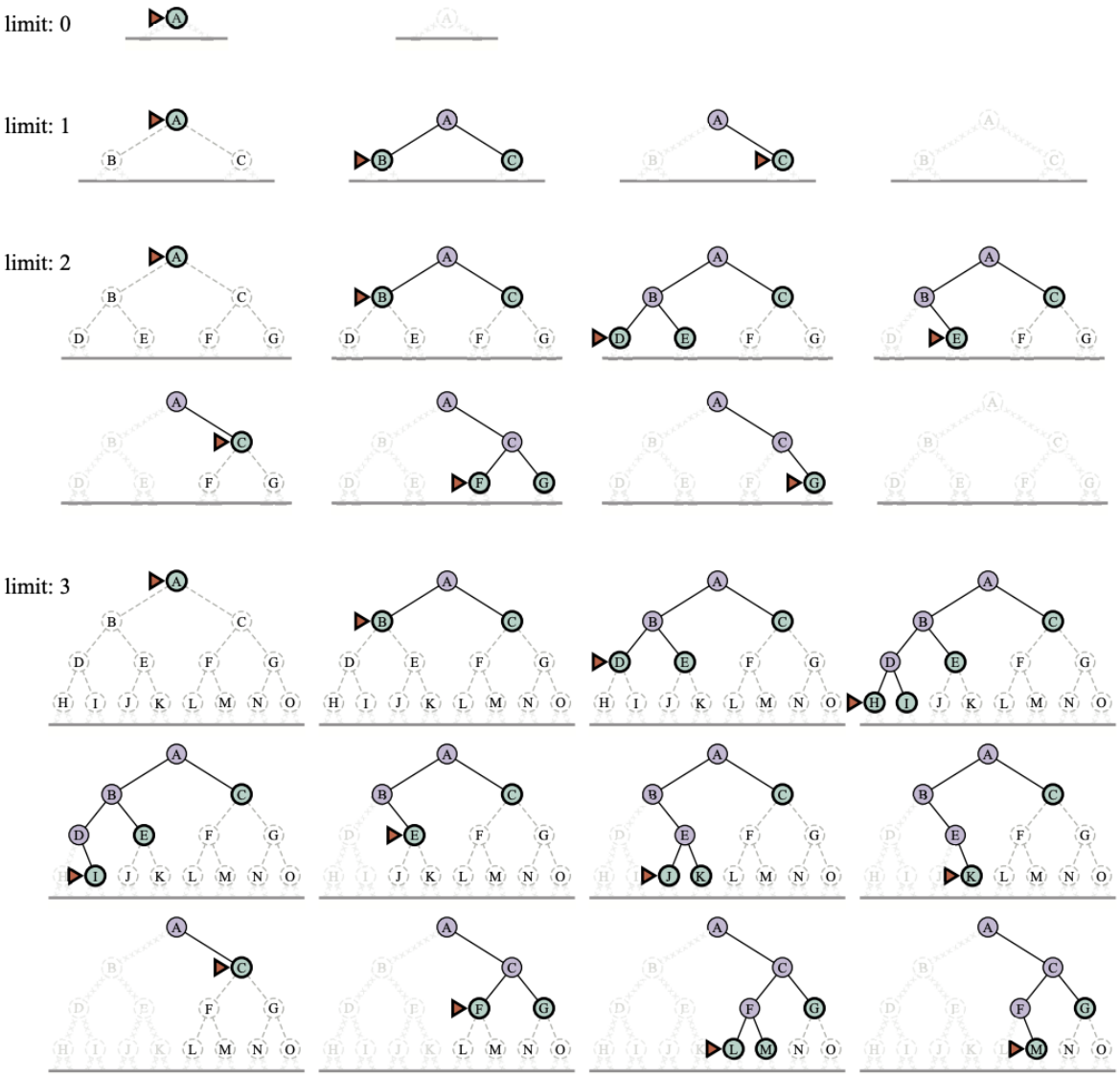
Búsqueda con profundización iterativa (Iterative Deepening Search, IDS)

La búsqueda con profundización iterativa resuelve el problema de elegir un buen valor para ℓ probando todos los valores: primero 0, luego 1, luego 2, y así sucesivamente, hasta que se encuentre una solución.

Este algoritmo combina muchos de los beneficios de la búsqueda en profundidad y la búsqueda en anchura. Al igual que la búsqueda en profundidad, sus requisitos de memoria son modestos: *O(bd)* cuando hay una solución, o *O(bm)* en espacios de estados finitos sin solución. Al igual que la búsqueda en anchura, es óptima para problemas donde todas las acciones tienen el mismo costo, y es completa en espacios de estados acíclicos finitos, o en cualquier espacio de estados finito cuando verificamos nodos para ciclos en todo el camino.

La complejidad temporal es $O(b^d)$ cuando hay una solución, o $O(b^m)$ cuando no la hay. Cada iteración de la búsqueda genera un nuevo nivel, de la misma manera que lo hace la búsqueda en anchura, pero esta lo hace almacenando todos los nodos en memoria, mientras que la iterativa lo hace repitiendo los niveles anteriores, ahorrando así memoria a costa de más tiempo.

Veamos un ejemplo de cuatro iteraciones de búsqueda con profundización iterativa en un árbol binario, donde la solución se encuentra en la cuarta iteración.



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 82.

En cada iteración, los nodos se regeneran desde la raíz, pero esto no implica un costo excesivo: la mayoría de los nodos se encuentran en los niveles más profundos, por lo que los niveles superiores se repiten pocas veces.

Este algoritmo es una de las estrategias más utilizadas cuando no se conoce la profundidad de la solución y el espacio de estados es demasiado grande para almacenar todos los nodos simultáneamente.

Hasta ahora hemos estudiado estrategias de búsqueda no informada, en las cuales el agente no dispone de ninguna guía adicional sobre la cercanía al objetivo. Estas estrategias, aunque garantizan resultados correctos, suelen ser costosas en tiempo y memoria, ya que exploran indiscriminadamente grandes porciones del espacio de estados.

En esta unidad ingresamos a un nuevo nivel de razonamiento: las búsquedas informadas o heurísticas, en las que el agente utiliza conocimiento adicional sobre el dominio para orientar su exploración hacia los caminos más prometedores.

Énfasis

Una estrategia de búsqueda informada es aquella que, además de la información del problema (estados, acciones, costos y metas), emplea pistas o estimaciones que le permiten decidir qué caminos parecen conducir más rápidamente al objetivo. Estas estimaciones se conocen como heurísticas, y constituyen una de las herramientas fundamentales de la inteligencia artificial contemporánea.

Una función heurística, denotada como *h(n)*, asigna a cada nodo *n* un valor numérico que representa una estimación del costo mínimo restante para alcanzar un estado objetivo desde el estado actual. El valor de *h(n)* no proviene de la formulación formal del problema, sino de conocimiento adicional sobre el entorno o de cálculos aproximados que permiten anticipar la dificultad restante. Por ejemplo, en un problema de búsqueda de rutas, una heurística natural es la distancia en línea recta entre la posición actual y el destino. Aunque el camino real pueda ser más largo por curvas, tráfico o desvíos, esta estimación es útil porque ofrece una medida aproximada de cuán lejos está el agente de la meta.

Veamos la representación de esta heurística:

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 85.

Búsqueda voraz del mejor primero (Greedy Best-First Search)

Este algoritmo siempre selecciona para expandir el nodo con el menor valor de *h(n)*, es decir, aquel que parece estar más cerca del objetivo según la heurística. La función de evaluación es:

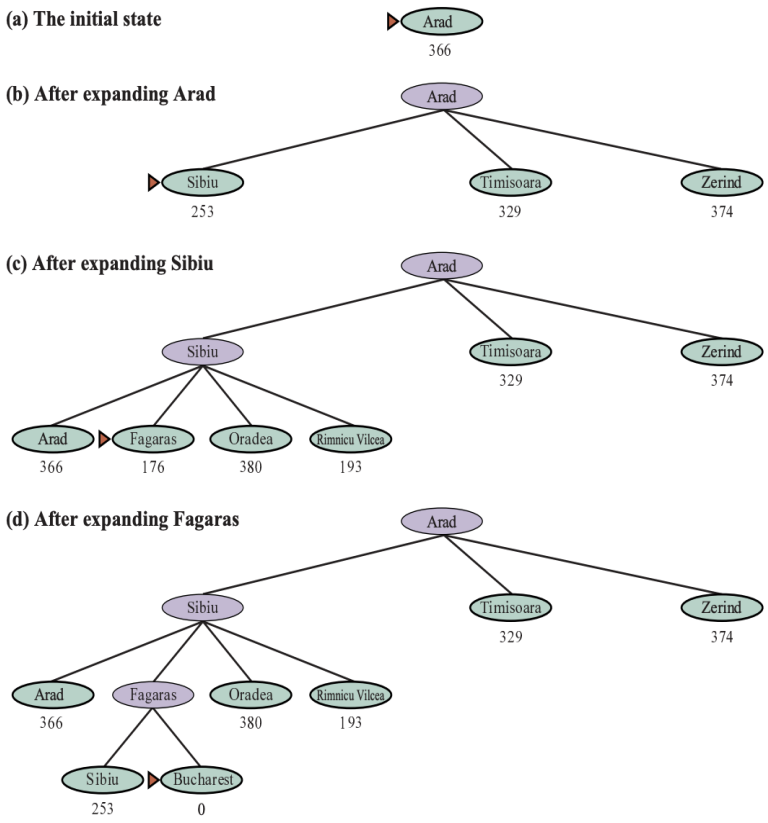
$$f(n) = h(n)$$

El proceso se denomina "voraz" porque en cada paso el agente elige el camino que parece más prometedor en el corto plazo, sin considerar el costo acumulado de lo recorrido. En el ejemplo del mapa, si el agente parte desde Arad, comparará las distancias en línea recta hacia Bucarest de sus vecinos:

- Sibiu (253)
- Timisoara (329)
- Zerind (374)

Como Sibiu tiene la menor distancia estimada, será el primer nodo expandido. Luego, desde Sibiu, los nuevos vecinos evaluados son Fagaras (176) y Rimnicu Vilcea (193). El agente elegirá Fagaras, que parece más próximo al objetivo. Finalmente, desde Fagaras se alcanza Bucarest.

Este procedimiento —ilustrado en la imagen más abajo— encuentra una solución válida, aunque no necesariamente óptima: el camino Arad → Sibiu → Fagaras → Bucarest es 32 millas más largo que la ruta óptima (Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucarest).



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 86.

La búsqueda voraz se guía eficazmente hacia el objetivo, pero puede quedar atrapada en callejones o elegir rutas más costosas, ya que solo considera la estimación de distancia restante y no el costo recorrido. A pesar de esto, resulta muy útil cuando se requiere rapidez y el costo exacto no es crítico, como en sistemas de navegación preliminar, planificación de itinerarios o búsqueda en mapas digitales.

Este algoritmo es completo en espacios de estados finitos, pero no en infinitos. La complejidad temporal y espacial en el peor de los casos es $O(|V|)$. Con una buena función heurística, sin embargo, la complejidad se puede reducir considerablemente, en ciertos problemas alcanzando $O(bm)$.

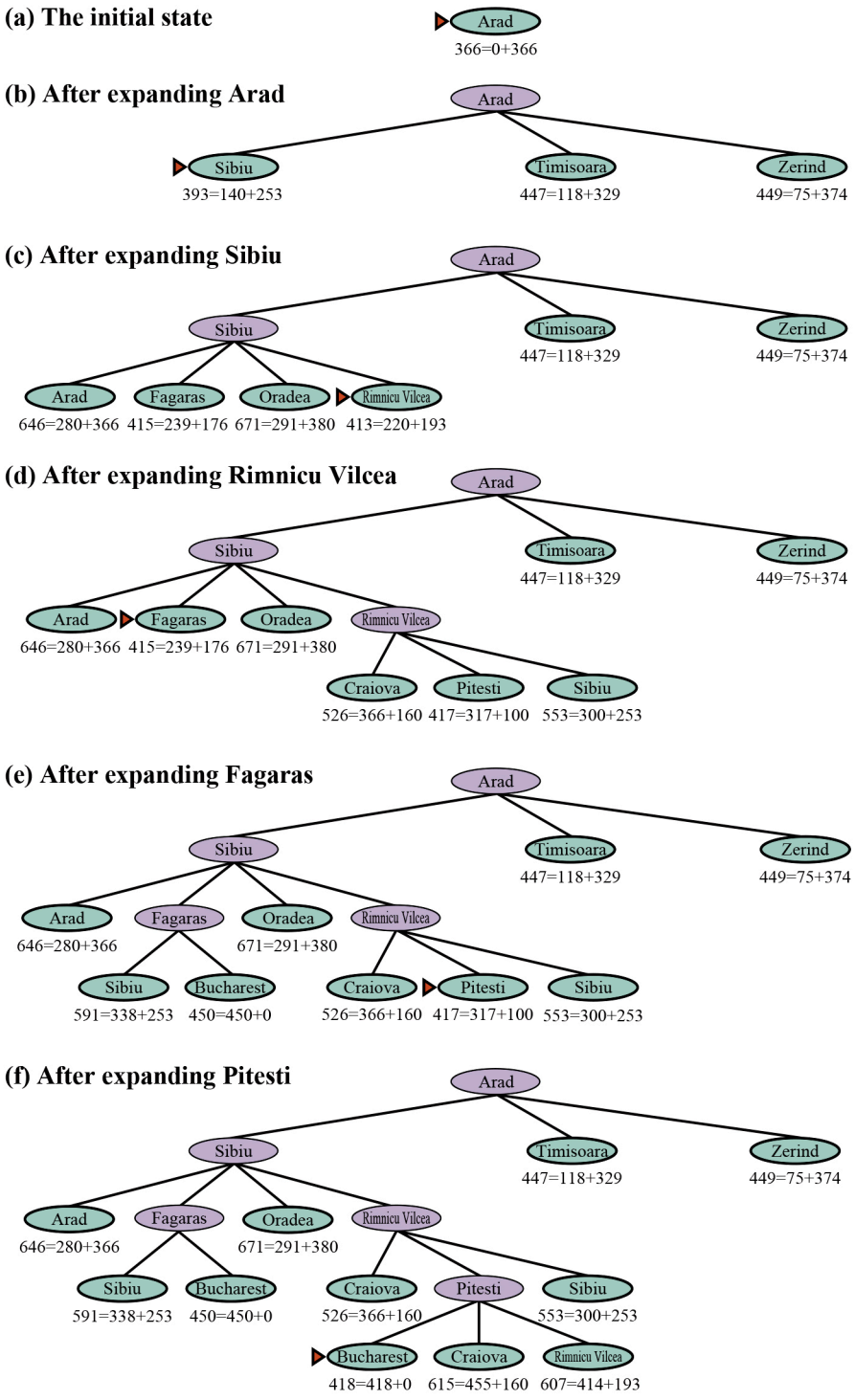
Búsqueda A*

Este algoritmo combina la exploración sistemática de la búsqueda de costo uniforme con la orientación de la búsqueda heurística. En lugar de guiarse únicamente por la distancia al objetivo (*h(n)*) o por el costo acumulado ((*g(n)*), *A** equilibra ambos factores mediante la función de evaluación:

$$f(n) = g(n) + h(n)$$

Donde *g(n)* es el costo del camino desde el estado inicial hasta el nodo *n*, y *h(n)* es el costo estimado desde *n* hasta el objetivo. Por lo tanto, *f(n)* representa el costo total estimado del mejor camino posible que pasa por *n*. El algoritmo expande en cada paso el nodo con el menor valor de *f(n)* lo que garantiza que se prioricen los caminos más prometedores considerando tanto el progreso realizado como la distancia restante.

Veamos un ejemplo de implementación de la búsqueda *A** desde Arad hasta Bucarest utilizando la heurística de distancia en línea recta, donde *g* son los costos de acción inicialmente definidos en la primera imagen de la unidad 5.



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 87.

Durante el proceso, el algoritmo evita expandir rutas que parecen más costosas que la mejor estimación actual. Por ejemplo, aunque Bucarest aparece en la frontera en una etapa temprana con *f=450*, el algoritmo prefiere expandir primero Pitesti con *f=417*, ya que existe la posibilidad de alcanzar una ruta más económica por ese camino. Finalmente, al llegar a Bucarest con *f=418*, el algoritmo concluye que ha encontrado la ruta óptima.

La búsqueda *A** presenta tres propiedades que la convierten en una de las estrategias más importantes y utilizadas en inteligencia artificial:

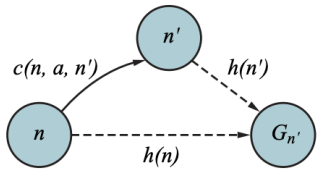
1. Completitud: *A** siempre encuentra una solución si esta existe, siempre que los costos de acción sean positivos.
2. Optimalidad: La solución hallada será de costo mínimo si la heurística utilizada cumple ciertas condiciones, que veremos a continuación.
3. Eficiencia: Entre todos los algoritmos que expanden nodos por orden de costo estimado, *A** es el más eficiente posible: ningún otro algoritmo con la misma información expandirá menos nodos para garantizar la solución óptima.

El desempeño de *A** depende directamente de las propiedades de la función heurística. Existen dos conceptos clave:

Por un lado, decimos que una heurística es admisible si nunca sobrestima el costo real para alcanzar el objetivo. Es decir, es siempre optimista. Por ejemplo, la distancia en línea recta nunca es mayor que la distancia real por ruta, por lo que la heurística definida al principio de esta unidad es una heurística admisible. Por otro lado, una heurística es consistente si por cada nodo n y cada sucesor n' de n generado por una acción a tenemos:

$$h(n) \leq c(n, a, n') + h(n')$$

Esto significa que el costo estimado desde un nodo nunca puede ser mayor que el costo de ir a su sucesor más la estimación desde ese sucesor al objetivo, es una forma de desigualdad triangular, que estipula que un lado del triángulo no puede ser más largo que la suma de los otros dos lados como se muestra en la figura a continuación:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 88.

Toda heurística consistente es también admisible. Cuando una heurística es consistente, *A** no necesita reconsiderar estados ya visitados: la primera vez que se alcanza un estado, el camino encontrado será el de menor costo posible. Con heurísticas inconsistentes, pueden aparecer caminos alternativos con menor costo posterior, lo que obliga al algoritmo a reinsertar y reevaluar estados, aumentando el consumo de tiempo y memoria.

A lo largo de este módulo aprendimos que resolver un problema implica mucho más que ejecutar acciones: requiere razonar sobre el futuro, evaluar alternativas y seleccionar caminos posibles dentro de un espacio de estados. El agente racional, enfrentado a una situación en la que no conoce de antemano la acción correcta, recurre a procesos de búsqueda para descubrir secuencias que lo conduzcan a su meta.

Comenzamos comprendiendo cómo formular un problema, identificando sus elementos fundamentales. Luego exploramos las estrategias de búsqueda no informada, donde el agente actúa sin orientación adicional, aplicando métodos sistemáticos para llegar a su objetivo. Posteriormente, incorporamos un nuevo nivel de racionalidad mediante las búsquedas informadas, en las cuales el agente utiliza heurísticas —estimaciones basadas en conocimiento del dominio— para guiar su exploración de manera más eficiente. Los algoritmos que aprendimos mostraron cómo esa información puede transformar una búsqueda ciega en un proceso inteligente, equilibrando rapidez, costo y calidad de la solución.

Con estas herramientas, el agente pasa de un comportamiento puramente exploratorio a un comportamiento planificado y orientado por conocimiento. Sin embargo, hasta aquí, su razonamiento sigue siendo operacional: el agente busca secuencias de acciones dentro de un entorno bien definido, pero no razona explícitamente sobre conceptos, relaciones o proposiciones.

A partir del próximo módulo avanzaremos hacia un nuevo nivel de representación cognitiva.

¡Continuemos!

Actividad

Para finalizar, realice las actividades de este módulo.

Módulo 3 | Glosario

Abstracción: proceso de simplificación mediante el cual un agente representa solo los aspectos relevantes de un problema, eliminando detalles innecesarios para la búsqueda.

Algoritmo de búsqueda: procedimiento sistemático que explora un espacio de estados para hallar una secuencia de acciones que conduzca desde el estado inicial al objetivo.

Árbol de búsqueda: estructura generada por el agente durante la exploración del espacio de estados; sus nodos representan estados alcanzables y sus ramas, las acciones que los conectan.

Búsqueda ciega o no informada: estrategia en la que el agente no dispone de conocimiento adicional sobre la distancia al objetivo y explora el espacio de manera sistemática.

Búsqueda informada o heurística: estrategia que utiliza conocimiento adicional del dominio para guiar la exploración hacia los caminos más prometedores mediante funciones heurísticas.

Función heurística: estimación del costo mínimo restante para alcanzar el objetivo desde un estado determinado; orienta las búsquedas informadas.

Optimalidad: propiedad de un algoritmo que garantiza que la solución hallada tiene el menor costo posible.

Módulo 3 | Actividades

ACTIVIDAD 1
Formulación de problemas

Formule de manera completa cada uno de los siguientes problemas, utilizando especificaciones lo suficientemente precisas como para poder implementarlos:

a) Seis cajas de cristal están alineadas, cada una con un candado. Las cinco primeras contienen la llave que abre la caja siguiente, y la última almacena una manzana. Usted dispone de la llave de la primera caja y su objetivo es obtener la manzana.

b) Se parte de la secuencia ABABAECCEC —o, en general, de cualquier secuencia formada por A, B, C y E—. Puede transformarse aplicando las igualdades AC = E, AB = BC, BB = E y Ex = x para cualquier x. Por ejemplo, la secuencia ABBC puede transformarse en AEC (usando BB = E), luego en AC (usando Ex = x) y finalmente en E (usando AC = E). El objetivo consiste en obtener la secuencia E.

c) Hay una cuadrícula *nxn* de cuadrados, cada uno de los cuales es inicialmente un suelo sin pintar o un pozo sin fondo. Usted comienza en una casilla de suelo sin pintar y puede pintar la casilla en la que se encuentra o desplazarse a una casilla adyacente que también sea suelo sin pintar. Su objetivo es pintar todas las casillas de suelo.

d) Un buque portacontenedores se encuentra en el puerto con una carga de 13 filas por 13 columnas y 5 niveles de altura. Usted controla una grúa capaz de desplazarse por encima del barco, recoger un contenedor y trasladarlo al muelle. El objetivo es descargar por completo el barco.

CC 1

ACTIVIDAD 1 | *Clave de corrección* CC 1

a) Estado inicial: tal y como se describe en el enunciado.
Estado objetivo: tienes una manzana.
Modelo de transición: abre cualquier caja para la que tengas la llave, obtén el contenido de cualquier caja abierta.
Función de costo: número de acciones.

b) Estado inicial: ABABAECCEC.
Estado objetivo: estado E
Modelo de transición: aplicar una igualdad sustituyendo una subsecuencia por otra.
Función de costo: número de transformaciones.

c) Estado inicial: todas las casillas del suelo sin pintar, empiezas de pie sobre una casilla sin pintar.
Estado objetivo: todas las casillas del suelo pintadas.

Modelo de transición: pintar la baldosa actual, moverse a la baldosa adyacente sin pintar.
Función de costo: número de movimientos.

d) Estado inicial: todos los contenedores apilados en el barco.
Estado objetivo: todos los contenedores descargados.
Modelo de transición: mover la grúa a una ubicación determinada, recoger un contenedor, depositar el contenedor.
Función de costo: tiempo empleado en descargar el barco.

ACTIVIDAD 2
Comparando algoritmos de búsqueda no informada

Consideremos un espacio de estados en el que el estado inicial es el número 1 y cada estado tiene dos sucesores: los números $2k$ y $2k+1$.

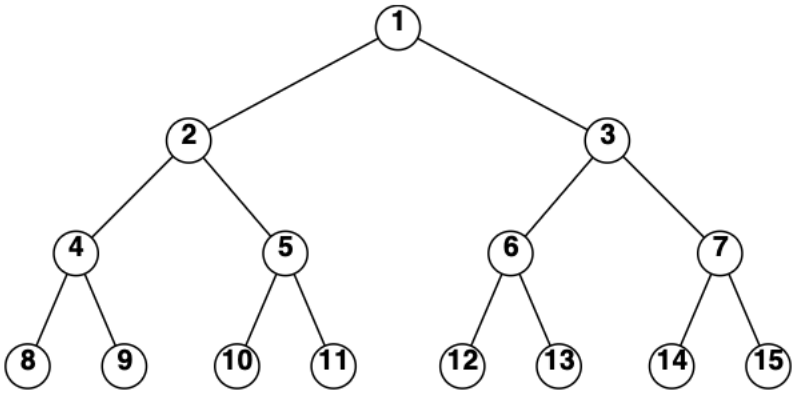
a) Dibuje la parte del espacio de estados correspondiente a los estados del 1 al 15.

b) Supongamos que el estado objetivo es el 11. Enumere el orden en el que se visitarán los nodos para la búsqueda en anchura, la búsqueda con profundidad limitada (con límite 3) y la búsqueda con profundización iterativa.

CC 1

ACTIVIDAD 2 | *Clave de corrección* CC 1

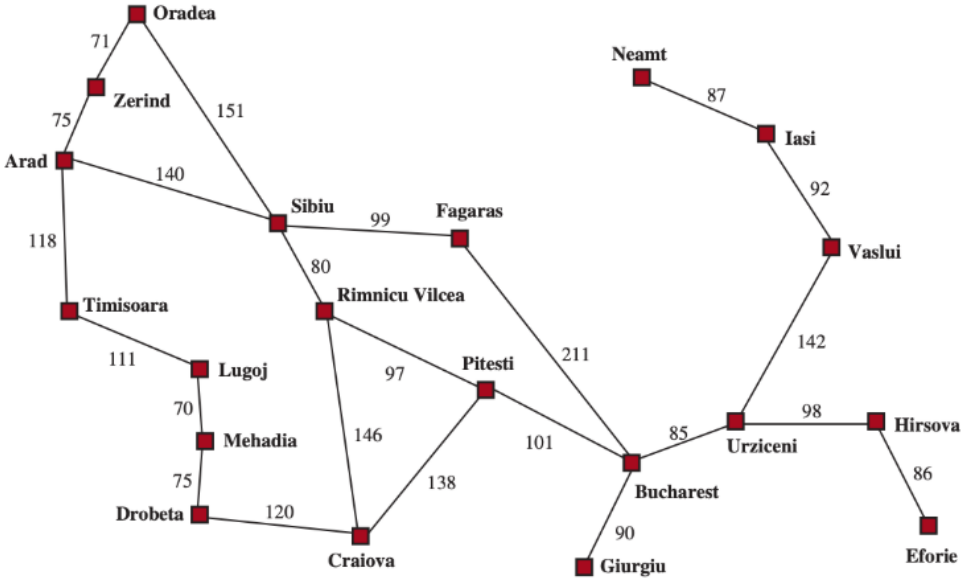
a) Un dibujo de los espacios de estado sería el siguiente



b) Búsqueda en anchura: 1 2 3 4 5 6 7 8 9 10 11
Búsqueda con profundidad limitada: 1 2 4 8 9 5 10 11
Búsqueda con profundización iterativa: 1; 1 2 3; 1 2 4 5 3 6 7; 1 2 4 8 9 5 10 11

ACTIVIDAD 3
Algoritmo A*

Describa el funcionamiento del algoritmo A* aplicado al problema de llegar a Bucarest desde Lugoj, utilizando como heurística la distancia en línea recta. Presente la secuencia de nodos considerados por el algoritmo y detalle, para cada uno, los valores de f, g y h que intervienen en el proceso.



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 64.

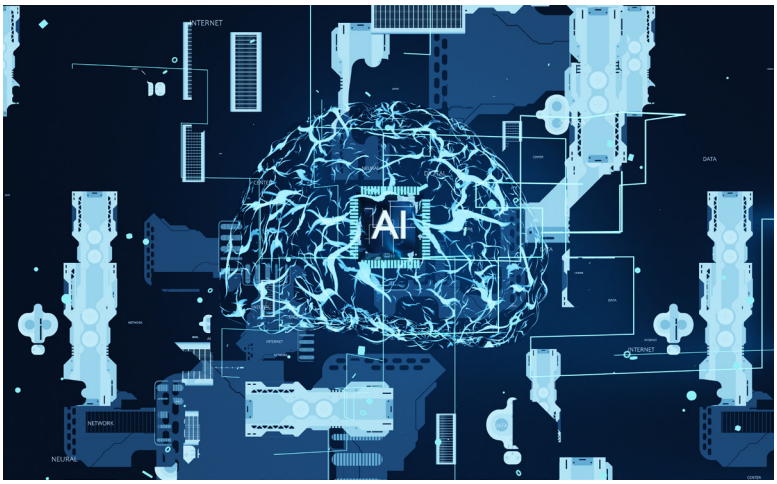
Módulo 4

Microobjetivos

- Comprender el concepto de representación del conocimiento para reconocer cómo los agentes racionales describen y estructuran información sobre su entorno mediante lenguajes formales.
- Distinguir los elementos de la lógica proposicional (sintaxis, semántica e inferencia) para identificar su papel en la construcción de sistemas lógicos y en la deducción de nuevas afirmaciones.
- Saber aplicar las reglas de inferencia y los métodos de prueba proposicional para derivar conclusiones válidas y comprender el funcionamiento de los mecanismos de razonamiento automático.
- Interpretar los principios de la lógica de primer orden para saber expresar hechos generales y relaciones complejas entre objetos.

Contenidos

REPRESENTACIÓN DEL CONOCIMIENTO Y RAZONAMIENTO LÓGICO



Fuente: https://www.freepik.es/

Contenidos

En los módulos anteriores aprendimos que los agentes racionales pueden percibir su entorno, formular problemas y buscar soluciones mediante la exploración de posibles acciones. Ese tipo de razonamiento, aunque eficaz, se basa en procedimientos: el agente actúa sin comprender el significado profundo de lo que hace.

En este módulo daremos un paso más hacia la inteligencia simbólica. Un agente verdaderamente inteligente no solo ejecuta acciones, sino que razona sobre lo que sabe, extrae conclusiones y explica sus decisiones. Para lograrlo, necesita una forma de representar

conocimiento: un lenguaje formal que le permita describir hechos, objetos y relaciones del mundo, y reglas lógicas que determinen qué puede deducirse a partir de ellos.

La representación del conocimiento convierte la información en estructuras simbólicas manipulables, y el razonamiento lógico le permite derivar nuevas verdades a partir de las existentes. Con estas herramientas, los agentes pasan de actuar por búsqueda a pensar por inferencia, empleando proposiciones, implicaciones y cuantificadores para comprender y transformar su entorno.

Este módulo introducirá los fundamentos del razonamiento lógico en inteligencia artificial, comenzando por la lógica proposicional, que permite representar afirmaciones simples, y avanzando hacia la lógica de primer orden, capaz de expresar relaciones entre objetos y propiedades. A través de ejemplos y entornos como el "mundo del Wumpus", veremos cómo los agentes basados en conocimiento utilizan estas representaciones para deducir, planificar y tomar decisiones racionales en entornos complejos.

¡Comencemos!

Unidad 8: Representación simbólica y conocimiento basado en lógica

En los módulos anteriores conocimos distintos tipos de agentes capaces de percibir, actuar y planificar mediante la búsqueda de soluciones. Esos agentes sabían cosas, pero de manera limitada: conocían sus posibles acciones y las consecuencias inmediatas de aplicarlas en un entorno, pero no podían razonar sobre el conocimiento en sí mismo ni expresar relaciones generales sobre el mundo.

Un agente que busca caminos en un mapa, por ejemplo, sabe calcular la ruta más corta, pero no "sabe" que una ruta no puede tener longitud negativa, o que dos ciudades distintas no pueden ocupar el mismo lugar. Este tipo de conocimiento que es abstracto, general y reutilizable, es esencial para comportarse de forma verdaderamente inteligente.

Los agentes basados en conocimiento surgen para superar esas limitaciones. A diferencia de los agentes puramente reactivos o de búsqueda, estos pueden almacenar, combinar y derivar información sobre el mundo de manera explícita. Su comportamiento no depende de reglas preprogramadas, sino de un proceso de razonamiento lógico sobre lo que saben.

El elemento central de un agente de este tipo es su base de conocimiento, o BC.



Una base de conocimiento está formada por **sentencias** expresadas en un lenguaje de representación del conocimiento, un sistema formal que permite describir hechos, objetos, propiedades y relaciones del entorno.

Cada sentencia representa una afirmación sobre el mundo, como "la habitación A contiene oro" o "si hay un pozo en una celda, las celdas vecinas son peligrosas". Cuando una afirmación se acepta como verdadera sin derivarse de otras, se la denomina **axioma**. A partir de estos axiomas, el agente puede inferir nuevas sentencias aplicando reglas lógicas. A este proceso se lo llama **inferencia**, y consiste en obtener nuevas conclusiones a partir de la información existente.

Veamos un ejemplo de un programa para un agente basado en conocimiento:

```
función AGENTE-BC(percepción) devuelve una acción
    variables estáticas: BC, una base de conocimiento
                         t, un contador, inicializado en 0, que indica el tiempo

    DECIR(BC, CONSTRUIR-SENTENCIA-DE-PERCEPCIÓN(percepción, t))
    acción ← PREGUNTAR(BC, PEDIR-ACCIÓN(t))
    DECIR(BC, CONSTRUIR-SENTENCIA-DE-ACCIÓN(acción, t))
    t ← t + 1
    devolver acción
```

Fuente: elaboración propia

Como todos los agentes que hemos visto, recibe una percepción como entrada y devuelve una acción. El agente mantiene su base de conocimiento, BC, que puede contener inicialmente algunos conocimientos previos.

Cada vez que se llama al programa del agente, este realiza tres acciones. Primero, le DICE a la base de conocimiento qué es lo que percibe. Segundo, le PREGUNTA qué acción debería ejecutar; en ese proceso, la base de conocimiento puede llevar adelante un razonamiento detallado sobre el estado actual del mundo, las posibles secuencias de acciones y sus consecuencias. Tercero, el programa del agente le DICE a la base de conocimiento qué acción fue seleccionada y devuelve esa acción para que pueda ejecutarse.

Los detalles del lenguaje de representación quedan ocultos dentro de tres funciones que actúan como interfaz entre los sensores y actuadores, por un lado, y el sistema central de representación y razonamiento, por el otro. CONSTRUIR-SENTENCIA-DE-PERCEPCIÓN genera una sentencia que afirma que el agente percibió la información correspondiente en un momento dado. PEDIR-ACCIÓN construye una sentencia que consulta qué acción debe ejecutarse en el momento actual. Por último, CONSTRUIR-SENTENCIA-DE-ACCIÓN crea una sentencia que afirma que la acción seleccionada fue efectivamente ejecutada. Los mecanismos de inferencia, a su vez, quedan encapsulados dentro de las funciones DECIR y PREGUNTAR. Estos aspectos serán desarrollados en detalle más adelante.

Este enfoque convierte al agente en algo más que un programa de control: lo transforma en una entidad que razona de forma explícita sobre su entorno. Gracias a ello, podemos describir su comportamiento en términos del **nivel de conocimiento**, es decir, especificando lo que el agente sabe y desea, sin necesidad de detallar cómo está implementado internamente.

Por ejemplo, si un taxi automatizado tiene el objetivo de trasladar a un pasajero desde San Francisco hasta el condado de Marin, y sabe que el único puente que une ambas zonas es el Golden Gate, entonces podemos anticipar que cruzará ese puente. Esta explicación es válida independientemente de cómo esté programado el taxi o de qué tipo de sensores use. Lo importante no es su mecanismo físico, sino su razonamiento simbólico: actúa como lo haría un ser racional que conoce hechos y los aplica para lograr un objetivo.

Esta capacidad de describir al agente en el nivel de conocimiento (y no solo en el nivel de implementación) marca una diferencia crucial en el estudio de la inteligencia artificial. En lugar de ver al agente como un conjunto de procedimientos, lo entendemos como una entidad que posee creencias, objetivos y reglas de inferencia, y que actúa conforme a ellos.

Podemos construir un agente de este tipo de dos maneras:

- Enfoque declarativo: se le "dice" al agente lo que necesita saber mediante sentencias expresadas en su lenguaje de representación. Por ejemplo, "si hay un brillo, entonces hay oro" o "si una celda huele mal, hay un pozo cerca".

- Enfoque procedimental: el conocimiento se incorpora implícitamente en el código del programa, sin que esté expresado de forma explícita como hechos o reglas.

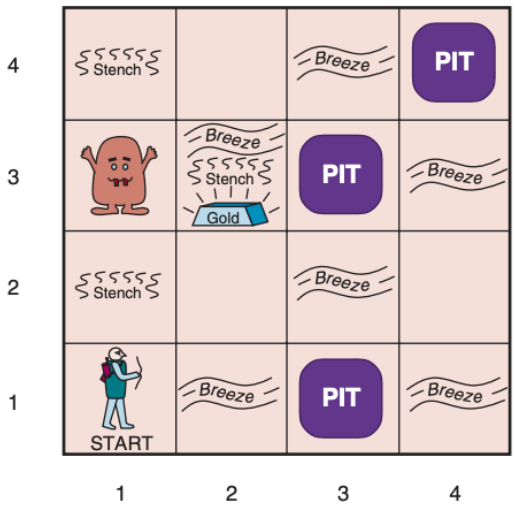
En la práctica, ambos enfoques suelen combinarse: el conocimiento declarativo puede transformarse luego en procedimientos más eficientes, y los agentes más avanzados integran ambos niveles. Finalmente, un agente basado en conocimiento puede **apren-**

der por sí mismo. A través de la experiencia y la observación, puede generar nuevas afirmaciones generales sobre el entorno a partir de percepciones individuales. Este tipo de aprendizaje le permite adaptarse y volverse verdaderamente autónomo.

Para comprender cómo opera un agente basado en conocimiento, resulta útil situarlo en un entorno controlado donde el razonamiento lógico sea imprescindible para la supervivencia. Utilizaremos el ejemplo propuesto en el libro Artificial Intelligence: A Modern Approach de RUSSELL, Stuart y NORVIG, Peter (2021) conocido como el mundo del Wumpus, un entorno diseñado para poner a prueba las capacidades de representación simbólica, inferencia y toma de decisiones racionales.

El mundo del Wumpus es una cueva compuesta por habitaciones interconectadas que el agente debe explorar en busca de oro, evitando trampas mortales y un monstruo oculto conocido como el Wumpus. El agente puede dispararle al monstruo, pero sólo tiene una flecha. Algunas habitaciones contienen pozos sin fondo que atraparán a cualquiera que se adentre en ellas. En este entorno, actuar al azar o sin razonamiento lógico conduce inevitablemente a la muerte; el éxito depende de la capacidad del agente para deducir información no observable a partir de sus percepciones parciales y del conocimiento que posee sobre las reglas del entorno.

Veamos una representación gráfica de este mundo:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 211.

Definamos el mundo del Wumpus según el modelo REAS aprendido en el módulo 2:

- **Medida de rendimiento:** el agente recibe +1000 puntos por salir de la cueva con el oro, –1000 si cae en un pozo o es devorado por el Wumpus, –1 por cada acción realizada y –10 por disparar su flecha. El juego termina si el agente muere o logra salir.

- **Entorno:** una cuadrícula de 4x4 habitaciones rodeadas de muros. El agente inicia siempre en la celda [1,1], orientado hacia el este. La ubicación del Wumpus y del oro se elige aleatoriamente entre las demás celdas. Cualquier celda distinta de la inicial puede contener un pozo con una probabilidad del 20%.

- **Actuadores:** el agente puede *Avanzar*, *GirarIzquierda* 90 grados, o *GirarDerecha* 90 grados. El agente muere si entra en una celda que contiene un pozo o un Wumpus vivo. Si un agente intenta moverse hacia adelante y choca con una pared, entonces no se mueve. La acción A*garrar* puede ser utilizada para recoger el oro si está en la misma celda que el agente. La acción *Disparar* puede ser utilizada para lanzar una flecha en línea recta en la dirección en la que está mirando el agente. La flecha continúa hasta que golpea (y por lo tanto mata) al Wumpus o a una pared. El agente tiene solo una flecha, por lo que solo la primera acción *Disparar* tiene algún efecto. Finalmente, la acción *Escalar* puede utilizarse para salir de la cueva, pero solo desde la casilla [1,1].

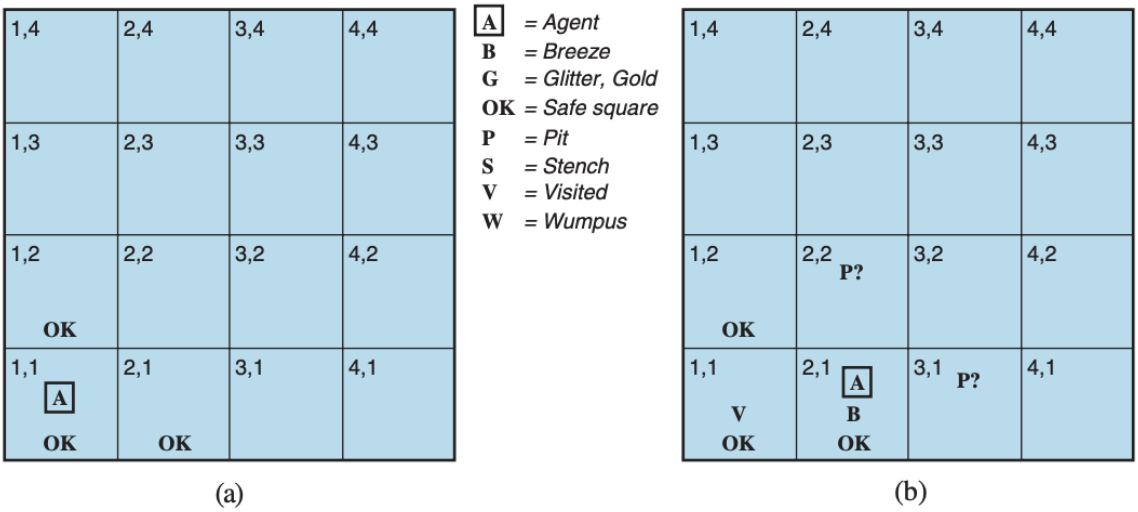
- **Sensores:** el agente percibe señales locales que le brindan información indirecta sobre los peligros cercanos:

  - Hedor: en las celdas adyacentes al Wumpus.
  - Brisa: en las celdas adyacentes a un pozo.
  - Brillo: en la celda donde está el oro.
  - Choque: al intentar avanzar contra una pared.
  - Grito: cuando el Wumpus es alcanzado por una flecha, puede percibirse desde cualquier celda de la cueva.

Cada percepción se recibe en forma de vector de cinco valores lógicos, por ejemplo: [Hedor, Brisa, Nada, Nada, Nada] indica que hay hedor y brisa, pero no hay brillo, choque ni grito.

En base a lo aprendido sobre los entornos en el módulo 2 podemos clasificar al mundo del Wumpus como determinista, porque las consecuencias de cada acción son predecibles. Discreto porque el agente se mueve entre un número finito de celdas. Estático porque el Wumpus y los pozos no se mueven. Secuencial, ya que las recompensas dependen de una serie de acciones encadenadas. Y parcialmente observable, ya que el agente no puede ver todo el entorno, sino solo percibir señales locales que debe interpretar.

Para el agente de este entorno, el desafío principal es la ignorancia inicial de cómo está configurado el mundo (dónde se encuentra el Wumpus, el oro y los pozos), superar esta ignorancia parece requerir un razonamiento lógico. Veamos al agente explorar el entorno de la figura presentada más arriba:

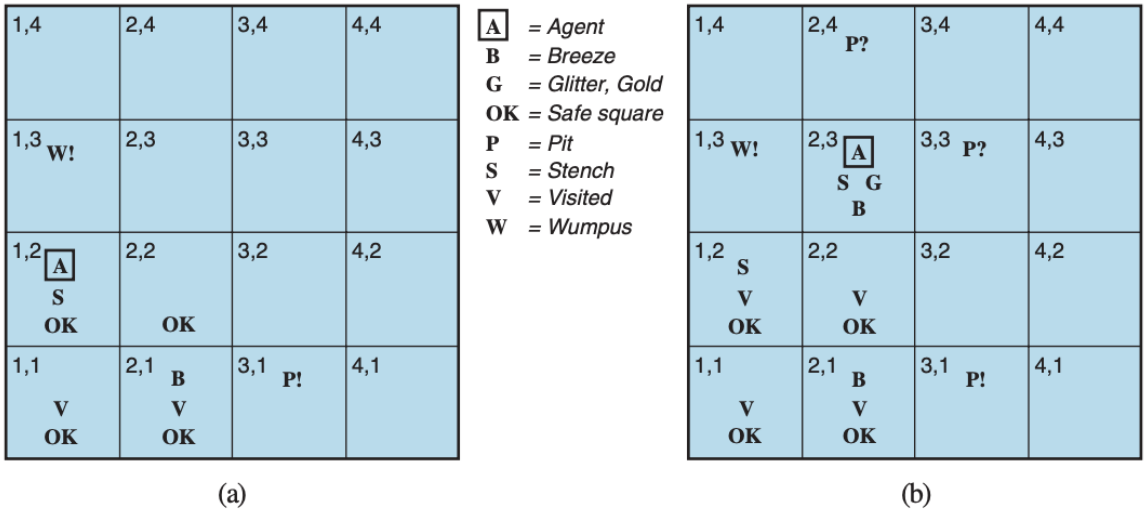


Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 212.

La base de conocimiento inicial del agente contiene las reglas del entorno, tal como se describió anteriormente; en particular, sabe que se encuentra en la celda [1,1] y que dicha celda es segura. Por eso, marcamos [1,1] con una "A" y un "OK". La primera percepción es [Nada, Nada, Nada, Nada, Nada], lo que permite concluir que las celdas vecinas —[1,2] y [2,1]— también están libres de peligro; es decir, están "OK". Son seguras porque la ausencia de brisa y de hedor implica que no hay pozos ni un Wumpus en las celdas adyacentes. El estado de conocimiento del agente hasta este punto se muestra en la figura (a) de la imagen anterior.

Supongamos que el agente decide avanzar a la celda [2,1]. Allí percibe una brisa, lo que sugiere que al menos una de las celdas vecinas contiene un pozo. Sabemos que el pozo no puede estar en [1,1], por las reglas del entorno, de modo que debe encontrarse en [2,2], en [3,1] o en ambas. La notación "P?" en la figura (b) marca justamente estas posibles ubicaciones. En este momento, solo queda una celda marcada como OK que aún no ha sido visitada, por lo que el agente regresa a [1,1] y se dirige a [1,2].

En [1,2], el agente percibe un hedor, lo que produce el estado de conocimiento mostrado en la figura (a) siguiente:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 213.

El hedor percibido en [1,2] indica que debe haber un Wumpus en una celda adyacente. Sin embargo, el Wumpus no puede estar en [1,1], por las reglas del entorno, ni en [2,2], ya que el agente habría detectado hedor cuando estuvo en [2,1]. Por lo tanto, puede inferir que el Wumpus se encuentra en [1,3]; la notación "W!" marca esta deducción. Además, la ausencia de brisa en [1,2] implica que no hay pozo en [2,2]. Pero el agente ya había inferido que debía existir un pozo en [2,2] o en [3,1], lo que permite concluir que dicho pozo está en [3,1]. Esta es una inferencia compleja, porque combina información obtenida en distintos momentos y lugares, y se apoya en la ausencia de una percepción para dar un paso clave en el razonamiento.

El agente ahora ha demostrado que en [2,2] no hay ni un pozo ni un Wumpus, por lo que es seguro avanzar hacia esa celda. No se muestra el estado de conocimiento del agente en [2,2]; simplemente se asume que gira y continúa hacia [2,3], lo que nos lleva a la figura (b). En [2,3], el agente percibe un brillo, por lo que debe tomar el oro y luego regresar a su punto de partida. Tengamos en cuenta que, en cada situación en la que el agente infiere algo a partir de la información disponible, esa conclusión será correcta siempre que dicha información también lo sea. Esta es una propiedad fundamental del razonamiento lógico.

Fundamentos formales del razonamiento lógico

El razonamiento lógico es el proceso mediante el cual un agente deduce nuevas verdades a partir del conocimiento que ya posee. Para que ese razonamiento sea confiable, debe apoyarse en un marco formal que especifique con precisión qué afirmaciones pueden expresarse, qué significan y en qué condiciones pueden considerarse verdaderas. Este marco lo proporcionan los principios fundamentales de la lógica, que definen la relación entre forma, significado y validez de las proposiciones.

Al comienzo de esta unidad se indicó que las bases de conocimiento están compuestas por sentencias. Estas sentencias se expresan conforme a la **sintaxis** del lenguaje de representación, la cual determina cuáles son las sentencias bien formadas. La noción de sinta-

xis es bastante clara en aritmética: por ejemplo, x + y = 4 es una sentencia bien formada, mientras que x4y+= no lo es.

La lógica también debe definir la **semántica**, o significado, de las sentencias. La semántica determina la verdad de cada sentencia con respecto a cada mundo posible. Por ejemplo, la semántica de la aritmética establece que la sentencia x + y = 4 es verdadera en un mundo donde x = 2 e y = 2, pero es falsa en un mundo donde x = 1 e y = 1. En las lógicas estándar, cada sentencia debe ser verdadera o falsa en cada mundo posible; no existe un "punto intermedio".

Cuando necesitamos ser precisos, usamos el término modelo en lugar de "mundo posible". Mientras que los mundos posibles podrían considerarse entornos (potencialmente) reales en los que el agente podría o no podría estar, los modelos son abstracciones matemáticas, cada una de las cuales tiene un valor de verdad fijo (verdadero o falso) para cada sentencia relevante. Informalmente, podemos pensar en un posible mundo como, por ejemplo, tener *x* hombres e *y* mujeres sentadas a una mesa jugando al bridge, y la sentencia x + y = 4 es verdadera cuando hay cuatro personas en total. De manera formal, los modelos posibles son todas las asignaciones de enteros no negativos a las variables x e y. Cada una de estas asignaciones determina la veracidad de cualquier sentencia aritmética cuyas variables sean x e y.

Si una sentencia α es verdadera en el modelo *m*, decimos que *m* satisface a α, o que *m* es un modelo de α. Usamos la notación $M(\alpha)$ para referirnos al conjunto de todos los modelos de α. Ahora que contamos con una notación de verdad, podemos abordar el razonamiento lógico. Este se ocupa de la relación de **implicación lógica** entre sentencias, es decir, de la idea de que una sentencia se deriva lógicamente de otra. En notación matemática, escribimos: $\alpha \models \beta$ para significar que la sentencia $\alpha\ \beta\ \alpha \models \beta$si y solo si,en cada modelo donde α es verdadero, β también lo es.Usando la notación que acabamos de ver podemos escribir:

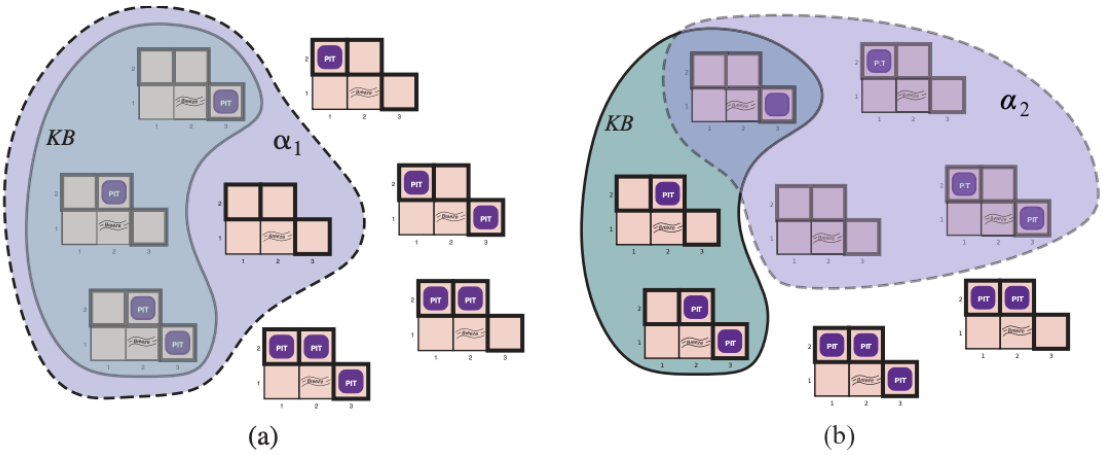
$$\alpha \models \beta \Leftrightarrow M(\alpha) \subseteq M(\beta)$$

La relación de implicación también es familiar en aritmética.Por ejemplo, podemos afirmar que la sentencia x = 0 implica la sentencia xy = 0.Esto se debe a que,en cualquier modelo en el que x sea igual a cero,el producto xy también será cero,independientemente del valor de y.

Apliquemos este mismo análisis al ejemplo de razonamiento del mundo del Wumpus que vimos más arriba.En la situación en que el agente está en la celda [2,1],ya ha registrado la ausencia de percepciones en [1,1] y actualmente percibe una brisa en [2,1].Estas percepciones,junto con el conocimiento de las reglas del entorno,conforman la base de conocimiento (BC).El agente está interesado en saber si las celdas adyacentes [1,2],[2,2] y [2,1] contienen pozos.Cada una de las tres celdas pueden o no contener un pozo por lo que,ignorando otros aspectos del mundo por ahora,existen 2^3= 8 modelos posibles.

La BC puede considerarse como un conjunto de sentencias o como una sola sentencia que afirma todas las sentencias individuales. La BC resulta falsa en aquellos modelos que contradicen el conocimiento del agente. Por ejemplo, cualquier modelo que ubique un pozo en [1,2] es incompatible con la BC, dado que en [1,1] no se percibe brisa. En rigor, solo

existen tres modelos en los que la BC es verdadera, y estos aparecen delimitados por una línea continua en la figura que se presenta a continuación.



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 215.

Ahora consideremos dos posibles conclusiones:

- $= \alpha_1$ "No hay pozo en [1,2]"
- $= \alpha_2$ "No hay pozo en [2,2]"

Los modelos α_1 y α_2 están encerrados por las líneas punteadas en la figura de arriba. Si miramos con más detalle la imagen, podemos ver que en cada modelo donde la BC es verdadera, α_1 también lo es. Por lo tanto, BC $\models \alpha_1$: no hay un pozo en [1,2]. A su vez, puede observarse que en algunos modelos en los que la BC es verdadera, α_2 resulta falsa. Por lo tanto, BC no implica α_2: el agente no puede concluir que no hay un pozo en [2,2], como tampoco que si lo haya.

El ejemplo anterior no solo ilustra la implicación, sino que también muestra cómo se puede aplicar la definición de implicación para derivar conclusiones, es decir, para llevar a cabo una inferencia lógica. Para comprender la diferencia entre implicación e inferencia, puede resultar útil pensar en el conjunto de todas las consecuencias de BC como un pajar y en como una aguja. La implicación lógica es la afirmación de que la aguja está en el pajar; la inferencia es el proceso de encontrarla. Esta distinción se expresa mediante una notación formal: si un algoritmo de inferencia i puede derivar α a partir de BC, escribimos BC $\vdash_i \alpha$, lo cual equivale a decir que "α se deriva de BC mediante i" o que "i deriva α a partir de BC".

Un procedimiento de inferencia es sólido si nunca deriva una proposición que no sea una consecuencia lógica de la base de conocimiento; es decir, si de BC $\models \alpha$ se sigue necesariamente que BC$\models \alpha$. Esta propiedad garantiza que el agente no incorpore afirmaciones incorrectas a su conocimiento (que no encuentre agujas que en realidad no existen).

Por otro lado, un procedimiento es completo si puede derivar todas las proposiciones que son consecuencias lógicas de la base de conocimiento, es decir, si BC$\models \alpha$ implica que BC

$\vdash_i \alpha$. En sistemas ideales, los mecanismos de inferencia son a la vez sólidos y completos, aunque en la práctica, la completitud puede sacrificarse parcialmente para lograr mayor eficiencia computacional.



Énfasis

En resumen, la relación entre sintaxis, semántica e inferencia define la arquitectura fundamental del razonamiento lógico:

- La **sintaxis** describe la estructura del lenguaje del conocimiento.
- La **semántica** proporciona el significado y las condiciones de verdad.
- La **inferencia** establece las reglas que permiten derivar conclusiones preservando la verdad.

Cuando estos tres componentes se integran, el agente puede construir representaciones simbólicas coherentes del mundo y razonar de forma racional y verificable.

A partir de estos principios, podemos definir lenguajes lógicos específicos para expresar y manipular conocimiento. El más simple y ampliamente utilizado es la lógica proposicional, que estudiaremos a continuación como el primer sistema formal de representación e inferencia en la inteligencia artificial.



Actividad

Antes de continuar, lo/a invito a resolver la primera actividad de este módulo

Unidad 9: Lógica proposicional

La lógica proposicional constituye el sistema más simple y fundamental de representación del conocimiento lógico. En ella, el mundo se describe mediante proposiciones elementales —afirmaciones que pueden ser únicamente verdaderas o falsas—, y a partir de ellas se construyen proposiciones compuestas mediante operadores lógicos. Este lenguaje formal permite representar el conocimiento de un agente en forma precisa y realizar inferencias válidas a partir de reglas de composición bien definidas.

Comencemos por aprender la sintaxis de la lógica proposicional, que define qué configuraciones de símbolos constituyen sentencias bien formadas. Las unidades más simples son las **sentencias atómicas**, representadas mediante **símbolos proposicionales** (por ejemplo, P, Q, R, $W_{1,3}$, $B_{2,1}$). Cada símbolo representa una afirmación concreta sobre el mundo, como "hay un Wumpus en [1,3]" o "hay una brisa en [2,1]", que puede ser verdadera o falsa.

Existen además dos símbolos con significados fijos: Verdadero, que representa una proposición siempre verdadera, y Falso, que representa una proposición siempre falsa.

A partir de estas sentencias atómicas se construyen sentencias complejas mediante conectivos lógicos, que expresan relaciones entre proposiciones. Los cinco conectivos fundamentales son:

Símbolo	Nombre	Descripción	Ejemplo
¬	Negación	La sentencia ¬P expresa que *no* es cierto P. Una sentencia atómica o su negación se denomina literal.	$\neg W_{1,3}$: "no hay Wumpus en [1,3]".
∧	Conjunción	Una sentencia cuya conectiva principal es se llama conjunción, y sus partes, conjuntores.	$W_{1,3} \wedge P_{3,1}$: "el Wumpus está en [1,3] **y** hay un pozo en [3,1]".
∨	Disyunción	Los componentes de una disyunción se llaman disyuntores.	$(W_{1,3} \wedge P_{3,1})$ $W_{2,2}$: "(hay un Wumpus en [1,3] y un pozo en [3,1]) **o** bien hay un Wumpus en [2,2]".
⇒	Implicación	Representa una relación condicional entre dos sentencias. La primera parte es el antecedente o premisa y la segunda, el consecuente o conclusión. También se conoce como regla o sentencia si-entonces.	$(W_{1,3} \wedge P_{3,1}) \neg W_{2,2}$: "si hay un Wumpus en [1,3] y un pozo en [3,1], **entonces** no hay Wumpus en [2,2]".
⇔	Bicondicional / "si y sólo si"	Expresa una equivalencia lógica: ambas sentencias son verdaderas o falsas al mismo tiempo.	$\mathbf{W_{1,3} \Leftrightarrow \neg W_{2,2}}$: "hay un Wumpus en [1,3] si y solo si no lo hay en [2,2]".

El conjunto de todas las sentencias posibles puede describirse formalmente mediante una gramática, conocida como BNF (Backus–Naur Form). Esta gramática, como la que se muestra en la figura a continuación, establece la jerarquía de los operadores y define el orden de precedencia entre ellos.

```
                Sentencia -> Sentencia Atómica | Sentencia Compleja
     Sentencia Atómica -> Verdadero | Falso | P | Q | R | …
   Sentencia Compleja -> ¬Sentencia
                       | Sentencia ∧ Sentencia
                       | Sentencia ∨ Sentencia
                       | Sentencia ⇒ Sentencia
                       | Sentencia ⇔ Sentencia

Precedencia de operadores: ¬,∧,∨,⇒,⇔
```

El operador de negación (¬) tiene la mayor precedencia, seguido por la conjunción (**∧**), la disyunción (**∨**), la implicación (⇒) y, finalmente, el bicondicional (⇔). Esto significa, por ejemplo, que en la expresión ¬A**∧**B la negación se aplica primero, interpretándose como (¬A)**∧**B y no como ¬(A**∧**B). El cumplimiento de estas reglas sintácticas garantiza que las proposiciones sean estructuralmente válidas y, por tanto, susceptibles de evaluación semántica.

Habiendo especificado la sintaxis de la lógica proposicional, veamos ahora su **semántica**. La semántica define las reglas para determinar la verdad de una sentencia con respecto a un modelo particular. En lógica proposicional, un modelo simplemente establece el valor de verdad —verdadero o falso— para cada símbolo proposicional. Por ejemplo, si las sentencias en la base de conocimiento usan los símbolos proposicionales $P_{1,2}$, $P_{2,2}$, y $P_{3,1}$, entonces un modelo posible puede ser:

$$m = \{P_{1,2} = \text{falso}, P_{2,2} = \text{falso}, P_{3,1} = \text{verdadero}\}$$

El número de modelos posibles crece de manera exponencial con la cantidad de sentencias: con tres símbolos, existen $2^3 = 8$ modelos distintos. La semántica de la lógica proposicional debe indicar cómo determinar el valor de verdad de cualquier sentencia dado un modelo, y esto se define de forma recursiva. Todas las sentencias se construyen a partir de sentencias atómicas y de los cinco conectivos, por lo que es necesario especificar tanto cómo se evalúan las sentencias atómicas como cómo se evalúan las sentencias formadas con cada conectivo. En el caso de las sentencias atómicas, el procedimiento es directo:

- *Verdadero* es verdadero en cada modelo y *Falso* es falso en cada modelo.
- El valor de verdad de cualquier otro símbolo proposicional debe ser especificado directamente en el modelo. Por ejemplo, en el modelo m_1 que vimos más arriba, $P_{1,2}$ es falso.



Para sentencias complejas, tenemos cinco reglas, que se cumplen para cualquier sub-sentencia *P* y *Q* (atómica o compleja) en cualquier modelo *m*:

- ¬*P* es verdadero sí y solo sí *P* es falso en *m*.

- $P \wedge Q$ es verdadero si y solo si ambos son verdaderos en m.
- $P \vee Q$ es verdadero si y solo si P o Q es verdadero en m.
- $P \Rightarrow Q$ es verdadero salvo que P sea verdadero y Q falso en m.
- $P \Leftrightarrow Q$ es verdadero si y solo si P y Q son ambos verdaderos o ambos falsos en m.

Estas reglas pueden representarse mediante **tablas de verdad**, como las de la figura que vemos a continuación, que muestran los valores resultantes para todas las combinaciones posibles de P y Q.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 219.

A partir de estas tablas es posible evaluar cualquier sentencia proposicional de manera recursiva. Por ejemplo, la sentencia $\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1})$, evaluada en m_1, se evalúa como *verdadero* $\wedge$ (*falso* $\vee$ *verdadero*) = *verdadero* $\wedge$ *verdadero* = *verdadero*.

Las tablas de verdad reflejan una interpretación estricta y matemática de los conectivos, que puede diferir de la intuición del lenguaje natural. Por ejemplo, en lógica proposicional, la implicación "$P \Rightarrow Q$" es verdadera en todos los casos excepto cuando P es verdadera y Q es falsa. Esto puede parecer contraintuitivo, pero tiene una justificación formal: una implicación solo afirma que "si P es verdadero, entonces Q también debe serlo"; no dice nada sobre los casos en que P es falso.

Ahora que hemos definido la semántica de la lógica proposicional podemos construir una base de conocimiento para el mundo del Wumpus. Primero nos concentraremos en los aspectos **inmutables** de este mundo, dejando los aspectos **mutables** para más adelante. Por ahora, necesitamos los siguientes símbolos para las ubicaciones [x,y]:

- $P_{x,y}$ es verdadero si hay un pozo en [x,y].
- $W_{x,y}$ es verdadero si hay un Wumpus en [x,y].
- $B_{x,y}$ es verdadero si hay una brisa en [x,y].
- $H_{x,y}$ es verdadero si hay hedor en [x,y].
- $L_{x,y}$ es verdadero si el agente está en la ubicación [x,y].

Las sentencias que escribimos son suficientes para derivar $\neg P_{1,2}$ (no hay un pozo en [1,2]), tal como hicimos informalmente más arriba. Vamos a etiquetar cada sentencia como R_i para que podamos referenciarlas más adelante.

- No hay un pozo en [1,1]:
  - $R_1: \neg P_{1,1}$
- Una celda tiene una brisa si y solo si hay un pozo en una celda vecina. Esto debe ser definido por cada celda, por ahora lo hacemos sólo en las celdas relevantes que hemos estado mencionando:
  - $R_2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
  - $R_3: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
- Las sentencias precedentes son verdaderas en todos los mundos de Wumpus. Ahora incorporamos las dos percepciones de brisa correspondientes a las celdas que el agente visitó en este mundo particular. Cuando el agente se encuentra en la celda [2,1], tenemos:
  - $R_4 : \neg B_{1,1}$ (no hay brisa en la celda inicial).
  - $R_5: B_{2,1}$ (se percibe brisa en la celda [2,1]).

A partir de estas reglas, puede aplicarse razonamiento proposicional para deducir información no observable. Realicemos un proceso de inferencia simple. Nuestro objetivo ahora es decidir si BC $\models \alpha$ para alguna sentencia α. Por ejemplo, ¿está $\neg P_{1,2}$ implicado por nuestra BC? Nuestro primer algoritmo de inferencia es el verificador de modelos, que implementa de forma directa la definición de implicación: enumerar los modelos y comprobar que α es verdadera en cada modelo en el que BC es verdadera. Los modelos son, en este contexto, asignaciones de valores de verdad (verdadero/falso) a cada símbolo proposicional.

Volviendo a nuestro ejemplo del mundo del Wumpus, los símbolos proposicionales relevantes son $B_{1,1}$, $B_{2,1}$, $P_{1,1}$, $P_{1,2}$, $P_{2,1}$, $P_{2,2}$, y $P_{3,1}$. Con siete símbolos, hay $2^7 = 128$ modelos posibles; en tres de estos, BC es verdadero (podemos verlo en la figura a continuación). En esos tres modelos, $\neg P_{1,2}$, es verdadera, por lo que no hay ningún pozo en [1,2]. En cambio, $P_{2,2}$, es verdadera en dos de los tres modelos y falsa en uno, de modo que aún no es posible determinar si hay un pozo en [2,2].

La siguiente imagen reproduce de manera más precisa el razonamiento que vimos en la unidad anterior al presentar la BC del mundo del Wumpus.

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	*false*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*false*	*false*
false	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*false*	*true*	*false*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	*true*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*false*	*true*	*true*	*false*
false	*true*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*false*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*true*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	*true*	*true*	*true*	*true*	*true*	*true*	*false*	*true*	*true*	*false*	*true*	*false*

Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 221.

Un algoritmo general para decidir la implicación en lógica proposicional sería como el siguiente, donde TV significa tabla de verdad. La función ¿VERDADERO-LP? Devuelve *verdadero* si una sentencia es verdadera en un modelo. La variable *modelo* representa un modelo parcial, una asignación de algunos de los símbolos.

```
función ¿IMPLICACIÓN-EN-TV?(BC,α) devuelve verdadero o falso
    entradas: BC, la base de conocimiento, una sentencia en lógica proposicional
              α, la sentencia implicada, una sentencia en lógica proposicional
    símbolos ← una lista de símbolos proposicionales de la BC y α
    devolver COMPROBAR-TV(BC,α,símbolos,{ })

función COMPROBAR-TV(BC,α,símbolos,modelo) devuelve verdadero o falso
    si ¿VACÍA?(símbolos) entonces
        si ¿VERDADERO-LP?(BC,modelo) entonces devolver ¿VERDADERO-LP?(α,modelo)
        sino devolver verdadero  //cuando la BC es falsa, siempre devolver falso
    sino hacer
        P ← PRIMERO(símbolos)
        resto ← RESTO(símbolos)
        devolver (COMPROBAR-TV(BC,α,resto,modelo U {P = verdadero}) y
            COMPROBAR-TV(BC,α,resto,modelo U {P = falso}))
```

Como puede verse, este algoritmo realiza una enumeración recursiva de un espacio finito de asignaciones a símbolos. Es sólido, porque garantiza conclusiones válidas siempre que la BC sea verdadera (implementa directamente la definición de implicación), y completo, ya que examina todos los modelos posibles. No obstante, su complejidad temporal es exponencial ($O(2^n)$), lo que limita su uso a dominios pequeños. Su complejidad espacial es solo O(n) ya que la enumeración se realiza en profundidad primero.

Hasta aquí, comprendimos cómo determinar si una proposición se sigue lógicamente de una base de conocimiento mediante la verificación de modelos: un proceso exhaustivo que evalúa todas las combinaciones posibles de valores de verdad. Si bien este método es conceptualmente claro y garantiza la corrección de las inferencias, su costo computacional crece exponencialmente con el número de símbolos involucrados. Por esta razón,

la inteligencia artificial recurre a métodos de inferencia basados en prueba de teoremas, que aplican reglas lógicas de deducción directamente sobre las sentencias, evitando la enumeración de modelos.

En lógica proposicional, demostrar un teorema consiste en construir una cadena de inferencias válidas que vinculan las proposiciones conocidas (premisas) con la proposición que se desea probar (conclusión). Este proceso, denominado demostración, permite establecer la validez de una afirmación mediante la manipulación simbólica, sin necesidad de examinar explícitamente todos los modelos posibles.

Antes de adentrarnos en los algoritmos que prueban teoremas necesitamos conocer algunos conceptos más relacionados con la inferencia.

El primer concepto se llama **equivalencia lógica.** Decimos que dos proposiciones α y β son lógicamente equivalentes si son verdaderas en el mismo conjunto de modelos, lo cual se representa como $\alpha \equiv \beta$. Por ejemplo, podemos ver fácilmente utilizando tablas de verdad que $P \wedge Q$ y $Q \wedge P$ son equivalentes, ya que en toda situación donde una es verdadera la otra también lo es. Estas equivalencias cumplen un papel similar al de las identidades algebraicas: permiten transformar y simplificar expresiones sin alterar su significado lógico. Una definición alternativa de equivalencia sería: dadas dos proposiciones cualesquiera α y β son equivalentes si y solo si cada una implica a otra:

$$\alpha \equiv \beta \text{ si y solo si } \alpha \models \beta \text{ y } \beta \models \alpha$$

Otras equivalencias lógicas estándar son las siguientes:

$$\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}$$

Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 223.

El segundo concepto que necesitamos es el de **validez**. Una sentencia es válida si es verdadera en *todos* los modelos. Por ejemplo, la sentencia $P \vee \neg P$ es válida. Las sentencias válidas también se conocen como tautologías: son *necesariamente* verdaderas. Debido a que la sentencia *Verdadero* es verdadera en todos los modelos, cada sentencia válida es lógicamente equivalente a *Verdadero*. Las tautologías desempeñan un papel esencial en la inferencia porque describen razonamientos universalmente correctos.

El concepto final que vamos a necesitar es el de **satisfacibilidad**. Una sentencia es satisfacible cuando resulta verdadera en algún modelo. Por ejemplo, la base de conocimiento

dada anteriormente, $(R_1 \wedge R_2 \wedge R_3 \wedge R_4 \wedge R_5 \wedge)$, es satisfacible porque hay tres modelos en los que es verdadera. La satisfacibilidad se puede comprobar enumerando los posibles modelos hasta que se encuentre uno que satisfaga la sentencia.

Validez y satisfacibilidad están conectadas: α es válida si y solo si ¬α es **insatisfacible**. En contraposición, α es satisfacible si y solo si ¬α no es válida. A su vez, obtenemos el siguiente resultado útil:

$$\alpha \models \beta \text{ si y solo si la sentencia } (\alpha \wedge \neg\beta) \text{ es insatisfacible.}$$

Demostrar β a partir de verificando la insatisfacibilidad de $(\alpha \wedge \neg\beta)$ corresponde exactamente a la técnica matemática estándar conocido como prueba por refutación o prueba por contradicción. Se asume temporalmente que β es falsa y se demuestra que ello conduce a una contradicción con los axiomas conocidos α . Esa contradicción es precisamente lo que se expresa al afirmar que la sentencia $(\alpha \wedge \neg\beta)$ es insatisfacible.

Reglas de inferencia y construcción de pruebas

La inferencia lógica se realiza mediante la aplicación de reglas de inferencia que derivan en una prueba, entendida como una cadena de conclusiones que conducen al objetivo deseado.

La regla más conocida se llama Modus Ponens (del latín "*modo que afirma*") y se escribe:

$$\frac{\alpha \Rightarrow \beta,\ \alpha}{\beta}$$

Esta notación indica que, si contamos con sentencias de la forma α⇒β y α , entonces podemos inferir β. Por ejemplo, si disponemos de *(WumpusAdelante ∧ WumpusVivo) ⇒ Disparar* y también de *(WumpusAdelante ∧ WumpusVivo)*, entonces es válido inferir Disparar.

Otra regla de inferencia útil se conoce como eliminación de la conjunción (And-Elimination) Esta regla establece que, a partir de una conjunción, puede inferirse cualquiera de sus componentes por separado:

$$\frac{\alpha \wedge \beta}{\alpha}$$

Por ejemplo, de (*WumpusAdelante ∧ WumpusVivo*), se puede inferir *WumpusVivo*.

Además de estas reglas, todas las equivalencias lógicas presentadas anteriormente pueden emplearse también como reglas de inferencia, dado que cada transformación preserva la verdad de las sentencias. Por ejemplo, la equivalencia correspondiente a la eliminación del bicondicional genera las siguientes dos reglas de inferencia:

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)} \text{ y } \frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

Considerando los posibles valores de verdad de α y β, se puede demostrar que Modus Ponens y la eliminación de la conjunción son sólidos. Estas reglas se pueden utilizar entonces en cualquier instancia particular donde apliquen, generando inferencias sólidas sin necesidad de enumerar modelos.

Veamos cómo aplicar estas reglas de inferencia y equivalencias en el mundo del Wumpus. Partimos de la base de conocimiento que contiene las sentencias R_1 a R_5 y planteamos la demostración de $\neg P_{1,2}$, es decir, que no hay pozo en [1,2]:

1. Aplicamos la eliminación bicondicional en R_2 para obtener

$$R_6: \left(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})\right) \wedge \left((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}\right)$$

2. Aplicamos eliminación de la conjunción en R6 para obtener la segunda parte

$$R_7: \left((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}\right)$$

3. La equivalencia lógica para los contrapositivos da

$$R_8: \left(\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1})\right)$$

4. Aplicamos Modus Ponens con R_8 y la percepción R_4 para obtener:

$$R_9: \neg(P_{1,2} \vee P_{2,1})$$

5. Por último, aplicamos la ley de De Morgan

$$R_{10}: \neg P_{1,2} \wedge \neg P_{2,1}$$

Así, el agente concluye que ni [1,2] ni [2,1] contienen un pozo. Esta demostración representa un ejemplo concreto de cómo las reglas de inferencia permiten derivar conclusiones válidas sin necesidad de examinar todos los modelos posibles.

Cualquier algoritmo de búsqueda de los que aprendimos en el módulo 3 puede ser usado para encontrar una secuencia de pasos que constituya una prueba como la que armamos utilizando las reglas. Sólo necesitamos definir un problema de prueba como el siguiente:

- Estado inicial: la base de conocimiento inicial.
- Acciones: el conjunto de acciones consiste en todas las reglas de inferencia aplicadas a todas las sentencias que coinciden con la mitad superior de la regla de inferencia.
- Resultado: el resultado de una acción es agregar la sentencia en la mitad inferior de la regla de inferencia.
- Objetivo: el objetivo es un estado que contenga la sentencia que estamos tratando de probar.

A diferencia de la verificación de modelos, que debe considerar todas las proposiciones de la base, la búsqueda de pruebas permite descartar información irrelevante. En el

ejemplo del Wumpus, la deducción de $\neg P_{1,2}$ se logra utilizando solo las reglas R_2 y R_4, sin necesidad de analizar otras proposiciones como $B_{2,1}$ o $P_{3,1}$. Esto muestra que, en muchos casos, la inferencia dirigida por reglas puede ser significativamente más eficiente.

Una característica importante de los sistemas lógicos clásicos es su monotonía: una vez que una conclusión ha sido inferida, agregar nueva información a la base de conocimiento no puede invalidarla. Formalmente:

$$si\ BC \vDash \alpha\ entonces\ BC \wedge \beta \Rightarrow \alpha$$

Esto significa que el conocimiento en lógica proposicional es cumulativo: cada nueva información puede generar conclusiones adicionales, pero nunca contradecir las ya derivadas. En el ejemplo del Wumpus, si el agente ya dedujo que no hay pozo en [1,2], incorporar la información de que "hay exactamente ocho pozos en toda la cueva" no altera esa conclusión, aunque podría habilitar nuevas inferencias globales sobre el entorno.

La monotonía distingue a la lógica clásica de otras formas de razonamiento más avanzadas (como las lógicas no monótonas o probabilísticas), donde las conclusiones pueden revisarse al incorporar nueva evidencia.

Prueba por resolución

Veamos ahora la regla de inferencia conocida como resolución, que produce un algoritmo completo de inferencia cuando se combina con cualquier algoritmo de búsqueda también completo.

Comenzamos usando una versión simple de la regla de resolución en el mundo del Wumpus. Consideremos el siguiente escenario: el agente regresa de [2,1] a [1,1] y luego va a [1,2], donde percibe un hedor, pero no una brisa. Agregamos los siguientes hechos a la base de conocimiento:

$$R_{11}: \neg B_{1,2}$$

$$R_{12}: B_{1,2} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{1,3})$$

Por el mismo proceso que nos llevó a R_{10} anteriormente, podemos derivar la ausencia de pozos en [2,2] y [1,3] (recordemos que ya es sabido que en [1,1] no hay un pozo):

$$R_{13}: \neg P_{2,2}$$

$$R_{14}: \neg P_{1,3}$$

También podemos aplicar la eliminación bicondicional a R_3, seguido de Modus Ponens con R_5, para obtener el hecho de que hay un pozo en [1,1], [2,2] o [3,1]:

$$R_{15}: P_{1,1} \vee P_{2,2} \vee P_{3,1}$$

Ahora aplicamos por primera vez la regla de resolución, el literal $\neg P_{2,2}$ en R_{13} se resuelve con el literal $P_{2,2}$ en R_{15} para dar el resolvente:

$$R_{16}: P_{1,1} \vee P_{3,1}$$

Dicho en palabras, si hay un pozo en uno de [1,1], [2,2] y [3,1] y no está en [2,2], entonces está en [1,1] o [3,1]. Del mismo modo, el literal $\neg P_{1,1}$ en R_1 se resuelve con el literal $P_{1,1}$ en R_{16} para dar:

$$R_{17}: P_{3,1}$$

En consecuencia, si hay un pozo en [1,1] o [3,1] y no está en [1,1], entonces está en [3,1].

La regla de resolución se aplica solo a cláusulas (es decir, disyunciones de literales), por lo que parecería ser relevante solo para bases de conocimiento y consultas que consisten en cláusulas. ¿Cómo, entonces, puede llevar a un procedimiento de inferencia completo para toda la lógica proposicional? La respuesta es que cada sentencia de la lógica proposicional es lógicamente equivalente a una conjunción de cláusulas.

Una sentencia expresada como una conjunción de cláusulas se dice que está en forma normal conjuntiva o CNF. Veamos ahora un procedimiento para convertir a CNF. Utilicemos como ejemplo la sentencia $B_{1,1} \Leftrightarrow (P_{1,2} \; P_{2,1})$. Los pasos son los siguientes:

1. Eliminar $\Leftrightarrow$, reemplazando $\alpha \Leftrightarrow \beta$ con $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

$$\left(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})\right) \wedge \left((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}\right)$$

2. Eliminar $\Rightarrow$, remplazando $\neg\alpha \vee \beta$.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. La CNF requiere que $\neg$ solo aparezca en literales, por lo que "movemos $\neg$ hacia adentro" aplicando de manera repetida las siguientes equivalencias lógicas:

$$\neg(\neg\alpha) \equiv \alpha \text{ (eliminación de la doble negación)}$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \text{ (De Morgan)}$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \text{ (De Morgan)}$$

En el ejemplo, sólo necesitamos una aplicación de la última regla:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge \left((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1}\right)$$

4. Ahora tenemos una sentencia que contiene operadores e anidados y aplicados a literales. Si aplicamos la ley distributivita, distribuyendo sobre donde sea posible

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

La sentencia original queda ahora en CNF, es decir, en una conjunción de tres cláusulas. Aunque resulta menos legible, puede utilizarse como entrada en un procedimiento de resolución.

La resolución, al reducir el razonamiento lógico a manipulación de cláusulas, constituye la base de los sistemas automáticos de demostración de teoremas. Además, garantiza solidez (solo produce conclusiones válidas) y completitud (si una conclusión es válida, puede derivarse por resolución).

Sin embargo, en muchas situaciones prácticas, no se necesita toda la potencia de la resolución. Algunas bases de conocimiento del mundo real satisfacen ciertas restricciones en la forma de sus sentencias, lo que les permite utilizar un algoritmo de inferencia más restringido y eficiente.

Una de estas formas es la cláusula definida, una disyunción de literales en la que exactamente uno es positivo. Por ejemplo, la cláusula $(\neg L_{1,1} \vee \neg Brisa \vee B_{1,1})$ es una cláusula definida, mientras que $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1})$ no lo es, porque tiene dos cláusulas positivas.

Un caso ligeramente más general es la cláusula de Horn, una disyunción de literales en la que como máximo uno es positivo. Todas las cláusulas definidas son cláusulas de Horn, así como aquellas sin literales positivos, conocidas como cláusulas objetivo. El razonamiento basado en cláusulas de Horn constituye la base de los sistemas expertos y de lenguajes de programación lógica como Prolog, que implementan estas reglas mediante inferencias automáticas.

Los sistemas que utilizan cláusulas de Horn aplican dos estrategias básicas de inferencia: **encadenamiento hacia adelante** (forward chaining) y **encadenamiento hacia atrás** (backward chaining). Ambas se apoyan en reglas de la forma "si se cumplen las condiciones, entonces se infiere la conclusión". El encadenamiento hacia adelante es un razonamiento dirigido por los datos. El sistema parte de los hechos conocidos y aplica las reglas cuya parte izquierda se cumple, agregando nuevas conclusiones hasta alcanzar el objetivo o no poder derivar más hechos. Se utiliza principalmente en sistemas expertos y entornos de diagnóstico, porque genera todas las conclusiones posibles a partir del conocimiento inicial.

El encadenamiento hacia atrás, en cambio, es un razonamiento dirigido por metas. El sistema parte de la conclusión buscada e identifica qué reglas podrían justificarla. Si encuentra una regla del tipo $(P_1 \wedge P_2) \Rightarrow Q$, intenta demostrar cada condición P_1 y P_2 como subobjetivos. Este método es la base de los motores de inferencia en Prolog y de los sistemas de consulta lógica.

Ambos enfoques son sólidos y completos para bases de conocimiento expresadas mediante cláusulas de Horn, y muestran cómo el razonamiento lógico puede implementarse operativamente en agentes racionales.

Actividad

Resuelva la **actividad 2** de este módulo para reforzar los contenidos de lógica proposicional y CNF.

Unidad 10: Lógica de primer orden

En la unidad anterior aprendimos que la lógica proposicional constituye una poderosa herramienta para representar hechos y razonar sobre ellos. Sin embargo, su alcance es limitado: aunque permite describir el estado de un entorno y realizar inferencias válidas, cada proposición se refiere a un hecho indivisible. Cuando el mundo está compuesto por muchos objetos y relaciones entre ellos, la lógica proposicional resulta insuficiente para expresar con concisión lo que deseamos representar.

Pensemos nuevamente en el mundo del Wumpus. En lógica proposicional fue necesario escribir una regla distinta sobre brisas y pozos para cada celda, por ejemplo:

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

Pero intuitivamente, podríamos expresar todo esto con una única afirmación general: "*Las celdas adyacentes a pozos son ventosas.*"



Esa capacidad de expresar reglas generales y relaciones entre objetos de manera compacta es justamente lo que introduce la lógica de primer orden (LPO), también llamada lógica de predicados. Esta lógica amplía el lenguaje proposicional al incorporar objetos, propiedades, relaciones y funciones, lo que permite modelar el mundo con un nivel de detalle más realista y estructurado.

Un lenguaje de representación establece la forma en que se describen los hechos, las entidades y las relaciones de un dominio. En programación, las estructuras de datos —como listas, arreglos o registros— pueden utilizarse para representar información, pero no ofrecen mecanismos formales de inferencia: las operaciones que permiten razonar sobre esos datos dependen siempre de procedimientos específicos definidos por el programador.

En cambio, los lenguajes lógicos son declarativos, lo que significa que separan el conocimiento de los procedimientos de inferencia. Las reglas de la lógica permiten derivar conclusiones de manera general, sin depender del dominio particular. Por ejemplo, una base de datos o un programa pueden registrar que "en [2,2] hay un pozo", pero solo la lógica puede expresar afirmaciones como: "Si hay un pozo en una celda, entonces las celdas adyacentes tendrán brisa."

La lógica proposicional ya nos brindaba esa capacidad mediante conectivos como la disyunción y la negación, pero no podía abstraer sobre los objetos ni expresar que una relación se aplica a "todas las celdas adyacentes". La lógica de primer orden resuelve esta limitación.

Por otro lado, el lenguaje natural, como el español, es extremadamente expresivo y permite hablar de personas, lugares, acciones, causas, tiempos o intenciones. Sin embargo, su ambigüedad y dependencia del contexto lo hacen inadecuado para un sistema de razonamiento automático. Cuando alguien dice "¡Mira!", el significado depende de la situación y no puede recuperarse solo a partir de la frase. Además, muchas palabras poseen múltiples significados según el contexto ("banco" puede ser una institución financiera o un asiento), y el mismo concepto puede expresarse de maneras muy diferentes.

La lógica formal busca captar la estructura conceptual del lenguaje natural, en particular su capacidad para referirse a objetos y relaciones, pero sin incorporar sus ambigüedades. Así, mientras el lenguaje natural es rico pero impreciso, la lógica de primer orden es precisa pero controlada, y su semántica está completamente definida.

Esta lógica combina lo mejor de ambos mundos:

- De la lógica proposicional, hereda la claridad formal y la independencia del contexto.

- Del lenguaje natural, toma la capacidad de describir objetos, propiedades y relaciones de manera estructurada.

La lógica de primer orden se organiza en torno a tres conceptos fundamentales que imitan la estructura del mundo real:

1. Objetos: son las entidades individuales que existen en el dominio. Pueden ser personas, lugares, números, cosas o conceptos específicos. Ejemplo: el Wumpus, la celda [2,3], la flecha, la cueva.
2. Relaciones: describen vínculos entre objetos. Pueden ser propiedades unarias (aplicadas a un solo objeto) o relaciones de varios elementos. Ejemplo: "ser ventosa" es una propiedad; "estar junto a" es una relación binaria entre celdas. En notación lógica: *Adyacente*([1,1],[1,2]) o *Ventosa*([2,1]).
3. Funciones: son relaciones especiales en las que a cada en trada le corresponde exactamente una salida. Por ejemplo: padre de, mejor amigo, más de un, principio de.

Estas categorías permiten describir cualquier afirmación del mundo real en términos de objetos que poseen propiedades y mantienen relaciones. Algunos ejemplos pueden ser:

- "Uno más dos es igual a tres". Objetos: uno, dos, tres, uno más dos; relación: es igual a; función: más. ("Uno más dos" es un nombre para el objeto que se obtiene aplicando la función "más" a los objetos "uno" y "dos." "Tres" es otro nombre para este objeto.)
- "Las celdas vecinas al Wumpus son apestosas." Objetos: Wumpus, celdas; propiedad: apestosas; relación: vecina.
- "El malvado Rey Juan gobernó Inglaterra en 1200." Objetos: Juan, Inglaterra, 1200; relación: gobernó durante; propiedades: malvado, rey.

De esta manera, la lógica de primer orden constituye un lenguaje más cercano a la forma en que pensamos y hablamos sobre el mundo, pero con la precisión formal necesaria para el razonamiento automatizado.

La principal diferencia entre la lógica proposicional y la lógica de primer orden radica en el compromiso ontológico hecho por cada lenguaje, es decir, lo que asume sobre la naturaleza de la *realidad*. Matemáticamente, este compromiso se expresa a través de la naturaleza de los modelos formales con respecto a los cuales se define la verdad de las sentencias. Por ejemplo, en la lógica proposicional el mundo se compone solo de hechos que pueden ser verdaderos o falsos. Cada proposición representa una afirmación completa del tipo "hay un pozo en [1,2]" o "la celda [1,1] es segura". La lógica de primer orden asume además que existen objetos y relaciones entre ellos. El mundo ya no es un con-

junto de hechos aislados, sino una red estructurada de entidades que interactúan. Cada proposición puede referirse a propiedades de objetos ("Juan es un rey") o a vínculos entre varios de ellos ("Juan gobernó Inglaterra").

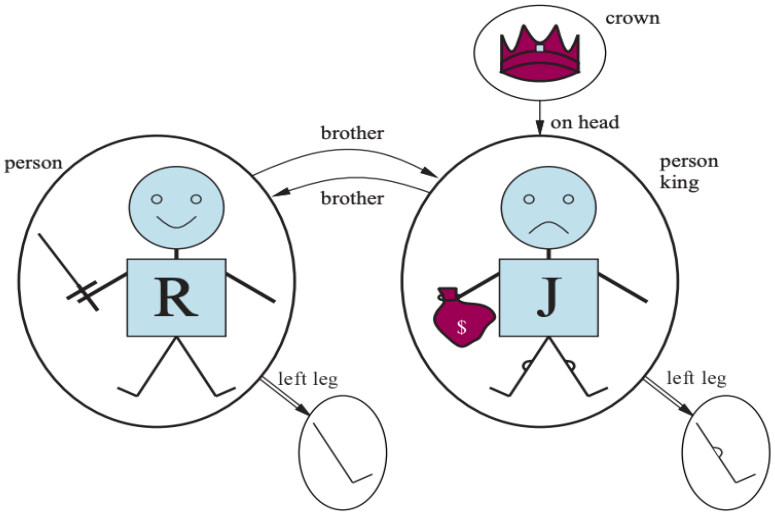
Este compromiso ontológico es una gran fortaleza de la lógica (tanto proposicional como de primer orden), porque nos permite comenzar con afirmaciones verdaderas e inferir otras afirmaciones verdaderas. Es especialmente poderoso en dominios donde cada proposición tiene límites claros, como las matemáticas o el mundo del Wumpus, donde una celda tiene o no tiene un pozo; ahí no hay posibilidad de una celda con una hendidura vagamente similar a un pozo.

Una lógica también puede caracterizarse por sus compromisos epistemológicos: los posibles estados de conocimiento que permite con respecto a cada hecho. Tanto en la lógica proposicional como en la de primer orden, una sentencia representa un hecho y el agente cree que la sentencia es verdadera, cree que es falsa o no tiene opinión. Por lo tanto, estas lógicas tienen tres estados posibles de conocimiento con respecto a cualquier sentencia.

Sintaxis y semántica

En unidades anteriores vimos que los modelos de un lenguaje lógico son representaciones formales de un mundo posible. Cada modelo vincula el vocabulario de las sentencias lógicas a elementos del mundo, de modo que la verdad de cualquier sentencia pueda ser determinada. En la lógica proposicional, los modelos asignan valores de verdad (Verdadero o Falso) a cada proposición. En cambio, los modelos de la lógica de primer orden incluyen objetos y relaciones entre ellos.

Cada modelo contiene un dominio, que es el conjunto de todos los objetos que existen en ese mundo. El dominio debe ser no vacío, pues todo modelo debe tener al menos un elemento del cual hablar. Además, los objetos pueden estar vinculados mediante relaciones, que se representan como conjuntos de tuplas ordenadas. Una tupla es una colección de objetos dispuestos en un orden fijo y se escribe con paréntesis angulares que rodean los objetos. Veamos un ejemplo:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 257.

La figura muestra un modelo con 5 objetos, Ricardo Corazón de León, Rey de Inglaterra desde 1189 hasta 1199; su hermano menor, el malvado Rey Juan, que gobernó desde 1199 hasta 1215; las piernas izquierdas de Ricardo y Juan; y una corona. Estos objetos están relacionados de varias maneras. Por ejemplo, Ricardo y Juan son hermanos, la relación de hermandad en este modelo es el conjunto:

$$\{\langle Ricardo\ Corazon\ de\ Leon,\ Rey\ Juan\rangle, \langle Rey\ Juan,\ Ricardo\ Corazon\ de\ Leon\rangle\}$$

De manera similar, el modelo incluye una corona colocada sobre la cabeza del Rey Juan, la relación "está sobre la cabeza de" contiene la tupla: $\langle Corona,\ Rey\ Juan\rangle$. Ambas relaciones ("hermano de" y "sobre la cabeza de") se conocen como binarias, porque vinculan pares de objetos. Mientras que las relaciones unarias describen propiedades que afectan a un solo elemento, como *ser persona* o *ser rey*.

A su vez, existen ciertos tipos de relaciones conocidas como **funciones**, en el sentido de que un objeto dado debe estar relacionado con exactamente un objeto de esta manera. Por ejemplo, cada persona tiene una pierna izquierda, por lo que el modelo tiene una función unaria de "pierna izquierda", una asignación de una tupla de un elemento a un objeto, que incluye las siguientes asignaciones:

$$\langle Ricardo\ Corazon\ de\ Leon\rangle \rightarrow pierna\ izquierda\ de\ Ricardo$$

$$\langle Rey\ Juan\rangle \rightarrow pierna\ izquierda\ de\ Juan$$

Las funciones en un modelo deben ser totales, es decir, deben tener un valor definido para cada objeto del dominio. En algunos casos esto obliga a introducir objetos ficticios —como una "pierna invisible"— para completar las asignaciones, aunque en la práctica rara vez se los menciona explícitamente.

Hasta ahora, hemos descripto los elementos que componen los modelos para la lógica de primer orden. La otra parte esencial de un modelo es el vínculo entre esos elementos y el vocabulario de las sentencias lógicas, veamos entonces la sintaxis de esta lógica.

Los elementos sintácticos básicos son los símbolos que representan objetos, relaciones y funciones. Podemos agruparlos en tres clases:

1. Constantes, que nombran objetos individuales (por ejemplo, *Juan*, *Ricardo*).
2. Predicados, que designan relaciones o propiedades (por ejemplo, *Hermano, Rey, Persona*, *SobreLaCabeza*, *Corona*).
3. Funciones, que asocian objetos con otros objetos (por ejemplo, *PiernaIzquierda*).

Cada símbolo de predicado y de función tiene una aridad que determina cuántos argumentos admite. Para que un modelo pueda establecer si una sentencia es verdadera o falsa, necesita algo más que objetos, relaciones y funciones: requiere una interpretación que indique con precisión a qué objetos, relaciones y funciones remiten los símbolos de constante, predicado y función. Una posible interpretación para nuestro ejemplo, que un lógico llamaría la interpretación deseada, es la siguiente:

- *Ricardo* se refiere a Ricardo Corazón de León y *Juan* se refiere al malvado Rey Juan.

- *Hermano* se refiere a la relación de hermandad, es decir, el conjunto de tuplas de objetos dados en la primera ecuación que vimos; *SobreLaCabeza* es una relación que se mantiene entre la corona y el Rey Juan; *Persona*, *Rey*, y *Corona* son relaciones unarias que identifican personas, reyes y coronas.

- *PiernaIzquierda* se refiere a la función de "pierna izquierda" como se define en la segunda ecuación que describimos.

Hay muchas otras interpretaciones posibles, por supuesto. Por ejemplo, una interpretación relaciona *Ricardo* a la corona y *Juan* a la pierna izquierda del Rey Juan. Hay cinco objetos en el modelo, por lo que hay 25 interpretaciones posibles solo para las constantes *Ricardo* y *Juan*.

En resumen, un modelo en lógica de primer orden consiste en un conjunto de objetos y una interpretación que mapea símbolos constantes a objetos, símbolos de función a funciones en esos objetos y símbolos de predicado a relaciones. Al igual que con la lógica proposicional, implicación y validez se definen en término de *todos los modelos posibles*. Debido a que el número de modelos de primer orden no está acotado, no podemos comprobar la implicación enumerándolos todos (como hicimos para la lógica proposicional). Incluso si el número de objetos es restringido, el número de combinaciones puede ser muy grande.

Por otra parte, un término es una expresión lógica que refiere a un objeto. Los símbolos constantes son términos simples, pero no siempre es conveniente tener un símbolo distinto para nombrar cada objeto. Una función aplicada a un objeto produce un término complejo, por ejemplo, *PiernaIzquierda*(Juan) o *Padre*(Ricardo). Ahora que tenemos términos para referirnos a objetos y símbolos de predicado para referirnos a relaciones, podemos combinarlos para formar **sentencias atómicas** que declaran hechos. Una sentencia atómica se forma a partir de un símbolo de predicado seguido opcionalmente de una lista de términos entre paréntesis, por ejemplo:

$$Hermano(Ricardo, Juan)$$

Dada la interpretación deseada que vimos anteriormente, esto establece que Ricardo Corazón de León es el hermano del Rey Juan. A su vez, las sentencias atómicas pueden tener términos complejos como argumentos. Por ejemplo, la sentencia:

$$Casado\big(Padre(Ricardo), Madre(Juan)\big)$$

Establece que el padre de Ricardo Corazón de León está casado con la madre del Rey Juan, siempre bajo una interpretación adecuada.

Una sentencia atómica es verdadera en un modelo si la relación correspondiente al predicado es válida para los objetos designados por sus argumentos. Las sentencias atómicas pueden combinarse mediante los mismos conectivos de la lógica proposicional: negación ($\neg$), conjunción ($\wedge$), disyunción ($\vee$), implicación ($\Rightarrow$) y bicondicional ($\Leftrightarrow$). Así, es posible construir expresiones como:

- ¬Hermano(PiernaIzquierda(Ricardo), Juan)
- Hermano(Ricardo, Juan) ∧ Hermano(Juan, Ricardo)
- Rey(Ricardo) ∨ Rey(Juan)
- ¬Rey(Ricardo) ⇒ Rey(Juan)

Estas sentencias complejas heredan su semántica de la lógica proposicional, pero ahora sus componentes se refieren a objetos y relaciones del dominio. Una vez que contamos con una lógica que incluye objetos, necesitamos expresar propiedades que involucren conjuntos completos de ellos, en lugar de enumerarlos uno por uno. Los cuantificadores permiten justamente eso. La lógica de primer orden incorpora dos cuantificadores estándar: el universal y el existencial.

Cuantificación universal (∀)

El cuantificador universal expresa que una proposición es verdadera para todos los objetos del dominio. Por ejemplo, $\forall x\ Rey(x) \Rightarrow Persona(x)$ se lee como "todos los reyes son personas". El símbolo ∀ significa "todo", por lo que la sentencia también podría leerse como "Para todo *x*, si *x* es un rey, entonces *x* es una persona".

El símbolo *x* se llama variable. Por convención las variables son letras en minúscula. Una variable es un término por sí sola, y como tal, también puede servir como argumento de una función, por ejemplo, *PiernaIzquierda(x)*. Un término sin variables se conoce como término base.

Intuitivamente, la sentencia ∀*x P*, donde *P* es cualquier sentencia lógica, dice que *P* es verdadero para cada objeto *x*. Más precisamente, ∀x P es verdadera en un modelo si P resulta verdadera en todas las interpretaciones ampliadas derivadas de ese modelo, es decir, en todas aquellas en las que x se asigna a cada objeto del dominio. Dicho de otra manera, ∀*x P* es verdadera si *P(x)* se cumple para todos los objetos.

Cuantificación existencial (∃)

La cuantificación universal hace declaraciones sobre cada objeto. De manera similar, podemos hacer una declaración sobre algún objeto sin nombrarlo, utilizando un cuantificador existencial. Para decir, por ejemplo, que el Rey Juan tiene una corona en la cabeza, escribimos

$$\exists x\ Corona(x) \wedge SobreLaCabeza(x, Juan)$$

∃x se lee "Existe un *x* tal que..." o "Para algún *x*...".

Intuitivamente, la sentencia ∃*x P* dice que *P* es verdadero para al menos un objeto *x*. Más precisamente, ∃*x P* es verdadero en un modelo dado si *P* es verdadero en al menos una interpretación ampliada que asigna *x* a un elemento del dominio.

Cuantificadores anidados

Podemos querer expresar sentencias más complejas usando múltiples cuantificadores. El caso más simple es donde los cuantificadores son del mismo tipo. Por ejemplo, "Los hermanos son parientes" se puede escribir como:

$$\forall x \, \forall y \, Hermano(x,y) \Rightarrow Pariente(x,y)$$

Los cuantificadores consecutivos del mismo tipo se pueden escribir como un cuantificador con varias variables. Por ejemplo, para decir que el parentesco es una relación simétrica, podemos escribir:

$$\forall x, y \, Pariente(x,y) \Leftrightarrow Pariente(y,x)$$

En otros casos podemos tener mezclas. "Todo el mundo ama a alguien" significa que, para cada persona, hay alguien a quien esa persona ama:

$$\forall x \, \exists y \, Ama(x,y)$$

Por otro lado, para decir "Hay alguien que es amado por todos", escribimos:

$$\exists y \, \forall x \, Ama(x,y)$$

El orden de cuantificación es fundamental. Esto se aprecia con mayor claridad al usar paréntesis. La expresión ∀x (∃y Ama(x, y)) dice que *todos* tiene una propiedad particular, a saber, la propiedad de que aman a alguien. En cambio, la expresión y (∀x Ama(x, y)) indica que *alguien* en el mundo tiene una propiedad particular, a saber, la propiedad de ser amado por todos.

Los dos cuantificadores están íntimamente conectados entre sí a través de la negación. Decir que a nadie le gusta el brócoli equivale a afirmar que no existe ninguna persona a la que le guste, y viceversa. Así:

∀ x ¬Gusta(x, Brócoli) es equivalente a ¬∃x Gusta(x, Brócoli).

Podemos ir un paso más allá: "A todo el mundo le gusta el helado" significa que no hay nadie a quien no le guste el helado:

∀x Gusta(x, Helado) es equivalente a ¬ ∃x ¬Gusta(x, Helado)

Esto significa que negar una existencia equivale a afirmar que nada cumple la condición, mientras que negar una universalidad equivale a afirmar que al menos un caso la contradice. A su vez, la lógica de primer orden también permite afirmar que dos términos se refieren al mismo objeto, utilizando el símbolo de igualdad (=). Por ejemplo: *Padre(Juan) = Enrique* afirma que el padre de Juan es Enrique.

Asimismo, la negación de la igualdad permite distinguir objetos: ¬(x = y) indica que *x* e *y* son diferentes. Este recurso permite expresar con precisión afirmaciones como "Ricardo tiene al menos dos hermanos distintos":

$$\exists x, y\ [\text{Hermano}(x, \text{Ricardo}) \land \text{Hermano}(y, \text{Ricardo}) \land \neg(x = y)]$$

Ahora que hemos definido un lenguaje lógico más expresivo, avancemos en su uso. Emplearemos sentencias de ejemplo dentro de algunos dominios sencillos. En representación del conocimiento, un dominio es simplemente una parte del mundo sobre la cual deseamos expresar algún conocimiento.

Las sentencias se agregan a una base de conocimiento usando DECIR, exactamente como en la lógica proposicional. Tales sentencias se llaman aserciones. Por ejemplo, podemos afirmar que Juan es un rey, Ricardo es una persona y todos los reyes son personas:

$$DECIR(BC,\ Rey(Juan))$$

$$DECIR(BC,\ Persona(Ricardo))$$

$$DECIR(BC,\ \forall x\ Rey(x) \Rightarrow Persona(x))$$

Podemos hacer preguntas a la base de conocimiento usando PREGUNTAR. Por ejemplo: *PREGUNTAR(BC, Rey(Juan))* devuelve *verdadero*. Las preguntas hechas con PREGUNTAR se llaman peticiones u objetivos y sus respuestas pueden ser *Verdadero/Falso* o un conjunto de valores que satisfacen la condición.

La novedad es que, en lógica de primer orden, las consultas pueden contener variables, y por tanto su respuesta ya no es simplemente un valor de verdad. Cuando preguntamos: *PREGUNTAR(BC, Hermano(x, Juan))* no queremos saber si "Hermano(x,Juan)" es verdadero o falso, sino qué objetos del dominio cumplen esa condición. La respuesta es un conjunto de sustituciones para la variable *x*. Por ejemplo: {x/Ricardo}, {x/Gerardo}. Esto muestra que la lógica de primer orden puede utilizarse no solo para comprobar propiedades, sino también para recuperar información estructurada, tal como lo hacen las bases de datos relacionales o los sistemas expertos.

Una consulta puede incluir múltiples variables, cuyas sustituciones deben ser consistentes. Por ejemplo: *PREGUNTAR(BC, Padre(x) = y)* devolverá los pares ordenados: {x/Juan, y/Enrique}, {x/Ricardo, y/Isabel} según el modelo. El agente utiliza estas consultas para deducir hechos nuevos, construir planes, verificar restricciones o identificar relaciones no directamente observables.



Lo/a invito a resolver la actividad número 3 de este módulo con el conocimiento adquirido sobre lógica de primer orden.

Representación del conocimiento en el mundo del Wumpus

Una vez comprendidas la sintaxis y la semántica de la lógica de primer orden, el paso siguiente consiste en utilizarlas para representar un dominio real y significativo. En este caso, retomamos el mundo del Wumpus —ya analizado en la lógica proposicional— para mostrar cómo la lógica de primer orden permite describirlo de manera más elegante, compacta y general.

Recordemos que el agente de este mundo recibe un vector de percepción con cinco elementos. La sentencia de primer orden correspondiente, almacenada en la base de conocimiento, debe incluir la percepción y el momento en el que ocurrió; de lo contrario, el agente se confundirá acerca de cuándo observó cada cosa. Para ello, usamos enteros para los instantes de tiempo. Una sentencia de percepción típica sería:

Percepcion([Hedor, Brisa, Resplandor, Nada, Nada], 5)

Percepción es un predicado binario y *Hedor, Brisa, etc.*, son constantes dentro de una lista. Las acciones en el mundo del Wumpus pueden representarse mediante términos lógicos:

Girar(Derecha), Girar(Izquierda), Avanzar, Disparar, Agarrar, Escalar

Para determinar cuál es la mejor acción, el programa del agente ejecuta la consulta:

PREGUNTAR(BC, MejorAccion(a, 5))

lo que devuelve una sustitución como {a / Agarrar}. A partir de esa respuesta, el agente selecciona Agarrar como la acción a realizar. Los datos de percepción sin procesar implican ciertos hechos sobre el estado actual. Por ejemplo:

$$\forall t, h, r, c, g \ \ Percepcion([h, Brisa, r, c, g], t) \Rightarrow Brisa(t)$$

$$\forall t, h, r, c, g \ \ Percepcion([h, Nada, r, c, g], t) \Rightarrow \neg Brisa(t)$$

$$\forall t, h, b, c, g \ \ Percepcion([h, b, Resplandor, c, g], t) \Rightarrow Resplandor(t)$$

$$\forall t, h, b, c, g \ \ Percepcion([h, b, Nada, c, g], t) \Rightarrow \neg Resplandor(t)$$

Y así sucesivamente. Estas reglas exhiben una forma trivial del proceso de razonamiento llamado percepción. Observe la cuantificación a lo largo del tiempo *t*. En lógica proposicional, en cambio, necesitaríamos copias de cada sentencia para cada instante de tiempo.

El comportamiento "reflejo" simple también se puede implementar mediante sentencias de implicación cuantificadas. Por ejemplo, tenemos:

$\forall$ *t Resplandor(t)* $\Rightarrow$ *MejorAcción(Agarrar, t)*

Ahora que hemos representado las entradas y salidas del agente, veamos la manera de representar al entorno en sí mismo. Comencemos con los objetos. Podríamos asignar un nombre propio a cada celda —por ejemplo, Celda1,2—, pero entonces la adyacencia entre celdas debería añadirse como un hecho adicional, lo cual implicaría un nombre para cada par posible. Por eso es mejor usar un término complejo en el que cada fila y columna sean enteros; por ejemplo, podemos usar el término lista [1,2], y la adyacencia de cualquier par de celdas puede definirse como:

$$\forall x, y, a, b \; Adyacente([x, y], [a, b]) \Leftrightarrow (x = a \wedge (y = b-1 \vee y = b+1)) \vee (y = b \wedge (x = a-1 \vee x = a+1))$$

Usaremos el predicado unario *Pozo* que es verdadero en cada celda que contenga uno. Además, como hay exactamente un Wumpus, la constante *Wumpus* resulta adecuada para representarlo.

La ubicación del agente cambia con el tiempo, por lo que escribimos *En(Agente, s, t)* para indicar que el agente está en la celda *s* en el momento *t*. En cambio, podemos fijar la posición del Wumpus para todos los tiempos mediante: $\forall t \; En(Wumpus, [1,3], t)$. Entonces podemos decir que los objetos pueden estar en una sola ubicación a la vez:

$$\forall x, s_1, s_2, t \; En(x, s_1, t) \wedge En(x, s_2, t) \Rightarrow s_1 = s_2$$

A partir de su ubicación actual, el agente puede inferir propiedades de la celda a través de sus percepciones. Por ejemplo, si el agente está en una celda y percibe una brisa, entonces esa casilla es ventosa:

$$\forall s, t \; En(Agente, s, t) \wedge Brisa(t) \Rightarrow Ventoso(s)$$

Es útil saber que una celda es ventosa porque sabemos que los pozos no pueden moverse alrededor. Observemos que *Ventoso* no tiene argumento de tiempo.

Habiendo descubierto qué lugares son ventosos (u olorosos) y, muy importante, no ventosos (u no olorosos), el agente puede deducir dónde están los pozos (y dónde está el Wumpus). Mientras que la lógica proposicional necesita un axioma separado para cada celda (repasemos las percepciones R_2 y R_3 de la unidad anterior) y, además, necesitaría un conjunto diferente de axiomas para cada diseño geográfico del mundo, la lógica de primer orden solo necesita un axioma general:

$$\forall s \; Ventoso(s) \Leftrightarrow \exists r \; Adyacente(r, s) \wedge Pozo(r)$$

De manera similar, en la lógica de primer orden podemos cuantificar el tiempo, por lo que solo necesitamos un axioma de estado sucesor para cada predicado, en lugar de una copia diferente para cada instante de tiempo. Por ejemplo, el axioma para la flecha sería:

$$\forall t \; TieneFlecha(t+1) \Leftrightarrow (TieneFlecha(t) \wedge \neg Accion(Disparar, t))$$

De estas dos sentencias de ejemplo, podemos ver que la formulación de la lógica de primer orden no es menos concisa que la descripción en lenguaje natural que dimos al inicio de este módulo.

Con esto último damos por finalizado el módulo 4. Hemos pasado desde la idea de que un agente puede "saber cosas" hasta la compresión profunda de cómo representar y razonar sobre ese conocimiento mediante sistemas formales. Exploramos la lógica proposicional como primera aproximación al razonamiento simbólico, y luego avanzamos hacia la lógica de primer orden, un lenguaje capaz de modelar relaciones, objetos y estructuras complejas.

También hemos visto que el razonamiento lógico permite que un agente pueda derivar conclusiones válidas, justificar sus acciones, representar regularidades del mundo, inferir

hechos no observables y estructurar su conocimiento en axiomas generales. Sin embargo, la lógica presenta límites importantes, ya que exige conocimiento exacto, completo y sin incertidumbre. Sus inferencias son deterministas, y sus modelos crecen complejamente a medida que el dominio se amplía. Además, los agentes lógicos no aprenden por sí mismos: solo razonan sobre lo que les fue dicho.

Esta tensión abre naturalmente la puerta al último módulo de esta asignatura: "Introducción al aprendizaje automático", donde exploraremos un paradigma complementario: sistemas que aprenden a partir de datos, generalizan patrones y toman decisiones en contextos inciertos. El aprendizaje automático no reemplaza la lógica, pero amplía de manera esencial las capacidades del agente, permitiéndole descubrir regularidades sin necesidad de axiomas explícitos.

Módulo 4 | Glosario

Axioma: sentencia aceptada como verdadera dentro de un sistema lógico, que sirve como base para derivar otras afirmaciones mediante reglas de inferencia.

Base de conocimiento (BC): conjunto de sentencias que describen hechos, reglas y relaciones sobre un dominio, expresadas en un lenguaje formal que permite al agente razonar.

Cláusula de Horn: disyunción de literales con, como máximo, un literal positivo. Constituye la base de lenguajes de programación lógica como Prolog.

Conectivos lógicos: símbolos que permiten construir proposiciones compuestas a partir de proposiciones simples (¬, , , ,).

Inferencia: proceso mediante el cual se obtienen nuevas proposiciones a partir de otras, aplicando reglas lógicas.

Lógica de primer orden (LPO): lenguaje lógico que amplía la lógica proposicional al incluir objetos, relaciones, funciones y cuantificadores, permitiendo expresar conocimiento más complejo.

Lógica proposicional: sistema formal que utiliza proposiciones simples (verdaderas o falsas) y operadores lógicos para representar y manipular conocimiento.

Modelo: interpretación formal que asigna valores de verdad a las proposiciones y determina si una sentencia es verdadera o falsa en ese contexto.

Razonamiento lógico: proceso de derivar conclusiones válidas a partir de un conjunto de premisas siguiendo reglas formales.

Semántica: parte de la lógica que determina el significado y la verdad de las sentencias en relación con los modelos.

Sentencia atómica: proposición básica que no puede descomponerse en otras más simples.

Sintaxis: conjunto de reglas que define la estructura formal de las expresiones en un lenguaje lógico.

Tautología: sentencia que es verdadera en todos los modelos posibles, independientemente de los valores de verdad de sus componentes.

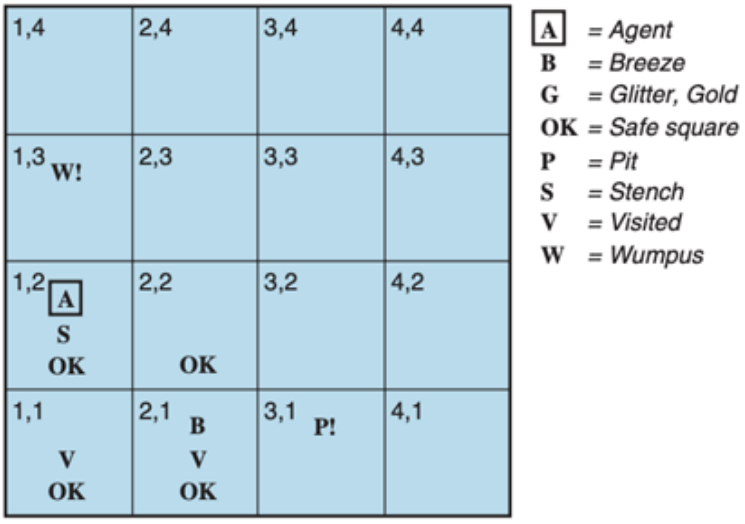
ACTIVIDAD 1

Análisis inferencial en el mundo del Wumpus

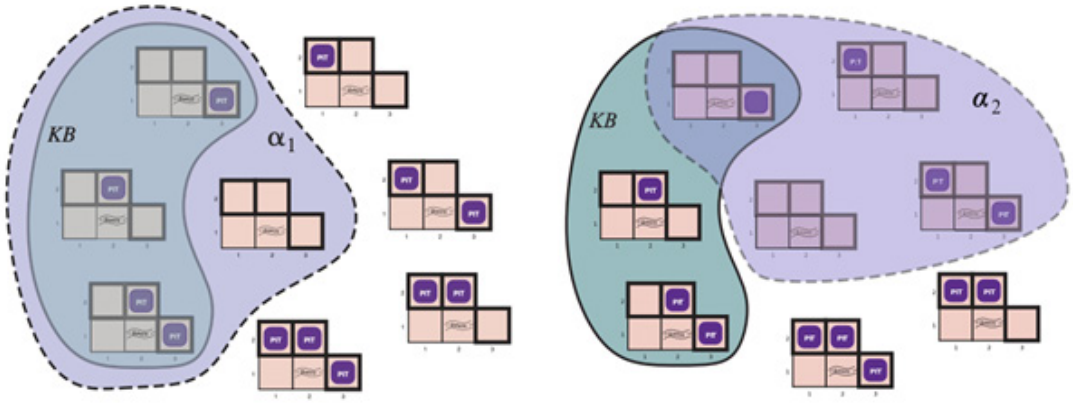
Supongamos que el agente ha avanzado hasta el punto mostrado en la figura (a): no percibió nada en [1,1], detectó una brisa en [2,1] y percibió un hedor en [1,2]. Ahora le preocupa el contenido de [1,3], [2,2] y [3,1]. Cada una de estas casillas puede contener un pozo y, como máximo, una de ellas puede alojar al Wumpus. Siguiendo el ejemplo de la figura (b), construya el conjunto de mundos posibles (debería obtener 32). Marque cuáles de esos mundos satisfacen la BC y en cuáles se cumple cada una de las siguientes sentencias:

- $= \alpha_2$ "No hay pozo en [2,2]"
- $= \alpha_3$ "Hay un wumpus en [1,3]»."

Por lo tanto, demuestre que BC $\models \alpha_2$ y BC $\models \alpha_3$.



(a)



(b)

ACTIVIDAD 2
Lógica proposicional y CNF

Considere la siguiente sentencia:

$$(\text{Comida} \Rightarrow \text{Fiesta}) \vee (\text{Bebidas} \Rightarrow \text{Fiesta}) \Rightarrow [(\text{Comida} \wedge \text{Bebidas}) \Rightarrow \text{Fiesta}]$$

a) Determine, mediante enumeración, si esta sentencia es válida, satisfactoria (pero no válida) o insatisfactoria.

b) Convierta ambos lados de la implicación principal en CNF, mostrando cada paso, y explique cómo los resultados confirman su respuesta a (a).

c) Demuestre su respuesta a (a) utilizando la resolución.

CC 1

ACTIVIDAD 2 | *Clave de corrección* CC 1

a) Una tabla de verdad simple tiene ocho filas y muestra que la sentencia es verdadera para todos los modelos y, por lo tanto, válida.

b) Para el lado izquierdo tenemos:

$$(Comida \Rightarrow Fiesta) \vee (Bebidas \Rightarrow Fiesta)$$
$$(\neg Comida \vee Fiesta) \vee (\neg Bebidas \vee Fiesta)$$
$$(\neg Comida \vee Fiesta \vee \neg Bebidas \vee Fiesta)$$
$$(\neg Comida \vee \neg Bebidas \vee Fiesta)$$

Y para el lado derecho tenemos:

$$(Comida \wedge Bebidas) \Rightarrow Fiesta$$
$$\neg(Comida \wedge Bebidas) \vee Fiesta$$
$$(\neg Comida \vee \neg Bebidas) \vee Fiesta$$
$$(\neg Comida \vee \neg Bebidas \vee Fiesta)$$

Las dos partes son idénticas en CNF, por lo que la frase original tiene la forma $P \Rightarrow P$, que es válida para cualquier P.

c) Para demostrar que la sentencia es válida, mostremos que su negación resulta insatisfactoria. Es decir, basta con negarla, convertirla a CNF y aplicar resolución hasta obtener una contradicción. Podemos utilizar el resultado en CNF ya obtenido para el lado izquierdo.

$$\neg[[(Comida \Rightarrow Fiesta) \vee (Bebidas \Rightarrow Fiesta)] \Rightarrow [(Comida \wedge Bebidas) \Rightarrow Fiesta]]$$

$$\neg Comida \Rightarrow Fiesta \vee Bebidas \Rightarrow Fiesta \Rightarrow Comida \wedge Bebidas \Rightarrow Fiesta$$

$$[(Comida \Rightarrow Fiesta) \vee (Bebidas \Rightarrow Fiesta)] \wedge \neg[(Comida \wedge Bebidas) \Rightarrow Fiesta]$$

$$Comida \Rightarrow Fiesta \vee Bebidas \Rightarrow Fiesta \wedge \neg(Comida \wedge Bebidas \Rightarrow Fiesta)$$

$$(\neg Comida \vee \neg Bebidas \vee Fiesta) \wedge Comida \wedge Bebidas \wedge \neg Fiesta$$

$$\neg Comida \vee \neg Bebidas \vee Fiesta \wedge Comida \wedge Bebidas \wedge \neg Fiesta$$

Cada una de las tres cláusulas unitarias se resuelve, una tras otra, contra la primera cláusula, lo que finalmente deja una cláusula vacía.

ACTIVIDAD 3
Lógica de primer orden

Consideremos un vocabulario con los siguientes símbolos:

- Ocupación(p,o): Predicado. La persona p tiene la ocupación o.
- Cliente(p_1,p_2): Predicado. La persona p_1 es cliente de la persona p_2.
- Jefe(p_1,p_2): Predicado. La persona p_1 es jefe de la persona p_2.
- Médico, Cirujano, Abogado, Ingeniero: Constantes que denotan ocupaciones.
- Emilia, Juan: Constantes que denotan personas.

Utilice estos símbolos para escribir las siguientes afirmaciones en lógica de primer orden:

a) Emilia es cirujana o abogada.
b) Juan es actor, pero también tiene otro trabajo.
c) Todos los cirujanos son médicos.
d) Juan no tiene abogado (es decir, no es cliente de ningún abogado).
e) Emilia tiene un jefe que es abogado.
f) Existe un abogado cuyos clientes son todos médicos.
g) Todos los cirujanos tienen abogado.

CC 1

ACTIVIDAD 3 | *Clave de corrección* CC 1

a) $O(E,C) \vee O(E,A)$
b) $O(J,I) \wedge \exists p\ p \neq I \wedge O(J,p)$
c) $\forall p\ O(p,C) \Rightarrow O(p,D)$
d) $\neg\ p\ C(J,p) \wedge O(p,A)$
e) $\exists p\ J(p,E) \wedge O(p,A)$
f) $\exists p\ O(p,A) \wedge \forall q\ C(q,p) \Rightarrow O(q,D)$
g) $\forall p\ O(p,C) \Rightarrow \exists q\ O(q,A) \wedge C(p,q)$

Módulo 5

Microobjetivos

- Comprender qué significa aprender a partir de datos para reconocer cómo los agentes mejoran su desempeño mediante ejemplos y retroalimentación.
- Analizar el espacio de hipótesis, el conjunto de entrenamiento y el conjunto de prueba para evaluar cómo los modelos buscan aproximar la función verdadera y generalizar a nuevos datos.
- Interpretar los conceptos de sesgo, varianza, subajuste y sobreajuste para valorar el equilibrio necesario que permita un buen rendimiento predictivo.
- Entender el funcionamiento del algoritmo k-NN y las métricas de distancia con el objetivo de saber analizar cómo este modelo no paramétrico realiza clasificación y regresión basándose en ejemplos cercanos.

Contenidos

INTRODUCCIÓN AL APRENDIZAJE AUTOMÁTICO



Fuente: https://www.freepik.es/

Contenidos

A lo largo de los módulos previos recorrimos los fundamentos clásicos de la Inteligencia Artificial: comenzamos definiendo qué es la IA y cuáles son sus objetivos, luego analizamos cómo perciben, razonan y actúan los agentes en distintos tipos de entornos. Más adelante estudiamos los métodos de búsqueda para resolver problemas y, finalmente, exploramos las formas simbólicas de representar conocimiento y realizar inferencias lógicas. Ese camino mostró cómo un agente puede actuar de manera racional cuando cuenta con reglas explícitas, modelos detallados del mundo o descripciones bien definidas del problema.

Sin embargo, muchos desafíos actuales de la IA, como reconocer imágenes, entender el lenguaje natural, anticipar comportamientos o hacer recomendaciones personalizadas no pueden resolverse solo mediante reglas fijas, búsquedas exhaustivas o conocimiento previamente codificado. En estos casos, los patrones relevantes no están escritos por diseñadores humanos, sino ocultos en grandes volúmenes de datos.

Por este motivo, surge la necesidad del aprendizaje automático que implica la capacidad de los agentes de mejorar su desempeño a partir de ejemplos. Este módulo introduce esa transición: pasamos de una IA que razona con estructuras y reglas predefinidas a una IA que aprende patrones a partir de la experiencia. Aquí nos enfocaremos en el aprendizaje supervisado, especialmente en las tareas de clasificación y regresión, que constituyen la base de la mayoría de los sistemas predictivos modernos. El propósito es comprender cómo, a partir de datos, una máquina puede construir funciones que permitan predecir, decidir y generalizar más allá de los casos observados.

¡Vamos!

Unidad 11: Fundamentos del aprendizaje supervisado

Decimos que un agente aprende cuando es capaz de mejorar su desempeño después de observar datos o experiencias, integrando información nueva para ajustar su conducta futura. Este aprendizaje puede ser simple, como registrar una lista de compras, o altamente sofisticado, como generar una teoría científica a partir de observaciones. Cuando quien aprende es un sistema informático, hablamos de aprendizaje automático (mayormente conocido como machine learning), donde el agente recibe datos, construye un modelo y utiliza ese modelo tanto como hipótesis sobre el mundo como mecanismo operativo para resolver problemas.

El aprendizaje automático no pretende reemplazar la programación tradicional o el razonamiento lógico, sino complementarlos. Se vuelve fundamental por dos motivos centrales: Primero, los diseñadores no pueden anticipar todas las situaciones futuras. Un robot que navega un entorno desconocido, un sistema que predice precios o una plataforma que recomienda productos deben adaptarse continuamente a nuevos escenarios. Aquí el aprendizaje permite incorporar regularidades que no fueron previstas explícitamente. En segundo lugar, en muchos casos, los humanos no sabemos formular las reglas que seguimos. Reconocer rostros, interpretar imágenes o identificar anomalías son tareas que hacemos de forma intuitiva, sin saber las reglas precisas detrás de nuestras decisiones. El aprendizaje automático permite que un modelo descubra esas regularidades directamente desde los datos.

Así, en este módulo nos centraremos en una de las formas de aprendizaje más extendidas y conceptualmente claras: *el aprendizaje supervisado.* Pero antes de llegar a él, es necesario presentar el panorama general de las formas de aprendizaje. El aprendizaje automático puede mejorar cualquier componente del diseño de un agente. De hecho, cada elemento que estudiamos en los módulos anteriores puede ser optimizado mediante técnicas de aprendizaje. Algunos ejemplos son:

- Reglas de acción: un agente puede aprender qué acción ejecutar ante ciertos estados del entorno, observando ejemplos o imitando a un experto.

- Interpretación de percepciones: a partir de miles de imágenes etiquetadas, un agente puede aprender a reconocer objetos o situaciones relevantes.

- Modelos de transición: un agente puede aprender los efectos de sus acciones observando cómo el entorno cambia cuando actúa.

- Funciones de utilidad o preferencias: al registrar respuestas de usuarios o consecuencias no deseadas, un agente puede ajustar su valoración de los estados.

- Información sobre la calidad de las acciones: en contextos donde se premian o penalizan decisiones, el agente puede aprender qué acciones llevan a mejores resultados.

- Generadores de problemas y críticos: un agente puede aprender a proponer nuevas estrategias de acción y a evaluarlas, afinando progresivamente sus criterios de exploración y revisión.

Esto puede verse, también, en múltiples situaciones. Un vehículo autónomo aprende a frenar observando el comportamiento de un conductor humano; un sistema de visión aprende a detectar autobuses a partir de imágenes etiquetadas; o, un modelo industrial aprende a prever fallos en la maquinaria analizando registros de sensores.



A diferencia del razonamiento lógico, donde las conclusiones válidas son necesariamente ciertas si las premisas lo son, el aprendizaje automático se basa en **inducción**. Inducir significa generalizar a partir de observaciones, por ejemplo, si el agente observó que el sol salió todos los días anteriores, induce que también saldrá mañana. A diferencia de la deducción, esta conclusión puede ser incorrecta lo que constituye una característica esencial del aprendizaje, sus predicciones son probabilísticas y no están garantizadas.

Entonces, a lo largo de este módulo nos concentraremos en problemas donde las entradas son una representación factorizada (un vector de atributos). Cuando la salida es un conjunto finito de valores (como *nublado/soleado/lluvioso* o *verdadero/falso*), el problema de aprendizaje se llama clasificación. Mientras que, cuando la salida es un número (como por ejemplo la temperatura de mañana) el problema de aprendizaje se conoce como regresión.

El aprendizaje automático puede clasificarse según el tipo de retroalimentación que recibe el agente durante su entrenamiento. Existen tres formas principales:

- Aprendizaje supervisado: el agente observa pares de entrada-salida y aprende una función que mapea unas con otras. Por ejemplo, las entradas podrían ser fotografías, cada una acompañada de una salida que diga "auto" o "peatón". A este tipo de salida se la denomina etiqueta. El agente aprende entonces una función que, cuando se le da una nueva imagen, predice la etiqueta apropiada.

- Aprendizaje no supervisado: el agente aprende patrones en la entrada sin ninguna retroalimentación explícita. La tarea de aprendizaje no supervisado más común es la agrupación, que consiste en detectar grupos de ejemplos de entrada potencialmente útiles. Por ejemplo, cuando se muestran millones de imágenes tomadas de Internet, un sistema de visión por computadora puede identificar un gran grupo de imágenes similares que un hablante de español llamaría "gatos".

- Aprendizaje por refuerzo: el agente aprende mediante recompensas y castigos a lo largo del tiempo. No se le dice qué acción es correcta, sino qué tan buena fue su secuencia de decisiones. Por ejemplo, al final de una partida de ajedrez, se le dice al agente que ha ganado (una recompensa) o perdido (un castigo). Depende del agente decidir cuáles de las acciones anteriores al refuerzo fueron las más responsables de ello, y alterar sus acciones para apuntar hacia más recompensas en el futuro.

Como ya se mencionó, en este módulo se pondrá el foco en el aprendizaje supervisado, la forma más estudiada y aquella que constituye la base de gran parte de las aplicaciones modernas, como el reconocimiento de imágenes, la clasificación de textos, la predicción de precios, la recomendación de contenidos y el diagnóstico médico, entre muchas otras.

Formalmente, la tarea del aprendizaje supervisado puede describirse de la siguiente ma-

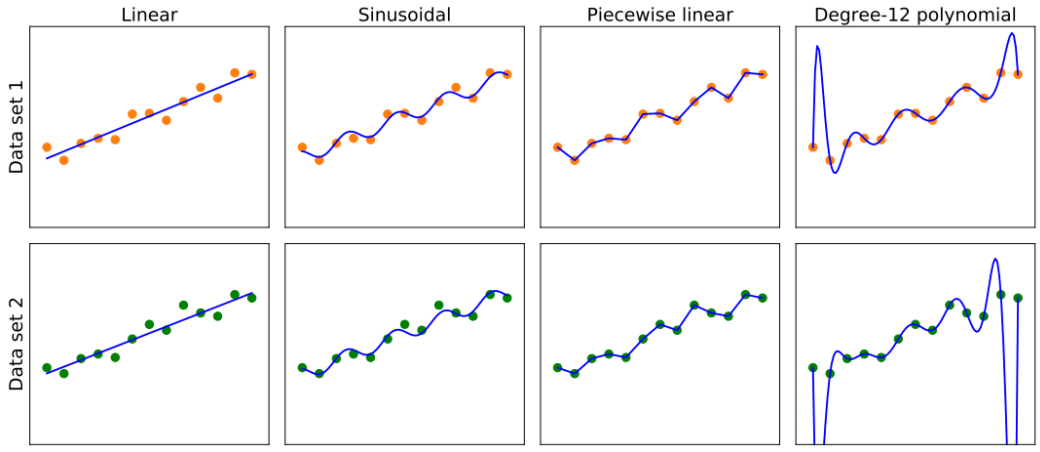
nera: dado un conjunto de entrenamiento con N pares de ejemplos de entrada y salida (x_1, y_1), (x_2, y_2), ..., (x , y), donde cada par fue generado por una función desconocida $y = f(x)$, el objetivo es descubrir una función h que se aproxime a la verdadera función f.

La función aprendida h recibe el nombre de hipótesis, y el conjunto de todas las funciones candidatas posibles constituye el espacio de hipótesis H. Llamamos a la salida y la verdad fundamental: es la respuesta correcta que le pedimos a nuestro modelo que prediga.

¿Cómo elegimos un espacio de hipótesis? En algunos casos, podemos contar con conocimiento previo sobre el proceso que generó los datos. Cuando esto no ocurre, es posible recurrir a un análisis exploratorio de datos, que consiste en examinar la información mediante pruebas estadísticas y visualizaciones como histogramas, diagramas de dispersión o diagramas de caja. Este análisis permite familiarizarse con los datos y obtener una primera intuición sobre qué espacio de hipótesis podría resultar adecuado. Otra alternativa es probar distintos espacios de hipótesis y evaluar empíricamente cuál ofrece mejores resultados.

¿Y Cómo elegimos una buena hipótesis dentro del espacio de hipótesis? Podríamos aspirar a una hipótesis coherente: una función h tal que, para cada x_i del conjunto de entrenamiento, se cumpla que $h(x_i) = y_i$. Sin embargo, cuando las salidas son de valor continuo, no podemos esperar una coincidencia exacta con la verdad fundamental; en su lugar, buscamos una función de mejor ajuste, para la cual cada $h(x_i)$ se encuentre lo más cerca posible de y_i.

La verdadera medida de una hipótesis no es cómo se desempeña en el conjunto de entrenamiento, sino qué tan bien maneja las entradas que aún no ha visto. Para evaluar esto, utilizamos una segunda muestra de pares (x_i, y_i), llamada conjunto de prueba. Decimos que una hipótesis h generaliza bien si predice con precisión las salidas correspondientes a este conjunto de prueba. Veamos un ejemplo en la figura siguiente, donde se muestra que una función h que descubre un algoritmo de aprendizaje depende del espacio de hipótesis H que considera y del conjunto de entrenamiento que se le proporciona.



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 654.

Cada una de las cuatro gráficas en la fila superior tiene el mismo conjunto de entrenamiento de 13 puntos de datos en el plano (*x*,*y*). Las cuatro gráficas en la fila inferior tienen un segundo conjunto de 13 puntos de datos; ambos conjuntos son representativos de la misma función desconocida *f(x)*. Cada columna muestra la hipótesis de mejor ajuste de un diferente espacio de hipótesis y podemos observar diferencias importantes:

- Los modelos lineales tienen una capacidad limitada para captar patrones de curvatura, poseen un fuerte sesgo hacia funciones rectas. No hay ninguna línea que sea una hipótesis coherente para los puntos de datos.

- Los modelos sinusoidales capturan variaciones suaves y, en el ejemplo, no son del todo coherentes, pero se ajustan muy bien a ambos conjuntos de datos.

- Los modelos lineales por tramos se ajustan exactamente a los datos, pero su forma depende fuertemente de la ubicación precisa de los puntos. Estas funciones son siempre coherentes.

- Los polinomios de grado 12 también ajustan perfectamente los datos, pero presentan oscilaciones extremas fuera del rango principal, evidenciando que un ajuste perfecto puede llevar a una mala generalización.

Una forma de analizar los espacios de hipótesis es a partir del sesgo que imponen, independientemente del conjunto de entrenamiento, y de la varianza que producen al pasar de un conjunto de entrenamiento a otro.

Por **sesgo** nos referimos a la tendencia de una hipótesis predictiva a desviarse del valor esperado cuando se promedia en diferentes conjuntos de entrenamiento. El sesgo a menudo resulta de restricciones impuestas por el espacio de hipótesis. Por ejemplo, el espacio de hipótesis de funciones lineales induce un fuerte sesgo, ya que solo permite funciones con forma de línea recta. Si en los datos existen patrones distintos de la tendencia general de una recta, una función lineal no podrá representarlos. En estos casos decimos que la hipótesis está subajustando, porque no logra capturar la estructura presente en los datos. En contraste, una función lineal por tramos presenta un sesgo bajo, dado que la forma de la función queda en gran medida determinada por los datos.

Por varianza entendemos la sensibilidad de un modelo ante pequeñas fluctuaciones en el conjunto de entrenamiento. Las dos filas de la figura analizada anteriormente representan conjuntos de datos que fueron muestreados a partir de la misma función *f(x)*, pero que resultaron ligeramente diferentes entre sí. En las primeras tres columnas, estas pequeñas diferencias en los datos generan solo variaciones menores en la hipótesis resultante, lo que caracteriza a un modelo de baja varianza. En cambio, los polinomios de grado 12 de la cuarta columna presentan una varianza alta: basta observar cuán distintas son las funciones obtenidas, especialmente en los extremos del eje x. Es evidente que, al menos uno de estos polinomios, constituye una mala aproximación de la verdadera *f(x)*. En estos casos decimos que el modelo está sobreajustando los datos, ya que presta demasiada atención al conjunto de datos concreto con el que se entrena, lo que deteriora su desempeño frente a datos nuevos o no observados.

Como vimos con los distintos modelos de ejemplo, aquellos que son más rígidos muestran alto sesgo y baja varianza, mientras que los más flexibles —como los polinomios de alto grado— muestran bajo sesgo, pero alta varianza. Este dilema entre simplicidad y flexibilidad se conoce como *compensación sesgo-varianza*. Un modelo que sea demasiado simple tendrá un sesgo alto y no podrá representar adecuadamente los datos, causando subajuste. Mientras que un modelo demasiado complejo tendrá una varianza alta y aprenderá los detalles accidentales del conjunto de entrenamiento, dificultando la generalización y causando sobreajuste.

Describamos ahora, en detalle, un ejemplo de problema de aprendizaje supervisado: decidir si conviene o no esperar una mesa en un restaurante. En este caso, la salida es una variable booleana que llamaremos *WillWait*, cuyo valor es verdadero en aquellos ejemplos en los que se decide esperar una mesa. La entrada consiste en un vector de diez atributos, cada uno de ellos con valores discretos, que describen las características relevantes de la situación:

1. Alternativa: indica si hay un restaurante alternativo adecuado cerca.
2. Bar: señala si el restaurante cuenta con una zona de bar cómoda para esperar.
3. Vie/Sab: verdadero los viernes y sábados.
4. Hambriento: si tenemos hambre ahora mismo.
5. Clientes: cantidad de personas presentes en el restaurante (los valores posibles son Ninguno, Algunos y Lleno).
6. Precio: el rango de precios del restaurante ($, $$, $$$).
7. Lloviendo: si está lloviendo afuera.
8. Reserva: si hicimos una reserva.
9. Tipo: el tipo de restaurante (francés, italiano, tailandés o hamburguesería).
10. Tiempo de espera estimado: estimación del tiempo de espera proporcionada por el anfitrión: 0–10, 10–30, 30–60 o más de 60 minutos.

Vemos un conjunto de 12 ejemplos:

Ex	Atributos de entrada										Salida
	Alt	Bar	Vie/Sab	Ham	Cli	Precio	Llov	Reserva	Tipo	Est	
x_1	Si	No	No	Si	Algunos	$$$	No	Si	Fran	0-10	y_1 = Si
x_2	Si	No	No	Si	Lleno	$	No	No	Tai	30-60	y_2 = No
x_3	No	Si	No	No	Algunos	$	No	No	Hamb	0-10	y_3 = Si
x_4	Si	No	Si	Si	Lleno	$	Si	No	Tai	10-30	y_4 = Si
x_5	Si	No	Si	No	Lleno	$$$	No	Si	Fran	>60	y_5 = No
x_6	No	Si	No	Si	Algunos	$$	Si	Si	Ita	0-10	y_6 = Si
x_7	No	Si	No	No	Ninguno	$	Si	No	Hamb	0-10	y_7 = No
x_8	No	No	No	Si	Algunos	$$	Si	Si	Tai	0-10	y_8 = Si
x_9	No	Si	Si	No	Lleno	$	Si	No	Hamb	>60	y_9 = No
x_{10}	Si	Si	Si	Si	Lleno	$$$	No	Si	Ita	10-30	y_{10} = No
x_{11}	No	No	No	No	Ninguno	$	No	No	Tai	0-10	y_{11} = No
x_{12}	Si	Si	Si	Si	Lleno	$	No	No	Hamb	30-60	Y_{12} = Si

Observemos lo escasos que son estos datos: existen $2^6 \times 3^2 \times 4^2 = 9216$ posibles combinaciones de valores para los atributos de entrada, pero solo se nos proporciona la salida correcta para 12 de ellas. Cada una de las 9204 restantes podría ser verdadera o falsa, y no lo sabemos. Esta es la esencia de la inducción: necesitamos formular nuestra mejor suposición acerca de esos 9 204 valores de salida faltantes, basándonos únicamente en la evidencia proporcionada por los 12 ejemplos disponibles.

Árboles de decisión

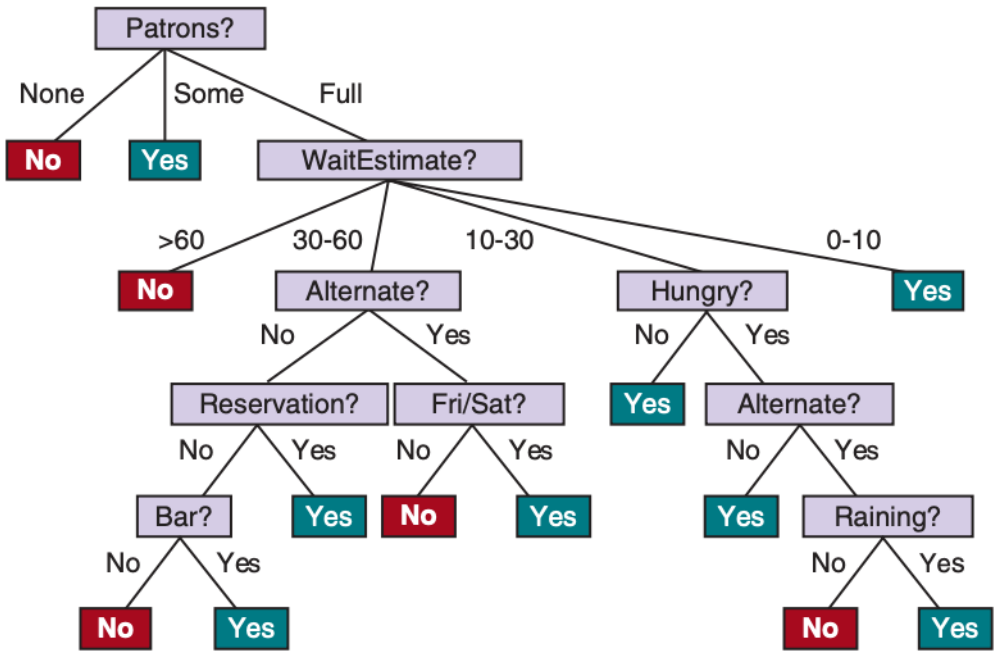
Énfasis

Los árboles de decisión constituyen uno de los métodos más intuitivos, comprensibles y ampliamente utilizados dentro del aprendizaje supervisado para construir algoritmos de aprendizaje. Son una representación de una función que mapea un vector de valores de atributos en un único valor de salida, es decir, una "decisión".

Un árbol de decisión alcanza su decisión realizando una secuencia de pruebas, comenzando en la raíz y siguiendo la rama apropiada hasta que se alcanza una hoja. Cada nodo interno en el árbol corresponde a una prueba del valor de uno de los atributos de entrada, las ramas desde el nodo están etiquetadas con los posibles valores del atributo y los nodos hoja especifican qué valor debe ser devuelto por la función.

En general, los valores de entrada y salida pueden ser discretos o continuos, pero en este módulo solo vamos a considerar entradas que consisten en valores discretos y salidas que son o *verdadero* (un ejemplo positivo) o *falso* (un ejemplo negativo). Llamamos a esto clasificación booleana. Vamos a usar j para indexar los ejemplos (x_j es el vector de entrada para el ejemplo j y y_j es la salida), y $x_{j,i}$ para el atributo i del ejemplo j.

El árbol que representa la función de decisión usada para el problema del restaurante sería el siguiente:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 658.

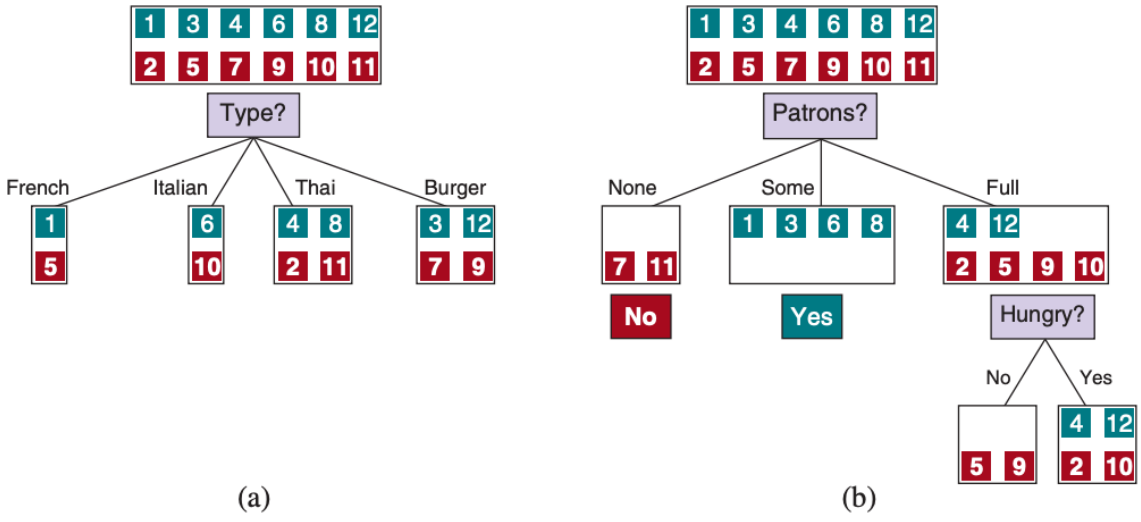
Siguiendo las ramas, vemos que un ejemplo con Clientes = Lleno y EsperaEstimada = 0-10 se clasificaría como positivo (si, se esperará por una mesa). En términos lógicos, un árbol de decisión representa una expresión en forma normal disyuntiva: cada camino desde la raíz hasta una hoja positiva equivale a una conjunción de pruebas, y el árbol completo es la disyunción de todos esos caminos. Esto significa que cualquier función en lógica proposicional puede ser expresada como un árbol de decisión.

Ahora queremos encontrar un árbol que sea consistente con los ejemplos en la tabla vista anteriormente y que sea, además, lo más pequeño posible. Desafortunadamente, resulta intratable garantizar la obtención del árbol consistente más pequeño. Sin embargo, mediante algunas heurísticas simples, es posible encontrar de manera eficiente un árbol que se aproxime bastante al tamaño mínimo.

El algoritmo APRENDER-ÁRBOL-DECISIÓN adopta una estrategia voraz de divide y vencerás: siempre prueba el atributo más importante primero y luego resuelve recursivamente los subproblemas más pequeños que están definidos por los posibles resultados de la prueba.

Por "atributo más importante" nos referimos al que hace la mayor diferencia en la clasificación de un ejemplo. De esa manera, esperamos obtener la clasificación correcta con un pequeño número de pruebas, lo que significa que todos los caminos en el árbol serán cortos y el árbol en su conjunto será poco profundo.

Veamos un ejemplo de cómo elegir el atributo más importante para el caso del restaurante.



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 659.

Como se ve en la figura, *Tipo* es un atributo pobre, porque nos deja con cuatro posibles resultados, cada uno de los cuales tiene el mismo número de ejemplos positivos que negativos. Por otro lado, en (b) vemos que *Clientes* es un atributo bastante importante, porque si el valor es *Ninguno* o *Algunos*, entonces nos quedamos con conjuntos de ejemplos para los cuales podemos responder definitivamente (No y Sí, respectivamente). Si el valor es

Lleno, nos quedamos con un conjunto mixto de ejemplos. Existen cuatro casos a considerar para estos subproblemas recursivos:

1. Si los ejemplos restantes son todos positivos (o todos negativos), entonces hemos terminado: podemos responder Sí o No. La Figura (b) muestra ejemplos de esto sucediendo en las ramas *Ninguno* y *Algunos*.

2. Si hay algunos ejemplos positivos y algunos negativos, entonces elegimos el mejor atributo para dividirlos. La Figura (b) muestra *Hambriento* siendo usado para dividir los ejemplos restantes.

3. Si no quedan ejemplos, significa que no se ha observado ningún ejemplo para esta combinación de valores de atributos y devolvemos el valor de salida más común del conjunto de ejemplos que se utilizaron para construir el nodo padre.

4. Si no quedan atributos, pero sí ejemplos positivos y negativos, significa que estos ejemplos tienen exactamente la misma descripción, pero diferentes clasificaciones. Esto puede suceder porque hay un error o ruido en los datos; porque el dominio es no determinista; o porque no podemos observar un atributo que distinguiría los ejemplos. Lo mejor que podemos hacer es devolver el valor de salida más común de los ejemplos restantes.

El algoritmo del que hablamos es el siguiente:

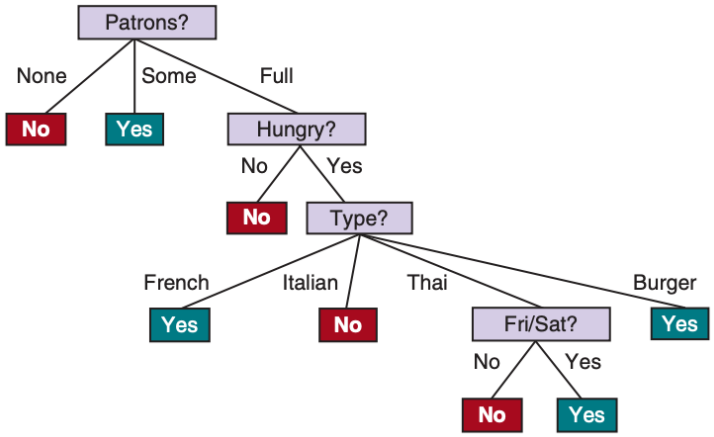
```
función APRENDER-ÁRBOL-DECISIÓN(ejemplos,atributos,ejemplos_padre) devolver árbol

    si ejemplos está vacío entonces devolver VALOR-MAYORÍA(ejemplos_padre)
    si no si todos ejemplos tienen la misma clasificación entonces devolver clasificación
    si no si atributos está vacío entonces devolver VALOR-MAYORÍA(ejemplos)
    si no
        A ← argmax_{a∈atributos} IMPORTANCIA(a,ejemplos)
        árbol ← un nuevo árbol de decisión con nodo raíz A
        para cada valor v de A hacer
            exs ← {e : e ∈ ejemplos y e.A = v}
            subárbol ← APRENDER-ÁRBOL-DECISIÓN(exs,atributos-A,ejemplos)
            añadir una rama a árbol con etiqueta (A = v) y subárbol subárbol
        devolver árbol
```

Fuente: elaboración propia.

Tengamos en cuenta que el conjunto de ejemplos constituye una entrada del algoritmo, pero en ningún momento los ejemplos aparecen explícitamente en el árbol que el algoritmo devuelve. Un árbol está compuesto por pruebas sobre atributos en los nodos interiores, valores de atributos en las ramas y valores de salida en los nodos hoja. La función VALOR-MAYORÍA selecciona el valor de salida más frecuente dentro de un conjunto de ejemplos y, en caso de empate, lo resuelve de manera aleatoria.

La salida del algoritmo de aprendizaje en nuestro conjunto de entrenamiento de muestra es el siguiente:

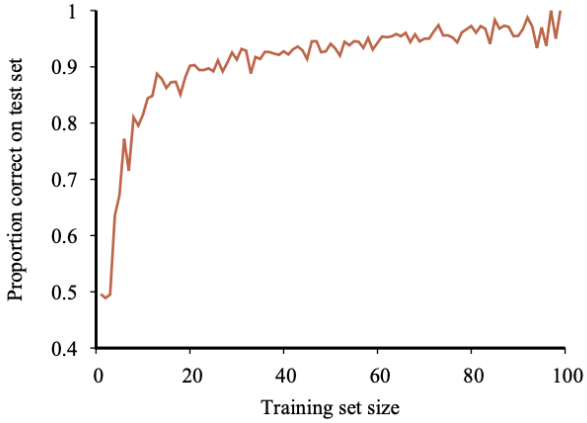


Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 660.

El árbol obtenido es claramente diferente del árbol original que vimos al comienzo. Uno podría concluir que el algoritmo de aprendizaje no está haciendo un muy buen trabajo de aprender la función correcta. Sin embargo, esta sería la conclusión equivocada. El algoritmo de aprendizaje no observa la función verdadera, sino únicamente los ejemplos y, de hecho, la hipótesis que construye no solo es consistente con todos ellos, sino que además resulta considerablemente más simple que el árbol original. Con un conjunto de ejemplos apenas distinto, el árbol aprendido podría variar de manera significativa, aunque la función que representa seguiría siendo esencialmente la misma.

El algoritmo de aprendizaje no tiene ninguna razón para incluir pruebas para *Lloviendo* y *Reserva*, porque puede clasificar todos los ejemplos sin utilizarlas. Además, ha detectado un patrón interesante y previamente insospechado: el cliente esperará comida tailandesa los fines de semana. Sin embargo, el algoritmo también está destinado a cometer algunos errores en situaciones para las que no ha observado ejemplos. Por ejemplo, nunca ha visto un caso donde la espera es de 0 a 10 minutos y el restaurante esté lleno. En esa situación, el modelo indica que no se debe esperar cuando Hambriento es falso, aunque en la práctica el cliente probablemente sí esperaría. Con un mayor número de ejemplos de entrenamiento, el programa de aprendizaje podría corregir este tipo de errores.

Podemos evaluar el rendimiento de un algoritmo de aprendizaje con una curva de aprendizaje como la siguiente:



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 37.

Para esta figura tenemos 100 ejemplos a nuestra disposición, que dividimos aleatoriamente en un conjunto de entrenamiento y un conjunto de prueba. Aprendemos una hipótesis con el conjunto de entrenamiento y medimos su precisión con el conjunto de prueba. Podemos hacer esto comenzando con un conjunto de entrenamiento de tamaño 1 y aumentando de uno en uno hasta el tamaño 99. Para cada tamaño, en realidad repetimos el proceso de dividir aleatoriamente en conjuntos de entrenamiento y prueba 20 veces, y promediar los resultados de los 20 ensayos. La curva muestra que a medida que aumenta el tamaño del conjunto de entrenamiento, la precisión aumenta. En este gráfico alcanzamos el 95% de precisión y parece que la curva podría seguir aumentando si tuviéramos más datos.

El algoritmo de aprendizaje del árbol de decisión elige el atributo con la importancia más alta. Veamos ahora cómo medir la importancia, utilizando la noción de ganancia de información, que se define en términos de entropía, que es la cantidad fundamental en la teoría de la información.

La entropía es una medida de la incertidumbre de una variable aleatoria; cuanta más información, menos entropía. Una variable aleatoria con un único valor posible, como una moneda que siempre sale cara, no presenta incertidumbre y, por lo tanto, su entropía se define como cero. En cambio, una moneda justa tiene la misma probabilidad de salir cara o cruz en cada lanzamiento, lo que corresponde a "1 bit" de entropía. El lanzamiento de un dado justo de cuatro caras tiene 2 bits de entropía, ya que existen 2^2 opciones igualmente probables. Ahora consideremos una moneda injusta que sale cara el 99% de las veces. Intuitivamente, esta moneda tiene menos incertidumbre que la moneda justa; si adivinamos cara, nos equivocaremos solo el 1% de las veces. Por ello, resulta razonable que su entropía sea cercana a cero, aunque estrictamente mayor que cero.

La ganancia de información de un atributo de prueba es la reducción esperada de la entropía. No entraremos en detalles de fórmulas en esta unidad ya que estos temas se verán más en profundidad en la materia Aprendizaje Automático del próximo cuatrimestre, pero quedémonos con la idea de que podemos diferenciar un mejor atributo de otro por cual tiene la mayor ganancia de información y, por ende, reduce la entropía.


Actividad

Realice a continuación la **actividad 1** correspondiente a este módulo para ejercitar sobre los árboles de decisión.

Para finalizar esta unidad, aprendimos los fundamentos del aprendizaje supervisado: qué significa aprender a partir de ejemplos, cómo se representan los datos, qué formas de aprendizaje existen y por qué nos concentramos en clasificación y regresión. Introdujimos un caso motivador, la decisión de esperar o no en un restaurante, y estudiamos un primer modelo: los árboles de decisión. A su vez, presentamos conceptos cruciales como generalización, sobreajuste, subajuste y validación.

Continuemos ahora con la próxima y última unidad de esta materia, en la que aprenderemos algunas técnicas básicas de clasificación y regresión.

Unidad 12: Técnicas de clasificación y regresión

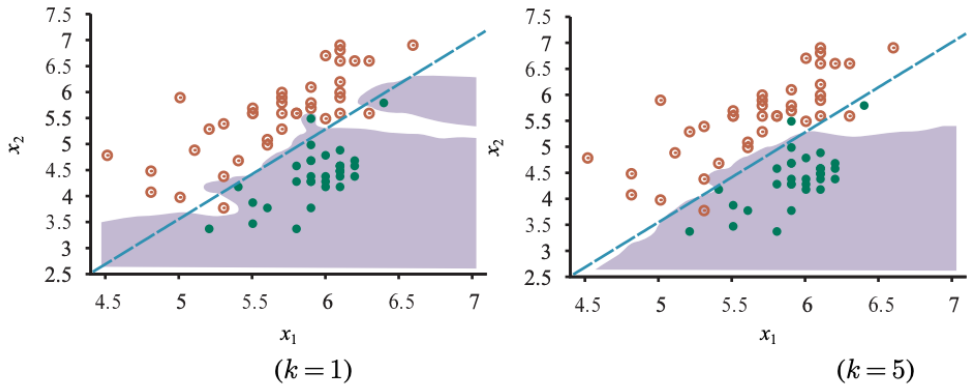
En esta unidad abordaremos dos tareas fundamentales del aprendizaje supervisado: la clasificación y la regresión. Ambas comparten la misma lógica general, a partir de ejemplos previamente etiquetados, se busca predecir la salida correspondiente a nuevas instancias. Para introducir estas técnicas utilizaremos un único modelo, simple en su formulación pero poderoso desde el punto de vista conceptual, el algoritmo de los k-vecinos más cercanos (k-Nearest Neighbors, o k-NN).

Este modelo pertenece a la familia de los modelos no paramétricos, llamados así porque no resumen los datos en un conjunto fijo de parámetros. En lugar de aprender una función general, conservan todos los ejemplos de entrenamiento y utilizan directamente esa información para realizar predicciones. Esta característica permite que el modelo sea extremadamente flexible, ya que no impone una forma predefinida a la relación entre las entradas y las salidas: las decisiones siempre se toman observando el comportamiento local de los datos.

La búsqueda de los k-vecinos más cercanos puede entenderse como: dada la consulta x_q, en lugar de encontrar un ejemplo que sea igual a x_q, buscar los *k* ejemplos que están más cerca de x_q. Usaremos la notación NN(*k*,x_q) para denotar el conjunto de *k* vecinos más cercanos a x_q.

Para realizar una clasificación, buscamos el conjunto de vecinos NN(*k*,x_q) y tomamos el valor de salida más común; por ejemplo, si k=3 y los valores de salida son $\langle Sí, No, Sí \rangle$, entonces, la clasificación será *Sí*. Para evitar empates en la clasificación binaria, suele elegirse un valor impar para *k*. En el caso de la regresión, podemos tomar la media o la mediana de los *k* vecinos, o bien podemos resolver un problema de regresión lineal sobre ellos. El modelo de función lineal por tramos que vimos en la unidad anterior resuelve un problema de regresión lineal con los dos puntos que están a la derecha y a la izquierda de x_q. Cuando los puntos x_i están espaciados equitativamente, estos corresponden a los dos vecinos más cercanos.

Veamos un ejemplo del límite de decisión en la clasificación de los k vecinos más cercanos para k = 1 y k = 5. El conjunto de datos se representa en un gráfico con dos parámetros de datos sísmicos, la magnitud de la onda corporal x_2 y la magnitud de la onda superficial x_1, correspondientes a terremotos (círculos naranjas abiertos) y explosiones nucleares (círculos verdes) ocurridos entre 1982 y 1990 en Asia y Oriente Medio.



Fuente:RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 687.

Un límite de decisión es una línea (o una superficie, cuando el espacio tiene más dimensiones) que divide el espacio de características en regiones, de modo que todos los puntos de una misma región son clasificados en la misma clase por el algoritmo.

Los métodos no paramétricos están igualmente sujetos al subajuste y al sobreajuste, al igual que los métodos paramétricos. En este caso, el método del vecino más cercano con k = 1 presenta sobreajuste, ya que reacciona de manera excesiva ante el valor atípico negro ubicado en la parte superior derecha y ante el valor atípico blanco en (5,4; 3,7). En cambio, el límite de decisión obtenido con k = 5 resulta adecuado, mientras que valores más altos de k conducirían a un subajuste. Para seleccionar el mejor valor de k puede utilizarse un concepto llamado validación cruzada, que se verá en profundidad en la materia Aprendizaje Automático del semestre siguiente.

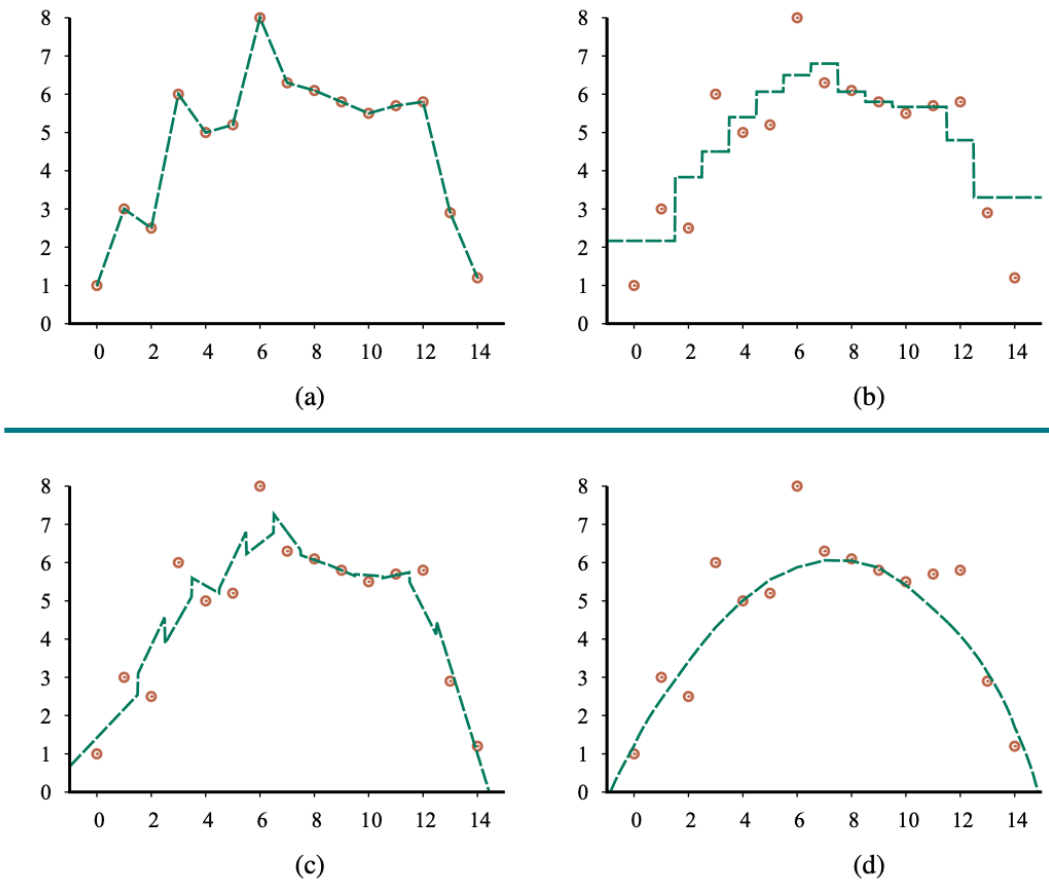
El término «más cercano» presupone la existencia de una métrica de distancia. La pregunta central es cómo medir la distancia entre un punto de consulta x_q y un punto de ejemplo x . Habitualmente, las distancias se miden utilizando la distancia de Minkowski, también llamada norma L , que se define como:

$$L^p(x_j, x_q) = (\sum_i |x_{j,i} - x_{q,i}|^p)^{1/p}$$

Con $p = 2$ tenemos lo que se conoce como distancia euclidiana y con $p = 1$ la distancia Manhattan. En atributos booleanos se utiliza la distancia de Hamming, que cuenta en cuántas posiciones difieren dos vectores.

A menudo se utiliza la distancia euclidiana cuando las dimensiones miden propiedades similares, como la anchura, la altura y la profundidad de los objetos. En cambio, la distancia Manhattan suele utilizarse cuando las dimensiones corresponden a magnitudes de naturaleza diferente, como la edad, el peso y el sexo de un paciente. Es importante tener en cuenta que, si se utilizan los valores numéricos sin procesar de cada dimensión, la distancia total se verá afectada por cualquier cambio en las unidades de medida. Por ejemplo, si la dimensión de la altura se expresa en millas en lugar de metros, mientras se mantienen constantes las dimensiones de anchura y profundidad, se obtendrán vecinos más cercanos distintos. Esto plantea el problema de cómo comparar, de manera significativa, una diferencia de edad con una diferencia de peso. Un enfoque habitual consiste en aplicar una normalización a las mediciones de cada dimensión. Para ello, se calcula la media μ y la desviación estándar de los valores correspondientes, y se reescalan los datos de modo que cada x , se transforme en $(x_{j,i} - \mu_i) / \sigma_i$.

Veamos ahora un enfoque no paramétrico de regresión en lugar de clasificación.



Fuente: RUSSELL, Stuart y NORVIG, Peter: Artificial Intelligence: A Modern Approach. Hoboken, Pearson, 2021. pp 690.

En (a), tenemos quizás el método más sencillo de todos, conocido informalmente como «unir los puntos» y, de manera más sofisticada, como «regresión no paramétrica lineal por tramos». Este modelo crea una función *h(x)* que, cuando se le proporciona una consulta x_q, considera los ejemplos de entrenamiento inmediatamente a la izquierda y a la derecha de x_q e interpola entre ellos. Cuando el ruido es bajo, este método trivial funciona razonablemente bien, por lo que se encuentra como característica estándar en el software de gráficos de hojas de cálculo. Sin embargo, cuando los datos presentan mucho ruido, la función resultante es irregular y no generaliza correctamente.

La regresión de *k*-vecinos más cercanos mejora el método de unir los puntos. En lugar de usar únicamente los dos ejemplos contiguos a un punto de consulta x_q, utilizamos los *k*-vecinos más cercanos (aquí utilizamos $k = 3$). Un valor mayor de *k* tiende a suavizar la magnitud de los picos, aunque la función resultante presenta discontinuidades. La figura más arriba muestra dos versiones de la regresión de los k-vecinos más cercanos. En (b), tenemos la media de los *k* vecinos más cercanos: *h(x)* es el valor medio de los *k* puntos, $\Sigma\, y_j/k$. Observemos que, en los puntos periféricos, cerca de $x = 0$ y $x = 14$, las estimaciones son deficientes porque toda la evidencia proviene de un lado (el interior) e ignora la tendencia. En (c), tenemos la regresión lineal *k*-vecinos más cercanos, que encuentra la mejor línea a través de los *k* ejemplos. Esto permite capturar mejor las tendencias, incluso en presencia de valores atípicos, pero la función sigue siendo discontinua. Tanto en (b) como en (c) surge la cuestión de cómo elegir un buen valor de k; como dijimos, la respuesta es la validación cruzada, que se estudiará más adelante.

La regresión ponderada localmente (figura (d)) nos ofrece las ventajas de los vecinos más cercanos, pero sin las discontinuidades. Para evitar estas discontinuidades en $h(x)$, es necesario que el conjunto de ejemplos utilizados para estimarla tampoco genere discontinuidades. La idea de la regresión ponderada localmente es que, en cada punto de consulta x_q, los ejemplos que están cerca de x_q se ponderan con mayor peso, los que se encuentran un poco más alejados reciben menor peso, y los más distantes no se ponderan en absoluto. La disminución del peso con la distancia suele ser gradual, no repentina.

El k-NN es una herramienta fundamental para introducir clasificación y regresión desde una perspectiva no paramétrica. Su simplicidad conceptual permite centrarse en los principios esenciales del aprendizaje supervisado: semejanza, proximidad, representación del espacio de atributos, generalización y sensibilidad al ruido. A través de este modelo, comprendemos que las predicciones pueden surgir de relaciones locales entre los datos, sin necesidad de imponer una forma funcional rígida. Esto establece una base para avanzar hacia modelos más complejos y paramétricos de aprendizaje automático en materias posteriores.



Actividad

Lo/a invito a realizar la actividad 2 de este módulo para profundizar sobre el algoritmo k-NN.

Con este último apartado hemos llegado al final de la materia Fundamentos de Inteligencia Artificial. A lo largo de esta asignatura recorrimos los pilares esenciales de la inteligencia artificial, desde sus definiciones y fundamentos teóricos hasta los métodos contemporáneos de aprendizaje. Comenzamos explorando los orígenes de la disciplina, sus enfoques clásicos y la evolución histórica que llevó de los primeros sistemas simbólicos a los agentes racionales capaces de percibir, razonar y actuar en entornos complejos.

Luego analizamos cómo los agentes toman decisiones a través de la búsqueda y la planificación, cómo representan el conocimiento mediante lenguajes lógicos y cómo infieren nuevas conclusiones. Finalmente, introdujimos el aprendizaje automático como la culminación de este recorrido: una inteligencia capaz de mejorar su comportamiento a partir de la experiencia y de los datos.

Este trayecto permitió comprender que la inteligencia artificial no es un único método, sino un campo interdisciplinario que combina razonamiento, percepción, aprendizaje y acción. Los conceptos adquiridos, como racionalidad, representación, búsqueda, inferencia y aprendizaje, constituyen los cimientos sobre los cuales se construyen las tecnologías actuales y futuras de la IA.



Evaluación

Usted ya está en condiciones de realizar la Parte 2 de la Evaluación Integradora.

Los invitamos a ver el siguiente video a modo de cierre

Módulo 5 | Glosario

Aprendizaje automático (Machine Learning): capacidad de un sistema para mejorar su desempeño mediante datos o experiencias, construyendo un modelo que le permite resolver problemas y operar como hipótesis del mundo.

Aprendizaje no supervisado: método en el cual el sistema aprende patrones en los datos sin retroalimentación explícita, por ejemplo, agrupando ejemplos similares.

Aprendizaje por refuerzo: tipo de aprendizaje basado en recompensas y castigos, donde el agente ajusta su comportamiento a partir del resultado de sus decisiones en el tiempo.

Aprendizaje supervisado: forma de aprendizaje en la que el agente observa pares entrada–salida y aprende una función que mapea $x \rightarrow y$, utilizando etiquetas proporcionadas por un experto o conjunto de datos.

Árbol de decisión: modelo de aprendizaje supervisado que clasifica un ejemplo realizando una secuencia de pruebas sobre atributos, siguiendo ramas hasta llegar a una hoja con la decisión final.

Atributo: valor o característica que compone la representación de cada ejemplo dentro del vector de entrada en un problema supervisado.

Clasificación: tarea del aprendizaje supervisado donde la salida es una categoría discreta, como verdadero/falso o selección entre un conjunto finito de clases.

Conjunto de entrenamiento: colección de ejemplos con entrada y salida conocidas usada para aprender un modelo. Cada hipótesis se ajusta primero sobre este conjunto.

Conjunto de prueba: ejemplos no vistos durante el entrenamiento y utilizados para evaluar si la hipótesis aprendida generaliza correctamente a nuevos casos.

Entropía: medida de incertidumbre asociada a una variable aleatoria, utilizada para evaluar la importancia de los atributos en los árboles de decisión y calcular ganancia de información.

Espacio de hipótesis (H): conjunto de todas las funciones candidatas que el algoritmo podría aprender a partir de los datos, dependiendo del tipo de modelo elegido (lineales, sinusoidales, polinomiales, etc.).

Ganancia de información: medida que indica cuánta reducción de entropía aporta un atributo al dividir los datos, utilizada para elegir el atributo más importante en un árbol de decisión.

Generalización: capacidad de una hipótesis aprendida para predecir correctamente salidas de ejemplos que no fueron vistos durante el entrenamiento.

Hipótesis (h): función aprendida por el modelo que intenta aproximar la verdadera rela-

ción f(x). Se define a partir del espacio de hipótesis seleccionado y los datos disponibles.

k-vecinos más cercanos (k-NN): algoritmo no paramétrico que clasifica o predice valores basándose en los k ejemplos más cercanos a la instancia de consulta según una métrica de distancia.

Límite de decisión: superficie o frontera que separa dos o más clases en el espacio de atributos, observable en gráficos de clasificación k-NN.

Matriz o métrica de distancia: Función que mide qué tan similares o distintos son dos ejemplos, clave para k-NN.

Modelo no paramétrico: modelo que no resume los datos en un número fijo de parámetros, sino que utiliza los ejemplos directamente para predecir, como k-NN.

Regresión: tarea en la que se predice un valor numérico a partir de atributos de entrada, por ejemplo, la temperatura o una variable continua.

Sesgo: tendencia de un modelo a no captar patrones presentes en los datos debido a restricciones del espacio de hipótesis. Se relaciona con subajuste.

Sobreajuste (overfitting): fenómeno donde el modelo se ajusta demasiado al conjunto de entrenamiento, perdiendo capacidad de generalización.

Subajuste (underfitting): cuando el modelo es demasiado simple para representar adecuadamente los datos; asociado a modelos con alto sesgo.

Varianza: sensibilidad del modelo a pequeñas variaciones en los datos; una varianza alta suele producir sobreajuste.

Módulo 5 | Actividades

ACTIVIDAD 1
Diseñando un árbol de decisión

Utilizando un agente de su elección, defina un problema de aprendizaje supervisado similar al presentado en el módulo sobre decidir si esperar o no una mesa en un restaurante. A continuación, genere un pequeño conjunto de datos de entre seis y diez ejemplos que reflejen distintas situaciones posibles para que el agente pueda aprender a tomar la decisión. Luego:

a) Intente construir de manera manual el primer y segundo nivel de árbol de decisión que clasifique estos datos.

b) Consulte nuevamente al agente cómo construiría él el mismo árbol.

c) Compare ambas soluciones e identifique:

   a) Diferencias en la elección del atributo raíz.
   b) Supuestos realizados por el agente.
   c) Posibles errores del agente si los hubiera.

d) Concluya qué tan confiable es la explicación del agente para este tipo de tareas.

ACTIVIDAD 2
k-NN en diferentes dominios

Investigue y seleccione tres dominios distintos donde se utilice k-NN y por cada uno de ellos indique lo siguiente:

a) ¿Cómo se representan las instancias (atributos utilizados)?

b) ¿Qué métrica de distancia se utiliza y por qué?

c) ¿Qué dificultades presenta k-NN en ese contexto (ruido, escala, dimensionalidad, densidad)?

d) ¿Qué técnicas suelen utilizarse para mejorar su desempeño (normalización, reducción de dimensionalidad, ponderación de vecinos, etc.)?